Algoritmi Distribuiti

Riassunto Completo per l'Esame Orale

Prof.ssa Montangero

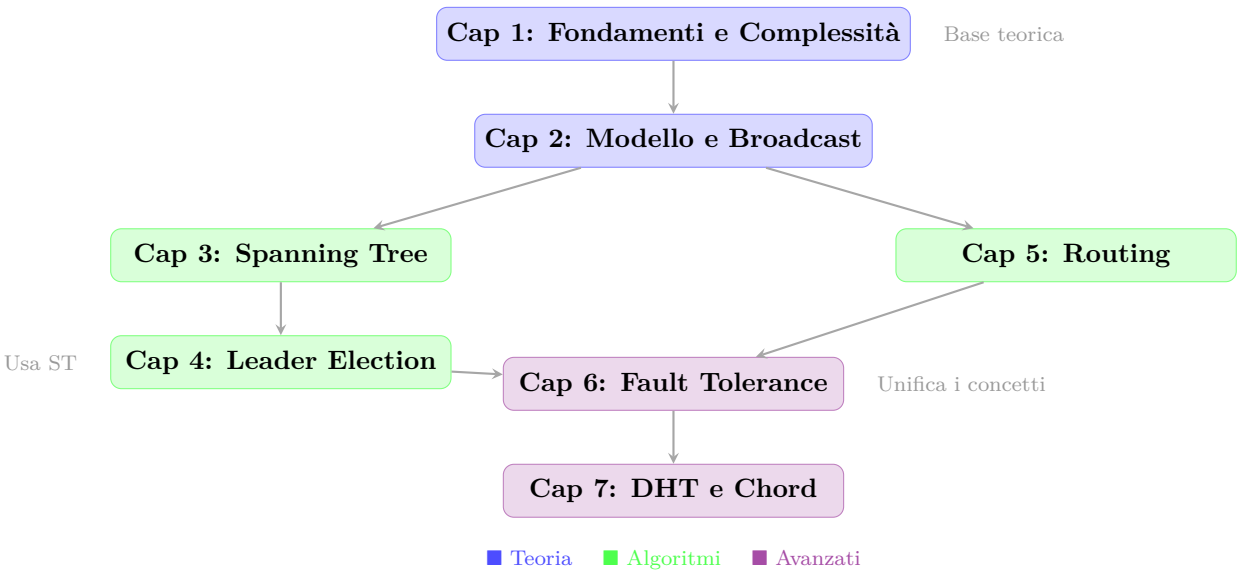
Riassunto a cura di: Francesco Caligiuri

Anno Accademico 2025/26

Questo documento contiene un riassunto completo del corso di Algoritmi Distribuiti, organizzato per argomenti e ottimizzato per la preparazione all'esame orale. Include definizioni formali, protocolli, analisi di complessità e domande tipiche d'esame.

Come Usare Questo Documento

Questo documento è organizzato per accompagnarti **da zero a superare l'esame**. Ecco la mappa dei contenuti:



Simbolo	Significato
Definizione	Formale definizione di un concetto
Teorema	Enunciato matematico o risultato teorico
Esempio	Applicazione pratica o caso d'uso
Nota	Approfondimento o chiarimento utile
Attenzione	Avviso su sottigliezze o trappole
Importante	Concetto critico per l'esame
Domanda Killer	Domanda frequente all'esame orale
Prerequisiti	Conoscenze richieste prima di iniziare
Key Takeaways	I punti fondamentali da portare a casa

Indice

1 Modello dei Sistemi Distribuiti, Broadcast e Wake-up **4**
1.1 Cos'è un Sistema Distribuito? . . . 4
1.1.1 L'intuizione . . . 4
1.1.2 Parallelo vs Distribuito: Una Distinzione Importante . . . 4
1.2 La Rete di Comunicazione . . . 5
1.2.1 Il modello a grafo . . . 5
1.3 Il Modello Computazionale: Come Funziona un'Entità . . . 5
1.3.1 Anatomia di un'entità . . . 5
1.3.2 Gli Eventi: Cosa Può Succedere . . . 5
1.3.3 Le Regole di Comportamento . . . 6
1.4 Gli Assiomi: Cosa Assumiamo Sempre Vero . . . 6
1.4.1 Assioma 1: Consegna Garantita . . . 6
1.4.2 Assioma 2: Orientamento Locale . . . 7
1.5 Misure di Complessità nei Sistemi Distribuiti . . . 7
1.5.1 Complessità dei Messaggi (Comunicazione) . . . 7
1.5.2 Complessità Temporale (Tempo Ideale) . . . 8
1.6 Il Problema del Broadcast . . . 8
1.6.1 Definizione del Problema . . . 8
1.6.2 Prima Idea (Sbagliata!) . . . 8
1.6.3 Soluzione Corretta: Protocollo Flooding . . . 9
1.6.4 Visual Walkthrough: Flooding in Azione . . . 10
1.6.5 Analisi della Complessità del Flooding . . . 11
1.6.6 Flooding è Ottimale? . . . 11
1.6.7 SimpleBroadcast: Ottimizzazione per Grafi Completi . . . 12
1.6.8 Message Complexity vs Bit Complexity . . . 12
1.6.9 Lower Bound: Dimostrazioni Formali . . . 13
1.7 Wake-up Problem: Una Generalizzazione . . . 14
1.7.1 Definizione . . . 14
1.7.2 Analisi della Complessità dei Messaggi (WFlood) . . . 14
1.7.3 Complessità Temporale . . . 14
1.8 Verso la Soluzione Ottimale . . . 14
1.9 Riepilogo del Capitolo . . . 15
1.10 Esercizi e Domande . . . 15
1.10.1 Domande Concettuali . . . 15
1.10.2 Esercizi di Dimostrazione . . . 15

2 Spanning Tree Construction e Computation in Trees **17**
2.1 Cos'è uno Spanning Tree e Perché ci Serve . . . 17
2.1.1 Definizione Intuitiva . . . 17
2.1.2 Perché lo Spanning Tree è Fondamentale . . . 17
2.2 Protocollo SHOUT: L'Idea Base . . . 18
2.2.1 L'Intuizione . . . 18
2.2.2 I Tre Tipi di Messaggi . . . 18
2.2.3 Come Funziona SHOUT . . . 18
2.2.4 Quali Archi Finiscono nell'Albero? . . . 19
2.2.5 Visual Walkthrough: SHOUT in Azione . . . 20
2.2.6 Analisi dei Messaggi di SHOUT: Da dove esce quel calcolo? . . . 20
2.3 Protocollo SHOUT+: L'Ottimizzazione . . . 21
2.3.1 L'Idea Geniale . . . 21

2.3.2 Complessità di SHOUT+ . . . 21
2.3.3 Gestione di Iniziatori Multipli . . . 21
2.4 Costruzione via DFS Traversal . . . 22
2.4.1 Un Approccio Diverso . . . 22
2.4.2 I Tre Tipi di Token . . . 22
2.4.3 Come Funziona il DFS (L'Esploratore Solitario) . . . 22
2.4.4 Complessità del DFS . . . 24
2.4.5 Ottimizzazione del Tempo (ACK) . . . 24
2.5 BFS Spanning Tree . . . 24
2.5.1 Confronto Visivo: SHOUT vs DFS . . . 24
2.6 La Tecnica della Saturation . . . 25
2.6.1 Computazione su Alberi . . . 25
2.6.2 L'Idea della Saturation: "L'Incendio Controllato" . . . 25
2.6.3 Visual Walkthrough: La Saturazione . . . 25
2.6.4 Trace Table: Esecuzione Saturation . . . 26
2.6.5 Complessità della Saturation . . . 26
2.7 Computation in Rooted Trees e Convergecast . . . 26
2.7.1 Alberi Radicati: Senso di Direzione . . . 26
2.7.2 Broadcast e Convergecast . . . 27
2.7.3 Come Funziona il Convergecast . . . 27
2.8 Riepilogo e Confronto . . . 27
2.9 Esercizi e Domande . . . 28
2.9.1 Domande Concettuali . . . 28
2.9.2 Esercizi di Dimostrazione . . . 28

3 Leader Election **30**
3.1 Il Problema: Rompere la Simmetria . . . 30
3.1.1 Perché Serve un Leader? . . . 30
3.1.2 Il Problema Fondamentale . . . 30
3.1.3 Il Setup: Anello Anonimo . . . 31
3.1.4 Perché è Impossibile? (La Logica dei Cloni) . . . 31
3.2 Leader Election su Anello con ID . . . 32
3.2.1 La Strategia Generale . . . 32
3.2.2 Protocollo All-the-Way (Le Lann, 1977) . . . 32
3.2.3 Visual Walkthrough: All-the-Way in Azione . . . 33
3.2.4 Protocollo "As Far As It Can" (AFAIC/LCR) . . . 34
3.2.5 Trace Table: AFAIC Esecuzione . . . 35
3.2.6 Analisi di AFAIC . . . 36
3.3 Protocollo Controlled Distance (Hirschberg-Sinclair) . . . 36
3.3.1 Il Problema di AFAIC . . . 36
3.3.2 L'Idea: Fasi di Distanza Crescente . . . 36
3.3.3 Perché Funziona? . . . 37
3.3.4 Analisi Approfondita: Il "Budget Costante" (Perché $O(n \log n)$?) . . . 38
3.3.5 Lower Bound $\Omega(n \log n)$ per Ring . . . 38
3.3.6 Visual Walkthrough: Controlled Distance in Azione . . . 40
3.4 Protocollo FloodMax (Sincrono) . . . 41
3.5 Leader Election su Grafi Generici . . . 41
3.5.1 L'Approccio: Spanning Tree + Saturation . . . 41
3.5.2 Altri Protocolli: Mega-Merger . . . 42
3.6 Sistemi Sincroni: Il Potere del Silenzio . . . 42
3.6.1 Il Silenzio è Informazione . . . 42
3.6.2 Protocollo "Universal Waiting" (Time-Slice) . . . 42

3.6.3 Protocollo Speeding . . . 43
3.7 Algoritmi Randomizzati: Rompere la Simmetria senza ID . . . 43
3.7.1 L'Idea . . . 43
3.7.2 Due Tipi di Algoritmi Randomizzati . . . 43
3.7.3 Un Protocollo Las Vegas . . . 43
3.7.4 Ottimizzazione: Probabilità $p = 1/n$ . . . 44
3.8 Riepilogo: Quale Algoritmo Usare? . . . 44
3.9 Esercizi e Domande . . . 45
3.9.1 Domande Concettuali . . . 45
3.9.2 Esercizi di Dimostrazione . . . 45

4 Routing Distribuito 47
4.1 Il Problema: Come Arriviamo a Destinazione? . . . 47
4.1.1 Cosa Significa "Routing"? . . . 47
4.1.2 Due Approcci al Routing . . . 47
4.1.3 La Routing Table . . . 48
4.2 Il Principio di Ottimalità: Perché Basta il Next Hop . . . 48
4.2.1 Il Teorema di Bellman . . . 48
4.2.2 L'Implicazione Pratica . . . 48
4.3 Map Gossip vs Bellman-Ford: Due Filosofie a Confronto . . . 48
4.4 Approccio 1: Map Gossip . . . 49
4.4.1 L'Idea: Condividiamo Tutto . . . 49
4.4.2 Complessità di Map Gossip . . . 49
4.4.3 Pro e Contro . . . 49
4.5 Approccio 2: Bellman-Ford Distribuito . . . 49
4.5.1 L'Idea: Condividiamo Solo le Distanze . . . 49
4.5.2 L'Equazione di Bellman-Ford . . . 50
4.5.3 Il Protocollo . . . 50
4.5.4 Esempio Passo-Passo . . . 50
4.5.5 Visual Walkthrough: Convergenza Bellman-Ford . . . 51
4.5.6 Complessità di Bellman-Ford . . . 51
4.6 Min-Hop Routing (BFS Distribuita) . . . 51
4.6.1 L'Algoritmo Distribuito . . . 52
4.6.2 Complessità Min-Hop . . . 52
4.7 Distributed Dijkstra (Shortest Path Spanning Tree) . . . 52
4.7.1 L'Idea . . . 52
4.7.2 Complessità . . . 53
4.8 Il Problema del Count-to-Infinity (Approfondimento) . . . 53
4.8.1 Cosa Succede Quando le Cose Vanno Male? . . . 53
4.8.2 Il Count-to-Infinity: Il Problema dell'Eco . . . 53
4.8.3 Esempio del Problema . . . 54
4.8.4 Visual Walkthrough: Il Bounce Disastroso . . . 54
4.8.5 Trace Table: Count-to-Infinity . . . 54
4.8.6 Soluzioni al Count-to-Infinity . . . 55
4.9 Gestione dei Cambiamenti (Dinamicità) . . . 56
4.9.1 Map Update (Link State) . . . 56
4.9.2 Distance Vector Update . . . 57
4.10 Riepilogo . . . 57
4.11 Esercizi e Domande . . . 58
4.11.1 Domande Concettuali . . . 58
4.11.2 Esercizi di Dimostrazione . . . 58

5 Fault Tolerance e Consenso Distribuito **60**
5.1 Il Modello dei Guasti: Definizioni e Classificazioni . . . 60
5.1.1 Classificazione dei Guasti . . . 60
5.1.2 Tipi di Fallimenti delle Entità . . . 61
5.1.3 Tipi di Fallimenti dei Link . . . 61
5.2 Fattori Topologici: Connettività e Robustezza . . . 61
5.2.1 Gerarchia Visuale dei Fallimenti . . . 61
5.2.2 Tipi di Fallimenti dei Link . . . 62
5.3 Fattori Topologici: Quanto Robusta Deve Essere la Rete? . . . 62
5.3.1 Connettività delle Topologie Comuni . . . 63
5.4 Il Problema dell'Accordo (Agreement) . . . 63
5.5 Il Problema dei Due Generali: Quando è Impossibile . . . 63
5.5.1 Lo Scenario . . . 63
5.5.2 Perché è Impossibile . . . 64
5.6 Il Problema del Consenso . . . 65
5.6.1 Definizione Formale . . . 65
5.6.2 Perché è Fondamentale? . . . 65
5.7 Consenso con Fallimenti di Link . . . 66
5.7.1 Protocollo TwoSteps (per grafi completi) . . . 66
5.7.2 Leader Election: Protocollo FT-BroadElection . . . 66
5.7.3 Leader Election: Protocollo FT-BroadElection . . . 66
5.8 Consenso con Crash: Il Teorema FLP . . . 66
5.8.1 L'Impossibilità nel Caso Asincrono (FLP) . . . 66
5.8.2 Dimostrazione Formale di FLP . . . 67
5.8.3 Come Aggirare FLP: Sfruttare il Tempo . . . 68
5.9 TellAll_Crash: Consenso Sincrono con Crash . . . 69
5.9.1 Perché servono $f + 1$ round? (Spiegazione Narrativa) . . . 69
5.9.2 Ottimizzazione dei Messaggi . . . 69
5.10 Ben-Or: Consenso Randomizzato Asincrono . . . 70
5.10.1 L'Idea Chiave . . . 70
5.10.2 Gestione dei Messaggi e Round . . . 70
5.10.3 Il Protocollo . . . 70
5.10.4 Perché Funziona? . . . 70
5.11 Fallimenti Bizantini: Il Caso Peggiore . . . 71
5.11.1 Cosa Può Fare un Nodo Bizantino . . . 71
5.11.2 L'Intuizione: Perché 3 non bastano per 1 traditore? (Il Triangolo del Sospetto) . . . 71
5.11.3 Limite Teorico . . . 71
5.11.4 Meccanismo RegisteredMail: La "Raccomandata con Ricevuta di Ritorno" 72
5.11.5 Protocollo TellZero_Byz (Sistema Sincrono) . . . 73
5.12 Riepilogo: Limiti di Tolleranza . . . 74
5.13 Esercizi e Domande . . . 74
5.13.1 Domande Concettuali . . . 74
5.13.2 Esercizi di Dimostrazione . . . 74

6 Distributed Hash Tables e Chord **76**
6.1 Contesto: Le Reti Peer-to-Peer (P2P) . . . 76
6.1.1 Definizione e Filosofia . . . 76
6.1.2 Unstructured vs Structured Networks . . . 76
6.1.3 Evoluzione delle Architetture P2P . . . 76
6.2 Il Problema: Trovare un Ago in un Pagliaio . . . 77
6.2.1 Lo Scenario . . . 77

6.2.2 Approcci e Problemi . . . 77
6.3 L'Idea Geniale: Distributed Hash Table . . . 78
6.3.1 Scalable Distributed Data Structures . . . 78
6.3.2 Cos'è una DHT? . . . 78
6.4 Consistent Hashing: La Base di Tutto . . . 78
6.4.1 L'Approccio Ingenuo: Hashing Modulare . . . 78
6.4.2 La Soluzione: Consistent Hashing (L'Anello) . . . 79
6.4.3 Perché è Geniale (La Magia del Join) . . . 79
6.5 Chord: L'Implementazione . . . 80
6.5.1 Identificatori e Spazio delle Chiavi . . . 80
6.5.2 Cosa Mantiene Ogni Nodo . . . 80
6.5.3 Il Lookup Semplice (e Lento) . . . 80
6.6 La Finger Table: Il Trucco per $O(\log N)$ . . . 80
6.6.1 L'Idea: Salti e Balzi verso l'Obiettivo . . . 80
6.6.2 Analisi Approfondita: Perché il Lookup è Veloce? . . . 81
6.7 L'Algoritmo di Lookup . . . 81
6.8 Join e Leave: Dinamicità . . . 82
6.8.1 Quando un Nodo Entra (Join) . . . 82
6.8.2 Stabilization: Il Cerchio che si Auto-Guarisce . . . 82
6.8.3 Visual Walkthrough: Join Simplificato . . . 83
6.8.4 Stabilization: Mantenere la Coerenza . . . 84
6.8.5 Trace Table: Stabilization in Azione . . . 84
6.8.6 Fix Fingers . . . 85
6.9 Tolleranza ai Guasti in Chord . . . 85
6.9.1 Lista di Successori . . . 85
6.9.2 Replicazione dei Dati . . . 85
6.9.3 Node Leave: Uscita Volontaria vs Crash . . . 85
6.9.4 Il Problema del Churn . . . 85
6.10 Complessità di Chord . . . 85
6.11 Confronto con Altri Sistemi . . . 86
6.11.1 Applicazioni P2P Attuali (Legali) . . . 86
6.12 Esercizi e Domande . . . 86
6.12.1 Domande Concettuali . . . 86
6.12.2 Esercizi di Dimostrazione . . . 87

A Cheatsheet - Formule e Teoremi **88**

B Fondamenti e Teoria della Complessità **90**
B.1 Problemi Computazionali . . . 90
B.1.1 Cos'è un problema computazionale? . . . 90
B.1.2 Classificazione dei Problemi . . . 90
B.2 Algoritmi: Cosa Sono e Cosa Vogliamo da Loro . . . 91
B.2.1 Definizione Precisa . . . 91
B.2.2 Correttezza . . . 91
B.2.3 Efficienza: Perché la Velocità Conta . . . 92
B.3 Notazione Asintotica: Il Linguaggio dell'Efficienza . . . 92
B.3.1 L'idea intuitiva . . . 92
B.3.2 Le tre notazioni . . . 92
B.3.3 Gerarchia delle complessità . . . 93
B.4 Classe P: I Problemi "Facili" (Risolvibili) . . . 93
B.5 Classe NP: I Problemi "Verificabili" . . . 93
B.6 La Chiave di Volta: La Riduzione ($A \leq_p B$) . . . 93

B.7 NP-Hard vs NP-Completo: Qual è la differenza? 94
B.7.1 NP-Hard: I "Cattivi" (Hardest) 94
B.7.2 NP-Completo: L'Elite (Hardest $\cap$ NP) 94
B.8 Algoritmi di Approssimazione: "Amazon non può aspettare" 95
B.8.1 L'idea (α-Approssimazione) 95
B.8.2 L'Algoritmo di Christofides: Guida Visiva Definitiva 96
B.8.3 Riassunto Finale del Costo 98
B.8.4 Teoria Formale per l'Esame 98
B.8.5 Visual Walkthrough: Christofides Step-by-Step 99
B.8.6 Tightness del Bound 3/2 di Christofides 100
B.8.7 Vertex Cover con Approssimazione 2 102
B.8.8 Tecnica Relax-and-Round ($\alpha = 2$) 103
B.8.9 Tightness dell'Analisi 103
B.9 Riepilogo del Capitolo 104
B.10 Esercizi e Domande 104
B.10.1 Domande Concettuali 104
B.10.2 Esercizi di Dimostrazione 104

1 Modello dei Sistemi Distribuiti, Broadcast e Wake-up

Prerequisiti

- Capitolo 1: Notazione asintotica O, Ω, Θ
- Concetto di grafo: nodi, archi, grado, diametro

In questo capitolo costruiremo il modello teorico per ragionare sui sistemi distribuiti: come funzionano, cosa possono fare, e quali sono i primi problemi fondamentali che affronteremo.

1.1 Cos'è un Sistema Distribuito?

1.1.1 L'intuizione

Quando invii un'email, il messaggio deve attraversare molti computer per arrivare a destinazione. Ma come fa ogni computer a sapere "dove mandarlo dopo"?

Analogia del Mondo Reale: La stanza buia

Un sistema distribuito è come un gruppo di persone in una stanza buia che devono contarsi.

- Non si vedono (memoria locale separata).
- Possono solo bisbigliare al vicino (messaggi).
- Non c'è un direttore d'orchestra (controllo distribuito).
- Devono seguire un protocollo per non contarsi due volte o dimenticare qualcuno.

Definizione: Sistema Distribuito

Un **sistema distribuito** è un insieme di entità autonome che:

1. **Non condividono memoria**: Ogni entità ha la propria memoria privata
2. **Non hanno clock sincronizzati**: Ogni entità ha il proprio "orologio interno"
3. **Comunicano tramite messaggi**: L'unico modo per scambiare informazioni
4. **Cooperano per un obiettivo comune**: Devono risolvere un problema insieme

1.1.2 Parallelo vs Distribuito: Una Distinzione Importante

Aspetto	Sistema Parallelo	Sistema Distribuito
Memoria	Condivisa: tutti vedono tutto	Separata: ognuno vede solo la propria
Clock	Sincronizzato	Ogni entità ha il proprio
Comunicazione	Lettura/scrittura in memoria	Invio di messaggi
Distanza fisica	Processori vicini (stesso chip)	Potenzialmente in continenti diversi
Esempio	GPU, multi-core CPU	Internet, sensori IoT, blockchain

Nota

Perché la distinzione è importante?

Gli algoritmi paralleli spesso assumono che tutti possano leggere la stessa variabile nello stesso momento. In un sistema distribuito, questa assunzione è *fondamentalmente impossibile*: è la radice di molte difficoltà!

1.2 La Rete di Comunicazione

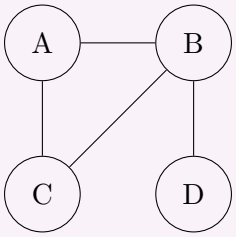
1.2.1 Il modello a grafo

La struttura di "chi può parlare con chi" è rappresentata come un **grafo**:

- **Nodi** = entità (processi, computer, sensori...)
- **Archi** = collegamenti di comunicazione diretta

Esempio:

Una rete con 4 nodi dove A può parlare con B e C, B può parlare con C e D:



In questo grafo, A può mandare messaggi a B direttamente, ma per raggiungere D deve passare per B (comunicazione indiretta).

1.3 Il Modello Computazionale: Come Funziona un'Entità

1.3.1 Anatomia di un'entità

Ogni entità nel sistema è come un piccolo computer autonomo:

1. **Stato**: L'entità si trova sempre in uno stato (es. IDLE, ACTIVE, DONE)
2. **Memoria locale**: Variabili private (es. contatori, liste di vicini)
3. **Regole di comportamento**: "Se sono in questo stato e succede questo, fai quest'altro"

1.3.2 Gli Eventi: Cosa Può Succedere

Un'entità non fa nulla finché non succede qualcosa. I possibili eventi sono:

Tipo di Evento	Descrizione
Ricezione messaggio	"Qualcuno mi ha mandato un messaggio"
Tick del clock	"Il mio timer è scaduto"
Impulso spontaneo	"Qualcosa di esterno mi ha svegliato" (es. un utente)

Attenzione

Regola fondamentale: Se non succede nulla (nessun evento), l'entità *non fa nulla*!
Le entità sono **reattive**, non proattive.

1.3.3 Le Regole di Comportamento

Il "programma" di un'entità è un insieme di regole del tipo:

```
stato_attuale; evento → azioni; nuovo_stato
```

Esempio:

Una regola per un protocollo di broadcast:

```
SLEEPING; receiving(INFO) ->
    send(INFO) to all neighbors except sender;
    become(DONE)
```

Lettura: "Se sono in stato SLEEPING e ricevo un messaggio INFO, allora invio INFO a tutti i miei vicini tranne chi me l'ha mandato, e passo allo stato DONE."

Un algoritmo distribuito deve essere:

- **Deterministico**: Data una coppia (stato, evento), esiste *esattamente una* azione definita.
- **Completo**: Per ogni possibile coppia (stato, evento), *esiste* un'azione definita (anche se è "non fare nulla").

Importante

Simmetria algoritmica
Tutte le entità eseguono lo *stesso* algoritmo (le stesse regole)!
Le differenze nel comportamento emergono da:

- Ruoli diversi (chi inizia vs chi aspetta)
- Eventi diversi (messaggi ricevuti in ordini diversi)
- Dati locali diversi (es. ID univoci)

1.4 Gli Assiomi: Cosa Assumiamo Sempre Vero

Gli assiomi sono le "regole del gioco" che assumiamo sempre valide.

1.4.1 Assioma 1: Consegna Garantita

Teorema: Tempo di Trasmissione Finito

In assenza di fallimenti, ogni messaggio inviato **arriva** alla destinazione in tempo finito.

Cosa significa? Non sappiamo *quando* arriverà (potrebbe metterci 1 millisecondo o 10 minuti), ma sappiamo che *arriverà*. Il ritardo è finito ma **imprevedibile**.

1.4.2 Assioma 2: Orientamento Locale

Teorema: Orientamento Locale

Ogni entità può **distinguere i propri vicini** attraverso "porte" etichettate.

Analogia del Mondo Reale: La Nebbia di Guerra (Fog of War)

Uno degli errori più comuni per chi inizia è avere una "visione divina" del sistema. Tu (che studi) vedi il disegno del grafo. Il nodo NO.

Immagina di essere in una stanza senza finestre (il nodo).

- Vedi solo 3 tubi pneumatici sul muro, etichettati "1", "2", "3".
- Non sai se il tubo "1" porta alla stanza accanto o in Giappone.
- Non sai se fuori ci sono altre 5 stanze o 5 miliardi di stanze.
- Non sai nemmeno se i tubi sono collegati in un cerchio o in una linea.

Tutto ciò che sai del mondo devi scoprirlo mandando messaggi dentro quei tubi e aspettando risposte. Questa è la sfida distribuita!

Esempio:

Se un nodo X ha tre vicini A, B, C, può pensarli come:

- Porta 1 $\rightarrow$ A
- Porta 2 $\rightarrow$ B
- Porta 3 $\rightarrow$ C

Quando X riceve un messaggio, sa *da quale porta* è arrivato (quindi da quale vicino). Quando invia, può dire "manda sulla porta 2".

1.5 Misure di Complessità nei Sistemi Distribuiti

Come misuriamo quanto è "costoso" un algoritmo distribuito?

1.5.1 Complessità dei Messaggi (Comunicazione)

Definizione:

La **complessità dei messaggi** è il numero totale di messaggi scambiati durante l'esecuzione.

Può essere analizzata a due livelli di granularità:

- **Message Complexity**: numero totale di messaggi.
- **Bit Complexity**: numero totale di bit trasmessi (rilevante se i messaggi hanno dimensioni molto diverse).

1.5.2 Complessità Temporale (Tempo Ideale)

Come definiamo il "tempo" in un sistema senza clock globale?

Definizione: Tempo Ideale vs Realistico

- **Modello Ideale (Sincrono)**: Ogni messaggio impiega esattamente 1 unità di tempo.
- **Modello Realistico (Asincrono)**: I messaggi hanno ritardi diversi. Il tempo è la **lunghezza della catena causale più lunga** di messaggi.

Esempio: Analisi del tempo tramite topologia

Sia $d(x, y)$ la distanza minima tra i nodi x e y.

- **Eccentricità** $r(x) = \max_y\{d(x, y)\}$: la distanza massima dal nodo x a qualsiasi altro nodo.
- **Diametro** $D(G) = \max_x\{r(x)\}$: la massima tra tutte le eccentricità.

Nel broadcast, se l'iniziatore è s, il tempo ideale è esattamente $r(s)$. Se non sappiamo chi è l'iniziatore, il tempo nel caso peggiore è $D(G)$.

1.6 Il Problema del Broadcast

Ecco il nostro primo problema fondamentale!

1.6.1 Definizione del Problema

Definizione: Broadcast

Un'entità speciale (l'**iniziatore**) possiede un'informazione I e vuole che **tutte le altre entità** la ricevano.

Terminazione corretta: Ogni entità sa quando il broadcast è completato.

Restrizioni standard:

- Un solo iniziatore
- Total reliability (nessun fallimento)
- Link bidirezionali
- Grafo connesso

1.6.2 Prima Idea (Sbagliata!)

```
INITIATOR:
  spontaneamente: send(I) to tutti i vicini

SLEEPING:
  receiving(I): send(I) to tutti i vicini
```

Attenzione

Problema: Questo algoritmo non termina mai!

Se A manda a B e B manda a C, poi C rimanda a B, B rimanda ad A... i messaggi rimbalzano all'infinito!

1.6.3 Soluzione Corretta: Protocollo Flooding

L'idea chiave: **cambia stato** dopo aver processato il messaggio, così non lo inoltri più volte.

```
INITIATOR:
  spontaneamente:
    send(I) to tutti i vicini
    become(DONE)

SLEEPING:
  receiving(I):
    send(I) to tutti i vicini ECCETTO il mittente
    become(DONE)

DONE:
  receiving(I):
    nil  // ignora, ho già ricevuto
```

Pseudocodice Formale (notazione standard):

```
STATI: {INITIATOR, SLEEPING, DONE}
VARIABILI: nessuna variabile locale richiesta

--- INITIATOR ---
IF stato = INITIATOR
  event: impulso spontaneo
    action:
      send(I) to N(x)      // N(x) = insieme dei vicini
      become(DONE)

--- SLEEPING ---
IF stato = SLEEPING
  event: ricevi(I) da sender
    action:
      send(I) to N(x) - {sender}   // tutti tranne mittente
      become(DONE)

--- DONE ---
IF stato = DONE
  event: ricevi(I)
    action:
      nil   // ignora messaggi duplicati
```

Perché funziona?

1. **Tutti ricevono**: Il grafo è connesso, quindi esiste un cammino dall'iniziatore a ogni nodo. L'informazione segue questi cammini.

2. **Terminazione**: Ogni nodo passa a DONE dopo aver ricevuto. In DONE, ignora i messaggi duplicati. Nessun messaggio viene generato all'infinito.

Attenzione

Il Paradosso della Terminazione: "Ho finito, ma abbiamo finito?"

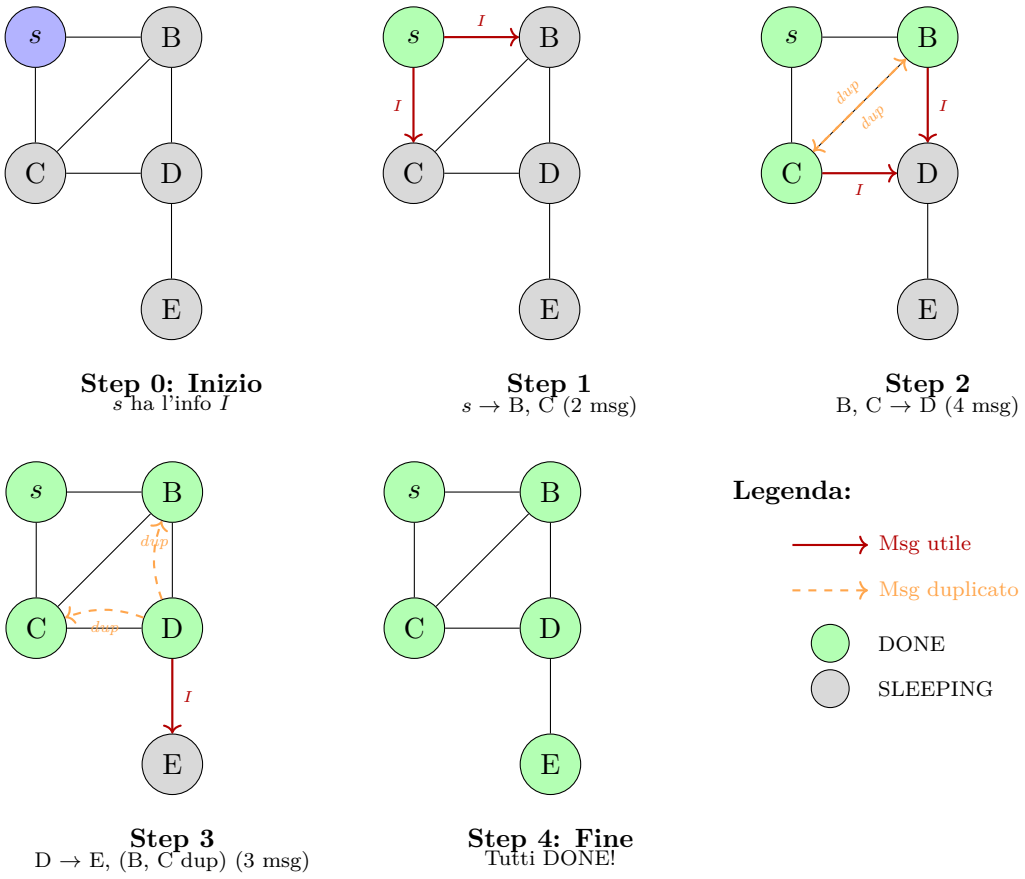
Nel Flooding c'è una distinzione fondamentale:

- **Terminazione Locale**: Ogni nodo sa quando *lui* ha finito (entra in stato DONE appena invia ai vicini).
- **Terminazione Globale**: Nessun nodo sa quando *tutti* hanno finito!

Analogia: Immagina di diffondere un pettegolezzo. Tu lo dici ai tuoi amici e "hai finito" (locale). Ma come fai a sapere quando l'ultimo abitante della città lo ha saputo (globale)? Non puoi saperlo senza un meccanismo di feedback (come un *ack* che torna indietro). Questo è il **Termination Detection Problem**.

1.6.4 Visual Walkthrough: Flooding in Azione

Vediamo come il flooding propaga l'informazione su un grafo di esempio. I nodi in grigio sono SLEEPING, quelli in verde sono DONE.



Importante

Analisi dell'esempio:

- **Nodi**: $n = 5$, **Archi**: $m = 6$
- **Messaggi totali**: $2+4+3 = 9$ (formula: $2m - n + 1 = 12 - 5 + 1 = 8$... ma abbiamo contato 9!)
- **Messaggi utili** (rosso pieno): 5 (uno per nodo tranne l'iniziatore, più D riceve 2 volte)
- **Messaggi duplicati** (arancione tratteggiato): 4 (B↔C, D→B, D→C)
- **Tempo**: 3 step (uguale al diametro $D = 3$ da s a E)

I messaggi duplicati sono lo "spreco" del flooding su grafi con cicli!

1.6.5 Analisi della Complessità del Flooding

Quanti messaggi vengono inviati?

Analizziamo arco per arco:

- Ogni arco può essere attraversato al massimo 2 volte (una per direzione)
- In realtà, gli archi del "percorso minimo" sono attraversati una sola volta

Conteggio preciso:

- L'iniziatore manda ai suoi $deg(init)$ vicini
- Ogni altro nodo manda a tutti i vicini *tranne* chi gli ha inviato I: $deg(x) - 1$ messaggi

$$\begin{aligned}\text{Messaggi totali} &= deg(init) + \sum_{x \neq init} (deg(x) - 1) \\ &= \sum_{x} deg(x) - (n-1) \\ &= 2m - (n-1) \\ &= 2m - n + 1\end{aligned}$$

Tempo ideale: Il tempo è determinato dal nodo più "lontano" dall'iniziatore.

$$\text{Tempo} = D \text{ (diametro del grafo)}$$

1.6.6 Flooding è Ottimale?

Per il tempo: Sì! Il nodo più lontano è a distanza D, e l'informazione deve raggiungerlo → lower bound $\Omega(D)$.

Per i messaggi: Dipende dalla topologia!

Topologia	Messaggi Flooding	Ottimale?
Albero ($m = n - 1$)	$2(n-1) - n + 1 = n - 1$	Sì!
Grafo completo ($m = \binom{n}{2}$)	$2\binom{n}{2} - n + 1 \approx n^2$	No! (basta $n - 1$)

Nota

Insight chiave: Flooding è ottimale su alberi perché gli alberi non hanno "archi di troppo". Su grafi densi, molti messaggi sono duplicati e sprecati.

1.6.7 SimpleBroadcast: Ottimizzazione per Grafi Completi

Se sappiamo che il grafo è **completo** (ogni nodo è direttamente connesso a tutti gli altri), possiamo fare molto meglio:

```
--- SimpleBroadcast (solo per grafi completi) ---
INITIATOR:
  spontaneamente:
    send(I) to N(x)    // tutti i vicini
    become(DONE)

SLEEPING:
  receiving(I):
    become(DONE)       // NON inoltra!
```

Complessità:

- **Messaggi**: $n - 1$ (solo l'iniziatore manda)
- **Tempo**: 1 (tutti i vicini ricevono in un round)

Importante

Confronto su Grafo Completo:

Algoritmo	Messaggi	Tempo
Flooding	$\approx n^2$	1
SimpleBroadcast	$n - 1$	1

SimpleBroadcast è ottimale in messaggi perché ogni nodo (eccetto l'iniziatore) deve ricevere almeno un messaggio.

1.6.8 Message Complexity vs Bit Complexity

Esistono due modi per misurare il costo di comunicazione:

Metrica	Definizione
Message Complexity	Numero totale di messaggi scambiati
Bit Complexity	Numero totale di **bit** trasmessi

Nota

La distinzione è importante quando i messaggi hanno dimensione variabile. Nel nostro corso, ci concentriamo principalmente sulla **Message Complexity**, assumendo messaggi di dimensione costante o limitata.

1.6.9 Lower Bound: Dimostrazioni Formali

Dimostriamo rigorosamente i lower bound per il problema del Broadcast.

Teorema: Lower Bound Temporale

Per qualsiasi algoritmo di broadcast: Time(Broadcast) $\geq D(G)$
dove $D(G)$ è il diametro del grafo G.

Dimostrazione. Sia v un nodo a distanza D dall'iniziatore s (esiste per definizione di diametro).

Argomento: L'informazione I deve raggiungere v. L'unico modo per trasferire informazione è tramite messaggi. Ogni messaggio attraversa un arco, e impiega almeno 1 unità di tempo (nel modello ideale).

Catena causale: Per raggiungere v da s, l'informazione deve attraversare almeno D archi in sequenza (definizione di distanza minima). Questo richiede una catena causale di almeno D messaggi.

Conclusione:

$$\text{Time(Broadcast)} \geq d(s, v) = D(G)$$

Il flooding raggiunge esattamente questo bound, quindi è **ottimale in tempo**. □

Teorema: Lower Bound sui Messaggi (Caso Generale)

Per qualsiasi algoritmo di broadcast: Message(Broadcast) $\geq n - 1$

Dimostrazione. Ci sono n nodi di cui 1 è l'iniziatore che possiede già I.

Argomento: Gli altri $n-1$ nodi devono ricevere I. Ogni nodo riceve l'informazione tramite almeno un messaggio (prima ricezione).

Conclusione: Servono almeno $n - 1$ messaggi.

Questo lower bound è raggiunto su alberi, dove flooding usa esattamente $n-1$ messaggi. □

Teorema: Lower Bound Messaggi su Grafi con m Archi

In assenza di conoscenza topologica: Message(Broadcast) $\geq m$

Dimostrazione. Questa dimostrazione usa un **argomento avversariale**.

L'Intuizione dell'Esploratore al Buio: Immagina di essere in un labirinto buio. Devi trovare un interruttore nascosto. Non hai la mappa. Se c'è un corridoio che **non** percorri, l'interruttore potrebbe essere proprio lì in fondo! Per essere sicuro al 100% di trovarlo, devi percorrere *tutti* i corridoi almeno una volta.

Formalizzazione: Supponiamo per assurdo che esista un algoritmo corretto che invia meno di m messaggi. Allora esiste almeno un arco (u, v) che non viene mai attraversato da nessun messaggio (in nessuna direzione).

L'avversario (che costruisce il grafo contro di noi) può modificare il grafo inserendo un nuovo nodo w proprio "in mezzo" a quell'arco (u, v), rendendolo l'unico ponte per raggiungere w. Poiché il nostro algoritmo non attraversa (u, v), il nodo w non riceverà mai il messaggio. Quindi l'algoritmo fallisce. Per garantire la correttezza su *qualsiasi* grafo, bisogna attraversare (tastare) tutti gli m archi. □

Importante

Riassunto Lower Bounds per Broadcast:

- **Tempo**: $\Omega(D)$ — Flooding raggiunge esattamente D: **ottimale**
- **Messaggi**: $\Omega(n-1)$ — Flooding usa $2m-n+1$: **non sempre ottimale**
- **Messaggi (senza conoscenza)**: $\Omega(m)$ — Ma flooding usa $2m$: **fattore 2**

Per avere meno di $2m$ messaggi, serve costruire una struttura ausiliaria (spanning tree).

1.7 Wake-up Problem: Una Generalizzazione

1.7.1 Definizione

Definizione: Wake-up Problem

All'inizio, **alcune** entità (gli "iniziatori") sono attive. Le altre dormono. L'obiettivo è che **tutte** le entità diventino attive.

Relazione con Broadcast:

- Broadcast = Wake-up con **1 solo iniziatore**.
- Wake-up = Broadcast con **più iniziatori possibili**.

1.7.2 Analisi della Complessità dei Messaggi (WFlood)

Se usiamo la strategia del flooding per il Wake-up (**WFlood**):

- Se c'è **1 solo iniziatore**: $2m-n+1$ messaggi.
- Se **tutti gli** n **nodi** sono iniziatori: $2m$ messaggi (ogni arco è attraversato in entrambi i sensi da messaggi "Wake-up" che si incontrano).
- Se ci sono k **iniziatori**:
$$\text{Messaggi} = 2m-(n-k)$$

Intuizione: Risparmiamo un messaggio per ogni nodo che non è iniziatore (quello che lo sveglia e che lui non deve rimandare indietro). Più iniziatori ci sono, meno messaggi "risparmiamo".

1.7.3 Complessità Temporale

Se c'è un solo iniziatore s, il tempo è $r(s)$. In generale, con più iniziatori, il tempo sarà limitato dal raggio del nodo più lontano da qualsiasi iniziatore. Nel caso peggiore, è comunque $O(D)$.

1.8 Verso la Soluzione Ottimale

Osservazione chiave: Flooding è ottimale su alberi.

Idea: E se costruissimo prima un albero che copre tutto il grafo (spanning tree) e poi facessimo broadcast solo su quello?

Messaggi totali:

1. Costruzione spanning tree: da determinare (prossimo capitolo!)
2. Broadcast su albero: $n-1$ messaggi

Se la costruzione costa meno dei messaggi "sprecati" dal flooding su grafi densi, conviene!

1.9 Riepilogo del Capitolo

Concetto	Punto Chiave
Sistema distribuito	Entità autonome, no memoria condivisa, comunicazione via messaggi
Modello computazionale	Stati + eventi + regole di comportamento
Assiomi	Consegna garantita + orientamento locale
Complessità	Messaggi (numero totale) + Tempo (catena causale più lunga)
Flooding	$2m - n + 1$ messaggi, tempo D

Key Takeaways

1. **Modello**: Entità autonome, no memoria condivisa, comunicazione via messaggi
2. **Complessità messaggi**: Conta il numero totale di messaggi
3. **Complessità tempo**: Lunghezza della catena causale più lunga
4. **Flooding**: $2m - n + 1$ messaggi, tempo $= D$ (diametro)
5. **Ottimale su alberi**: Su alberi, flooding usa $n - 1$ messaggi (minimo possibile)

Errori Comuni da Evitare

- **Flooding senza cambio stato**: Loop infinito! Serve cambiare stato dopo aver ricevuto
- **Parallelo $\neq$ Distribuito**: Nel parallelo c'è memoria condivisa, nel distribuito no
- **Tempo ideale**: Non è il tempo reale in secondi, ma il numero di "hop" sequenziali

1.10 Esercizi e Domande

1.10.1 Domande Concettuali

1. Perché le entità in un sistema distribuito devono eseguire tutte lo stesso algoritmo?
2. Cosa succederebbe se il flooding non cambiasse stato dopo aver ricevuto?
3. Su quale tipo di grafo il flooding spreca più messaggi? Perché?
4. Perché il tempo ideale si misura come "lunghezza della catena causale più lunga" invece che in secondi?
5. Se il grafo non fosse connesso, il flooding funzionerebbe? Perché?
6. Cos'è il Wake-up Problem e come si relaziona al Broadcast?

1.10.2 Esercizi di Dimostrazione

1. **Descrivi il modello computazionale** per sistemi distribuiti e gli assiomi del sistema.
2. **Scrivi il protocollo Flooding** e dimostra la correttezza.
3. **Analizza la complessità** del Flooding: deriva la formula $2m - n + 1$ per i messaggi.
4. **Dimostra i lower bound** per tempo ($\Omega(D)$) e messaggi ($\Omega(m)$) del broadcast.
5. **Perché Flooding è ottimale sugli alberi** ma non sui grafi completi? Qual è la strategia ottimale per il broadcast su grafi generici?

Tracce di Soluzione

Soluzioni Domande Concettuali:

1. **Same algorithm**: Simmetria algoritmica - le differenze emergono da ruoli, eventi e dati locali diversi.
2. **No state change**: Loop infinito! I messaggi rimbalzano all'infinito tra nodi.
3. **Max spreco**: Grafi completi ($m = \binom{n}{2}$). Flooding usa $\approx n^2$ msg, ma bastano $n - 1$.
4. **Tempo ideale**: Perché i ritardi reali sono imprevedibili, ma la catena causale è invariante.
5. **Grafo sconnesso**: No! Nodi in componenti separate dall'iniziatore non riceveranno mai I.
6. **Wake-up**: $k \geq 1$ iniziatori spontanei. Broadcast = Wake-up con $k = 1$.

Soluzioni Esercizi di Dimostrazione:

1. **Modello**: Grafo connesso, entità reattive, comunicazione asincrona via messaggi. Assiomi: consegna garantita + orientamento locale.
2. **Flooding**: Invia a tutti tranne mittente, diventa DONE. Corretto perché grafo connesso + cambio stato previene loop.
3. **Complessità**: $deg(init) + \sum_{x \neq init}(deg(x) - 1) = 2m - (n - 1) = 2m - n + 1$.
4. **Lower bounds**: Tempo $\geq D$ (catena causale minima). Messaggi $\geq m$ (argomento avversariale: arco non attraversato potrebbe nascondere un nodo).
5. **Ottimalità alberi**: $m = n - 1 \Rightarrow$ messaggi $= n - 1$ (minimo). Strategia ottimale: costruire spanning tree, poi broadcast su albero.

2 Spanning Tree Construction e Computation in Trees

Prerequisiti

- Capitolo 2: Modello distribuito, flooding, complessità messaggi/tempo
- Concetto di albero: grafo connesso senza cicli, $n - 1$ archi

In questo capitolo impariamo a costruire una struttura fondamentale: lo spanning tree. Vedremo perché è utile, come costruirlo in modo distribuito, e come sfruttarlo per computazioni efficienti.

2.1 Cos'è uno Spanning Tree e Perché ci Serve

2.1.1 Definizione Intuitiva

Immagina di avere una rete di computer (un grafo) e di voler "potare" alcuni collegamenti in modo che:

- Tutti i computer rimangano raggiungibili tra loro
- Non ci siano "cicli" (percorsi circolari)
- Usiamo il minor numero possibile di collegamenti

Definizione: Spanning Tree

Dato un grafo connesso $G = (V, E)$, uno **spanning tree** $T = (V, E')$ è:

- Un **sottografo** di G (usa solo archi esistenti: $E' \subseteq E$)
- Che include **tutti i nodi** (stesso insieme V)
- Ed è un **albero** (connesso e senza cicli)

Quanti archi ha? Esattamente $n - 1$ (proprietà degli alberi).

2.1.2 Perché lo Spanning Tree è Fondamentale

Ricordi il problema del Flooding? Su grafi densi sprecavamo molti messaggi.

Importante

Insight Chiave

Il Flooding su un albero usa esattamente $n - 1$ messaggi (ottimale)!

Strategia: Prima costruisci uno spanning tree, poi fai tutte le comunicazioni sull'albero invece che sul grafo completo.

Il costo della costruzione si "ammortizza" se usiamo l'albero per molte operazioni successive.

Analogia del Mondo Reale: L'Organigramma Aziendale

Immagina un'azienda dove tutti parlano con tutti (grafo completo): il caos! Uno **Spanning Tree** è come creare un organigramma gerarchico:

- Il CEO (radice) parla ai Manager.
- I Manager parlano agli Impiegati (foglie).
- L'informazione fluisce ordinata, senza duplicati e senza loop.

2.2 Protocollo SHOUT: L'Idea Base

2.2.1 L'Intuizione

Immagina l'iniziatore come una persona che "grida" ai vicini: *"Vuoi essere mio vicino nell'albero?"*

- Chi riceve la domanda **per primo** risponde **SÌ** e diventa parte dell'albero
- Chi ha già risposto SÌ a qualcun altro risponde **NO**

Questo crea naturalmente un albero radicato nell'iniziatore!

2.2.2 I Tre Tipi di Messaggi

Messaggio	Significato
Q (Question)	"Vuoi far parte del mio albero?"
YES	"Sì, divento tuo figlio nell'albero"
NO	"No, sono già nell'albero di qualcun altro"

2.2.3 Come Funziona SHOUT

1. **L'iniziatore** manda Q a tutti i suoi vicini
2. **Un nodo IDLE che riceve Q**:
   - Risponde YES al mittente (diventa suo "figlio")
   - Manda Q a tutti gli *altri* vicini
   - Aspetta le risposte
3. **Un nodo già ATTIVO che riceve Q**:
   - Risponde NO (è già nell'albero)
4. **Terminazione**: Quando un nodo ha ricevuto risposta (YES o NO) da tutti i vicini a cui ha mandato Q

Pseudocodice Formale:

```
STATI: {INITIATOR, IDLE, ACTIVE, DONE}
VARIABILI LOCALI:
    root: booleano (sono la radice?)
    parent: il nodo da cui ho ricevuto il primo Q
    Tree_neighbours: insieme dei vicini nell'albero
    counter: numero di risposte attese
```

```
--- INITIATOR ---
IF stato = INITIATOR
  event: impulso spontaneo
    action:
      root = TRUE
      parent = NIL
      Tree_neighbours = {}
      counter = |N(x)|            // numero vicini
      send(Q) to N(x)            // manda Q a tutti
      become(ACTIVE)

--- IDLE ---
IF stato = IDLE
  event: ricevi(Q) da sender
    action:
      root = FALSE
      parent = sender
      Tree_neighbours = {sender}
      counter = |N(x)| - 1        // aspetta tutti tranne parent
      send(YES) to sender
      if counter == 0:            // caso foglia
          become(DONE)
      else:
          send(Q) to N(x) - {sender}
          become(ACTIVE)

--- ACTIVE ---
IF stato = ACTIVE
  event: ricevi(Q) da sender
    action:
      send(NO) to sender         // gia' nell'albero

  event: ricevi(YES) da sender
    action:
      Tree_neighbours = Tree_neighbours U {sender}
      counter = counter - 1
      CHECK()

  event: ricevi(NO) da sender
    action:
      counter = counter - 1
      CHECK()

PROCEDURA CHECK():
    if counter == 0:
        become(DONE)
        // Notifica: ho Tree_neighbours come vicini nell'albero
```

Nota

Correttezza: Alla terminazione, gli archi (x, y) tali che $y \in$ Tree_neighbours(x) formano uno spanning tree radicato nell'initiator. Ogni nodo tranne l'initiator ha esattamente un parent, e i YES creano una catena connessa.

2.2.4 Quali Archi Finiscono nell'Albero?

Gli archi dell'albero sono quelli dove:

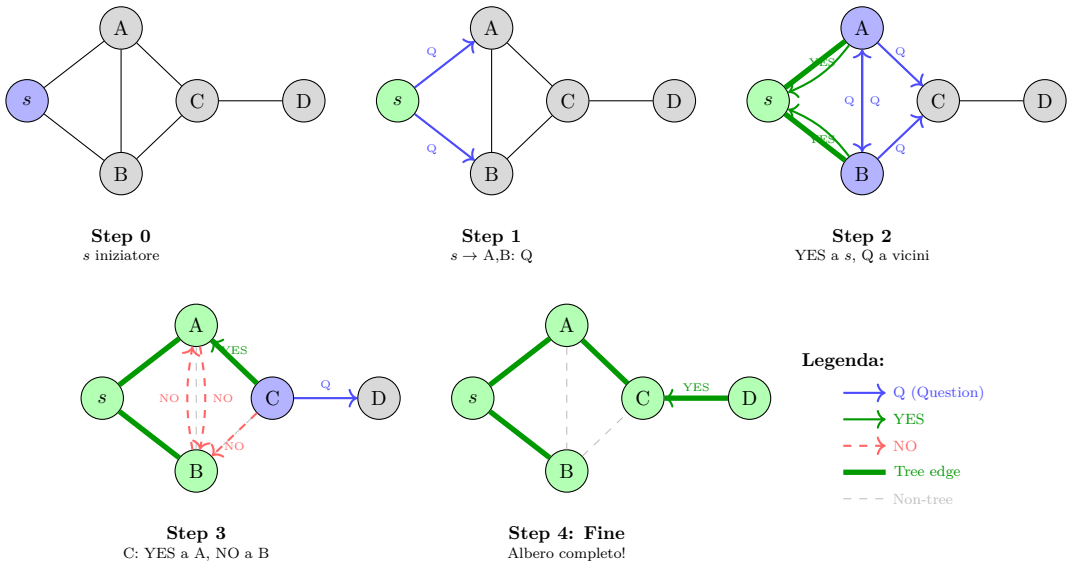
- È stata mandata una Q
- È tornato un YES

Nota

Alla fine, ogni nodo conosce i suoi **Tree-neighbors**: i vicini che fanno parte dell'albero. Non conosce tutto l'albero, solo la sua "porzione locale"!

2.2.5 Visual Walkthrough: SHOUT in Azione

Vediamo SHOUT su un grafo di 5 nodi. Gli archi verdi diventano parte dell'albero.



Importante

Risultato: Lo spanning tree ha archi s-A, s-B, A-C, C-D (4 archi $= n - 1$).

Messaggi contati:

- Tree edges (4): Q + YES = 8 messaggi
- Non-tree edges (A-B, B-C): Q + Q + NO + NO = fino a 8 messaggi
- Totale: fino a 16 messaggi (formula $4m - 2n + 2 = 24 - 10 + 2 = 16$)

2.2.6 Analisi dei Messaggi di SHOUT: Da dove esce quel calcolo?

Molti studenti imparano a memoria la formula $4m - 2n + 2$, ma è molto più facile capirla se analizziamo cosa succede su ogni singolo "tubo" (arco) della rete.

Immaginiamo la rete come un insieme di tubi. Abbiamo due tipi di tubi alla fine dell'esecuzione:

1. **Tubi dell'Albero (Tree Edges)**: Sono gli archi che "hanno avuto successo".
   - Qui è passata una domanda Q.
   - È tornata una risposta YES.
   - **Totale**: 2 messaggi per arco.
   - Poiché un albero ha $n - 1$ archi, questo ci costa $2(n - 1)$ messaggi.
2. **Tubi Scartati (Non-Tree Edges)**: Sono gli archi che collegano nodi già visitati. Qui avviene lo "spreco".

- Immagina che i nodi A e B siano collegati, ma entrambi siano già entrati nell'albero (tramite altri padri).
- A manda Q a B. B risponde NO. (2 messaggi)
- B manda Q ad A. A risponde NO. (2 messaggi)
- **Totale**: Fino a 4 messaggi per ogni arco inutile!

Il Calcolo Totale:

$$\text{Messaggi} = \underbrace{2(n-1)}_{\text{Archi Utili}} + \underbrace{4(m-(n-1))}_{\text{Archi Inutili}}$$

Svolgendo la matematica:

$$2n - 2 + 4m - 4n + 4 = 4m - 2n + 2$$

Questo è il **caso peggiore**. Nel caso migliore (se siamo fortunati con la sincronizzazione), sugli archi inutili potrebbe passare meno traffico, ma non possiamo contarci.

2.3 Protocollo SHOUT+: L'Ottimizzazione

2.3.1 L'Idea Geniale

Attenzione

Osservazione: Se due nodi si mandano Q a vicenda *contemporaneamente*, entrambi capiscono che l'altro è già nell'albero senza bisogno di un NO esplicito!

SHOUT+: Elimina i messaggi NO. Quando ricevi Q da qualcuno a cui avevi mandato Q, sai che non siete vicini nell'albero.

2.3.2 Complessità di SHOUT+

Ogni arco è attraversato esattamente da 2 messaggi:

- Arco albero: Q + YES
- Arco non-albero: Q + Q (nelle due direzioni)

$$Message(SHOUT+) = 2m$$

Miglioramento: Da $4m - 2n + 2$ a $2m$, quasi dimezzato!

2.3.3 Gestione di Iniziatori Multipli

Cosa succede se più nodi decidono di iniziare SHOUT contemporaneamente? Se i nodi hanno **ID univoci**, possiamo usare una regola di priorità (es. vince l'ID più piccolo).

- Ogni albero viene etichettato con l'ID del suo iniziatore.
- Se un albero "incontra" un altro albero con ID minore, si ferma e si lascia inglobare.
- Alla fine, sopravvive solo l'albero dell'iniziatore con ID minimo.

2.4 Costruzione via DFS Traversal

2.4.1 Un Approccio Diverso

Invece di "gridare" a tutti, facciamo una **visita in profondità** (DFS) del grafo usando un **token**.

Definizione: Token

Il **token** è un "testimone" unico che può possedere solo un nodo alla volta. Chi ha il token è "attivo" e può esplorare.

2.4.2 I Tre Tipi di Token

Token	Significato
Forward	"Vai avanti ed esplora"
Return	"Ho finito di esplorare, torna indietro"
Back-edge	"Questo nodo l'ho già visitato"

2.4.3 Come Funziona il DFS (L'Esploratore Solitario)

A differenza di SHOUT, dove tutti urlano contemporaneamente (approccio parallelo), il DFS è un approccio **sequenziale**.

Analogia del Mondo Reale: L'Esploratore con la Torcia

Immagina un labirinto buio e **un solo esploratore** con una torcia (il **Token**).

- Non può dividersi. Può essere solo in un punto alla volta.
- Quando arriva in una stanza (nodo), marca le porte da cui è arrivato.
- Sceglie una porta non marcata ed entra (Forward).
- Se finisce in un vicolo cieco o in una stanza già visitata, deve tornare indietro (Return o Back-edge) per provare un'altra porta.

Il protocollo usa tre messaggi per simulare questo movimento:

1. **Forward (Vai!)**: L'esploratore si sposta in un nuovo nodo non visitato. Questo crea un arco dell'albero.
2. **Back-edge (Occupato!)**: L'esploratore prova ad aprire una porta ma vede la luce dall'altra parte (il nodo è già stato visitato da un'altra strada). Richiude la porta e non passa.
3. **Return (Finito qui!)**: L'esploratore ha controllato tutte le porte della stanza corrente. Non c'è più nulla da fare, quindi torna indietro al nodo precedente (parent) per vedere se lì c'erano altre strade inesplorate.

Pseudocodice Formale:

```
STATI: {INITIATOR, IDLE, VISITED, DONE}
VARIABILI LOCALI:
    parent: nodo da cui ho ricevuto il primo Forward
    unvisited: insieme dei vicini non ancora esplorati
    Tree_neighbours: insieme dei vicini nell'albero
```

```
--- INITIATOR ---
IF stato = INITIATOR
  event: impulso spontaneo
    action:
      parent = NIL
      unvisited = N(x)
      Tree_neighbours = {}
      VISIT()

--- IDLE ---
IF stato = IDLE
  event: ricevi(Forward) da sender
    action:
      parent = sender
      unvisited = N(x) - {sender}
      Tree_neighbours = {sender}
      become(VISITED)
      VISIT()

--- VISITED ---
IF stato = VISITED
  event: ricevi(Forward) da sender
    action:
      // Gia' visitato: mando BackEdge
      send(BackEdge) to sender

  event: ricevi(BackEdge) da sender
    action:
      // Arco non-albero: continuo esplorazione
      VISIT()

  event: ricevi(Return) da sender
    action:
      // Figlio ha finito: aggiungo all'albero
      Tree_neighbours = Tree_neighbours U {sender}
      VISIT()

PROCEDURA VISIT():
    if unvisited ≠ {}:
        // Scegli un vicino non visitato
        next = estrai un elemento da unvisited
        send(Forward) to next
    else:
        // Ho esplorato tutti i vicini
        if parent == NIL:
            // Sono l'initiator: ho finito
            become(DONE)
        else:
            // Torno al parent
            send(Return) to parent
            become(DONE)
```

Nota

Token: In ogni istante, c'è esattamente UN token (Forward/BackEdge/Return) in transito. Questo garantisce la sequenzialità del DFS e semplifica la correttezza, ma limita il parallelismo.

2.4.4 Complessità del DFS

Messaggi: Ogni arco è attraversato esattamente 2 volte

$$Message(DFS) = 2m$$

Tempo: Poiché il token si muove *sequenzialmente* (un arco alla volta)

$$Time(DFS) = 2m$$

Il lower bound teorico per il tempo è $n - 1$ (bisogna visitare n nodi), ma DFS è molto più lento.

2.4.5 Ottimizzazione del Tempo (ACK)

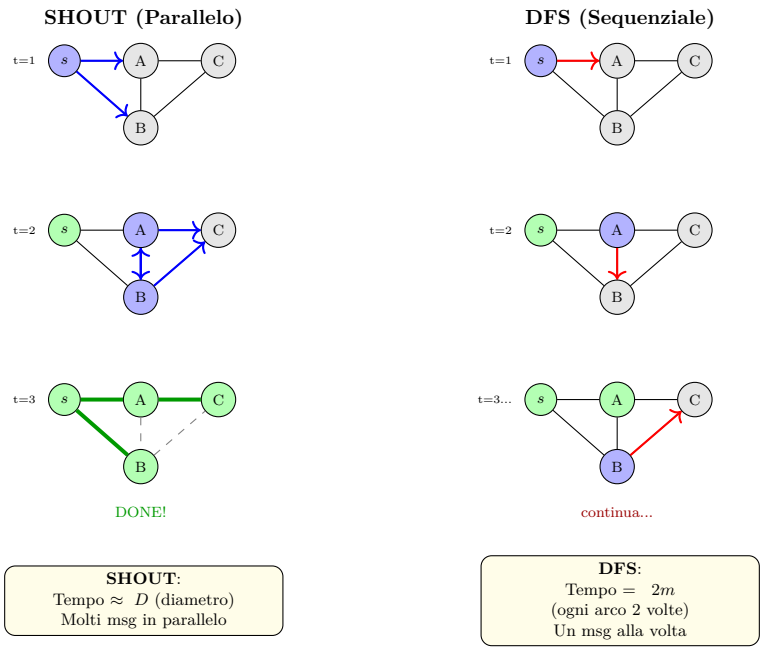
Possiamo evitare di inviare il token su archi "inutili" (verso nodi già visitati). Quando un nodo viene visitato, invia subito un messaggio **Visited** ("Ack") a tutti i suoi vicini. I vicini rimuovono quel nodo dalla lista `unvisited`. Così il token viaggia *solo* sugli archi dell'albero o quando scopre nuovi nodi, riducendo le traversate inutili (ma aumentando il numero di messaggi locali di notifica).

2.5 BFS Spanning Tree

Per ottimizzare il broadcast futuro, l'ideale non è un albero qualsiasi, ma un **BFS Tree** (Breadth-First Search) radicato nel **centro** del grafo. Questo minimizza l'altezza dell'albero e quindi il tempo di broadcast. Costruirlo è però più costoso in termini di messaggi rispetto a un DFS o SHOUT casuale.

2.5.1 Confronto Visivo: SHOUT vs DFS

Lo stesso grafo, due approcci molto diversi. Osserva la differenza nel parallelismo.



2.6 La Tecnica della Saturation

2.6.1 Computazione su Alberi

Una volta che abbiamo uno spanning tree, come facciamo computazioni distribuite?

Esempio: Ogni nodo ha un valore. Vogliamo che tutti scoprano qual è il valore **minimo**. Ma la tecnica è generale e si applica a:

- **Minimum Finding**: trovare il valore minimo nella rete.
- **Center Finding**: trovare il centro (o i centri) dell'albero.
- **Median Finding**: trovare il nodo mediano.
- **Computing Diameter**: calcolare il diametro della rete.

2.6.2 L'Idea della Saturation: "L'Incendio Controllato"

La tecnica della Saturazione è il metodo standard per fare calcoli su alberi **non radicati** (dove non c'è un capo prefissato). Il problema è: se nessuno comanda, chi inizia? E dove raccogliamo il risultato?

L'idea è simulare un incendio che parte dalle periferie (le foglie) e si propaga verso l'interno.

La Regola d'Oro (Attivazione): Un nodo può "attivarsi" (inviare il suo messaggio) **SOLO SE** ha ricevuto messaggi da **tutti i suoi vicini tranne uno**.

Analizziamo questa regola:

- **Foglie** (Leaves): Hanno solo 1 vicino. Quindi "tutti tranne uno" significa "zero vicini". Soddisfano la regola immediatamente! Sono le prime a partire.
- **Nodi Interni**: Hanno, diciamo, 4 vicini. Devono aspettare che 3 di questi si siano "spenti" (abbiano mandato il loro dato). Quei 3 rami sono completi.
- **Il Flusso**: L'informazione fluisce dai rami morti verso l'unico ramo ancora aperto. È come se ogni nodo dicesse: "Ho raccolto tutto dai miei sotto-rami, ora passo la palla a te che sei verso il centro".

Il Punto di Incontro (Saturazione): Poiché il grafo è un albero, questo flusso dalle foglie verso l'interno deve per forza scontrarsi da qualche parte.

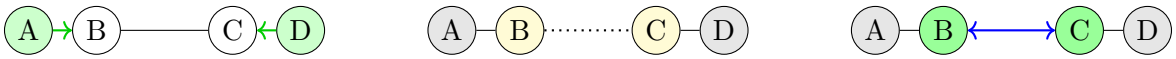
- Ci saranno due nodi adiacenti, A e B, che si attivano contemporaneamente (o quasi).
- A ha ricevuto da tutti tranne B. Manda a B.
- B ha ricevuto da tutti tranne A. Manda ad A.
- Quando questo scambio avviene, l'intero albero è stato "drenato". A e B sanno tutto. Sono **Saturi**.

Lemma 2.1 (Proprietà della Saturation)**.** In qualsiasi esecuzione su un albero, esattamente **due nodi adiacenti** diventeranno saturi (o, più formalmente, la saturazione avviene su un singolo arco).

Perché? In un albero, c'è sempre un "centro" formato da 1 o 2 nodi. La saturazione converge verso il centro.

2.6.3 Visual Walkthrough: La Saturazione

Visualizziamo come l'informazione "collasso" dalle foglie verso il centro.



(a) Step 1: Foglie attive (b) Step 2: Attesa (c) Step 3: Saturazione

Figura 1: A e D hanno 1 solo vicino → inviano. B aspetta solo C (ha ricevuto da A), C aspetta solo B (ha ricevuto da D). Quando si scambiano msg, sono Saturi!

2.6.4 Trace Table: Esecuzione Saturation

Immaginiamo un albero semplice: **A** – **B** – **C** – **D**, con valori 10, 5, 8, 2.

Tempo	Nodo A (Leaf)	Nodo D (Leaf)	Nodo B	Nodo C
0	Attivo (Leaf)	Attivo (Leaf)	Aspetta	Aspetta
1	Invia M(10) → B	Invia M(2) → C	Riceve da A	Riceve da D
2	Sonno	Sonno	Attivo	Attivo
3	-	-	Calcola min(10, 5)	Calcola min(2, 8)
4	-	-	Invia M(5) → C	Invia M(2) → B

Nota: B riceve da A e D riceve da C (passo 1). Ora B ha ricevuto da tutti tranne C, quindi diventa attivo verso C. C ha ricevuto da D, quindi attivo verso B. Si saturano a vicenda.

2.6.5 Complessità della Saturation

Fase	Messaggi
Saturation (foglie → centro)	n
Resolution (centro → foglie)	$n - 2$
Totale	$\leq 2n - 2$

Se includiamo una fase di wake-up iniziale: $\leq 4n - 4$ messaggi.

2.7 Computation in Rooted Trees e Convergecast

Quando l'albero ha una **radice** (un nodo speciale che tutti conoscono), le computazioni diventano più semplici.

2.7.1 Alberi Radicati: Senso di Direzione

In un albero radicato, ogni nodo sa:

- Chi è il suo **parent** (verso la radice)
- Chi sono i suoi **children** (verso le foglie)

Questo è chiamato **sense of direction** (up-down): ogni nodo distingue la direzione "verso la radice" dalla direzione "verso le foglie".

La radice è un **leader naturale** che può iniziare computazioni.

Teorema: Impossibilità di Radicare senza ID

Senza **identificatori unici**, è **impossibile** radicare un albero non radicato in modo deterministico.

Idea della dimostrazione. In un albero simmetrico (es. un cammino), senza ID unici tutti i nodi sono indistinguibili. Non c'è modo di "rompere la simmetria" e scegliere una radice unica. □

Nota

Rooted vs Unrooted:

- **Albero radicato**: C'è un leader naturale (la radice). Protocolli semplici: broadcast/convergecast.
- **Albero non radicato**: Nessun leader prefissato. Serve la **Saturation** per far convergere la computazione verso il centro.

2.7.2 Broadcast e Convergecast

Due pattern fondamentali sugli alberi radicati:

Operazione	Direzione	Uso
Broadcast	Root → Foglie	Diffondere informazione
Convergecast	Foglie → Root	Raccogliere/aggregare dati

2.7.3 Come Funziona il Convergecast

1. Le **foglie** inviano il proprio valore al parent
2. I **nodi interni** aspettano di ricevere da *tutti* i children
3. Aggregano i valori (es. MIN, SUM, AND) con il proprio
4. Inviano il risultato al parent
5. La **radice** riceve il risultato finale

Complessità: $n - 1$ messaggi (uno per arco), tempo = altezza dell'albero

Nota

Pattern comune: Broadcast + Convergecast

La radice invia una "domanda" (broadcast), i nodi raccolgono le "risposte" (convergecast). Usato in: minimum finding, leader election, sincronizzazione.

2.8 Riepilogo e Confronto

Algoritmo	Messaggi	Tempo	Parallelismo
SHOUT	$4m - 2n + 2$	-	Alto
SHOUT+	$2m$	-	Alto
DFS	$2m$	$2m$	Nessuno
DFS ottimizzato	$\leq 4m$	$O(n)$	Basso
Saturation	$\leq 4n - 4$	-	Medio

Key Takeaways

1. **SHOUT**: $4m - 2n + 2$ messaggi, alto parallelismo
2. **SHOUT+**: $2m$ messaggi (elimina i NO), stesso parallelismo
3. **DFS**: $2m$ messaggi, ma tempo $2m$ (sequenziale!)
4. **Saturation**: Computazione su alberi senza radice, converge verso il centro
5. **Convergecast**: Aggregazione foglie → radice, $n - 1$ messaggi

Errori Comuni da Evitare

- **DFS parallelo**: Il DFS base è sequenziale, tempo $O(m)$ non $O(D)$!
- **Saturation $\neq$ BFS**: La saturation parte dalle foglie, non dalla radice
- **SHOUT worst case**: Su grafi completi, $4m \approx 2n^2$ messaggi

2.9 Esercizi e Domande

2.9.1 Domande Concettuali

1. Perché uno spanning tree ha esattamente $n - 1$ archi?
2. Qual è la differenza chiave tra SHOUT e SHOUT+?
3. Perché il DFS ha tempo $2m$ mentre il flooding ha tempo D?
4. In che situazioni preferiresti usare DFS invece di SHOUT?
5. Come fa la saturation a "trovare il centro" dell'albero?
6. Cosa succede con iniziatori multipli in SHOUT?
7. Cos'è il Convergecast e in che contesto si usa?

2.9.2 Esercizi di Dimostrazione

1. **Descrivi il protocollo SHOUT** e deriva la formula $4m - 2n + 2$ per i messaggi.
2. **Qual è l'ottimizzazione di SHOUT+**? Dimostra perché usa esattamente $2m$ messaggi.
3. **Analizza il DFS Traversal**: perché il tempo è $2m$? Come si può migliorare a $O(n)$?
4. **Descrivi la Saturation Technique**: perché converge verso 2 nodi adiacenti?

Tracce di Soluzione

Soluzioni Domande Concettuali:

1. $n-1$ **archi**: Un albero connette n nodi senza cicli $\Rightarrow$ esattamente $n-1$ archi.
2. **SHOUT vs SHOUT+**: SHOUT+ elimina i messaggi NO. Se ricevo Q da chi ho mandato Q, capisco implicitamente che è "già visitato".
3. **DFS tempo**: Token sequenziale, attraversa ogni arco 2 volte. Flooding invece è parallelo.
4. **Quando DFS**: Quando servono proprietà DFS (ordine visita, back-edges) o memoria limitata.
5. **Saturation centro**: Le foglie partono per prime. L'informazione fluisce verso l'interno. Due nodi si saturano sull'arco centrale.
6. **Multipli iniziatori**: Alberi separati crescono e si scontrano. Serve meccanismo di fusione (es. vince ID minore).
7. **Convergecast**: Aggregazione foglie $\rightarrow$ radice. Usato per MIN, SUM, COUNT su alberi radicati.

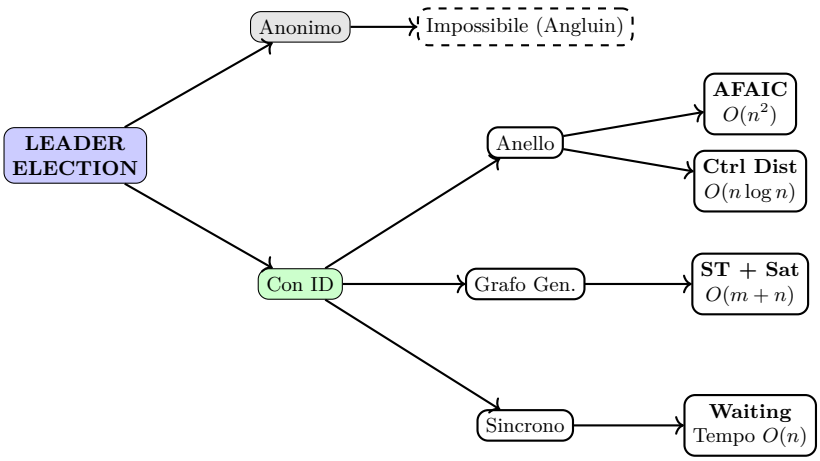
Soluzioni Esercizi di Dimostrazione:

1. **SHOUT formula**: Tree edges: $2(n-1)$ msg. Non-tree edges: fino a $4(m-n+1)$ msg. Totale: $4m-2n+2$.
2. **SHOUT+ = 2m**: Ogni arco attraversato esattamente 2 volte (Q+YES o Q+Q).
3. **DFS migliorato**: Notificare i vicini quando visitato $\Rightarrow$ token solo su tree edges $\Rightarrow$ tempo $O(n)$.
4. **Saturation convergenza**: Albero ha centro unico (1-2 nodi). Flusso dalle foglie converge lì.

3 Leader Election

Prerequisiti

- Capitolo 2: Modello distribuito, assiomi, restrizioni
- Capitolo 3: Spanning tree, saturation (per grafi generici)
- Concetto di anello: ogni nodo ha esattamente 2 vicini



In un sistema distribuito, spesso è necessario (ma difficile) scegliere un "capo".

3.1 Il Problema: Rompere la Simmetria

3.1.1 Perché Serve un Leader?

Molti algoritmi distribuiti hanno bisogno che **una sola** entità abbia un ruolo speciale:

- **Radice dello spanning tree**: Chi inizia la costruzione?
- **Coordinatore**: Chi prende decisioni finali?
- **Iniziatore del broadcast**: Chi ha l'informazione da diffondere?

Definizione: Leader Election

Problema: Tutte le entità partono in stato AVAILABLE. Alla fine:

- Esattamente **una** entità è in stato LEADER
- Tutte le altre sono in stato FOLLOWER

3.1.2 Il Problema Fondamentale

Attenzione

La Sfida della Simmetria

Se tutte le entità:

- Eseguono lo stesso algoritmo
- Partono nello stesso stato
- Hanno la stessa "vista" del sistema

Come può una distinguersi dalle altre?

Analogia del Mondo Reale: L'Orchestra Senza Direttore

Immagina un'orchestra dove tutti i musicisti sono bendati e suonano lo stesso strumento. Vogliono scegliere un direttore. Se tutti iniziano a suonare la stessa nota nello stesso momento (simmetria), nessuno riesce a distinguersi. Serve qualcuno che suoni una nota diversa, o più forte, o in un momento diverso (rottura della simmetria)!

3.1.3 Il Setup: Anello Anonimo

Consideriamo il caso più semplice ma rivelatore:

- Topologia: un **anello** (ogni nodo ha esattamente 2 vicini)
- Nodi **anonimi**: non hanno identificatori univoci
- Algoritmo **deterministico**: niente casualità
- Start **simultaneo**: tutti partono insieme

Teorema: Angluin, 1980

In un anello anonimo, bidirezionale, connesso, senza guasti e con start simultaneo, è **impossibile** eleggere un leader con un algoritmo deterministico.

Domanda Killer: Angluin Theorem

Questa è una domanda classica! Ricorda: il problema è la **simmetria**. Se tutti fanno la stessa cosa partendo dallo stesso stato, rimangono nello stesso stato.

3.1.4 Perché è Impossibile? (La Logica dei Cloni)

La dimostrazione formale usa l'induzione, ma il concetto logico è più semplice se immaginiamo uno scenario da incubo: **Il Mondo degli Specchi**.

Immagina un anello con 4 nodi: A, B, C, D. Sono anonimi (non hanno nomi), quindi per l'algoritmo sono identici.

1. **Round 0**: Tutti si svegliano nello stato "Available".
2. **Azione**: Poiché hanno lo stesso codice e lo stesso stato, devono fare la stessa cosa. Diciamo che tutti inviano un messaggio "Ciao" a destra.

3. **Percezione**:
   - A riceve "Ciao" da sinistra (da D).
   - B riceve "Ciao" da sinistra (da A).
   - Tutti ricevono esattamente lo stesso input!
4. **Transizione**: Poiché erano nello stesso stato e hanno ricevuto lo stesso input, passeranno tutti allo stesso nuovo stato S_1.

Il Loop Infinito: Non importa quanto sia complesso l'algoritmo. Se A decide "Divento Leader", allora B (che ha avuto la stessa storia di A) deve decidere "Divento Leader". Avremo 4 Leader. Se A decide "Aspetto", tutti aspettano.

Nota

Conclusione Logica: Senza ID univoci (che rendono A diverso da B) o randomizzazione (che permette ad A di lanciare Testa e a B Croce), la simmetria è una prigione perfetta da cui non si può uscire.

Nota

L'insight profondo

La simmetria iniziale non può essere rotta senza:

- **ID univoci** (rompono la simmetria iniziale), oppure
- **Casualità** (rompono la simmetria durante l'esecuzione)

3.2 Leader Election su Anello con ID

Da ora assumiamo che ogni nodo abbia un **ID univoco**.

3.2.1 La Strategia Generale

Idea: Il nodo con l'ID più piccolo (o più grande) diventa leader.

Problema: Come fa un nodo a sapere di avere l'ID minimo?

3.2.2 Protocollo All-the-Way (Le Lann, 1977)

Il primo protocollo storico per leader election su anello.

Idea: Ogni ID **fa il giro completo** dell'anello.

```
Ogni nodo x con ID_x:
    invia il proprio ID in senso orario
    per ogni messaggio ricevuto con ID_received:
        inoltra il messaggio
        aggiorna: min_seen = min(min_seen, ID_received)

    quando hai visto n messaggi distinti:
        se min_seen == ID_x:
            diventa LEADER
        altrimenti:
            diventa FOLLOWER
```

Pseudocodice Formale (Le Lann, 1977):

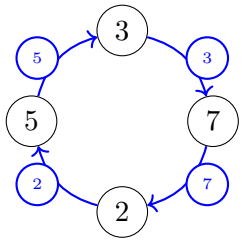
```
STATI: {ASLEEP, AWAKE, LEADER, FOLLOWER}
VARIABILI LOCALI:
    known: insieme di ID gia' visti (inizialmente vuoto)
    min_id: ID minimo visto finora
    count: contatore messaggi ricevuti
    ringsize: dimensione anello (nota a priori)

INIZIALIZZAZIONE:
    known = {id(x)}
    min_id = id(x)
    count = 0
    send("Election", id(x)) al vicino destro

REGOLE:
  IF stato = AWAKE
    event: ricevi("Election", id_received)
      action:
        if id_received NOT IN known:
            known = known U {id_received}
            min_id = min(min_id, id_received)
            count = count + 1
            send("Election", id_received) al vicino destro

        if count == ringsize - 1:  // ho visto tutti gli altri
            CHECK()

PROCEDURA CHECK():
    if min_id == id(x):
        send("Notify") al vicino destro
        become(LEADER)
    else:
        become(FOLLOWER)
```

Nota

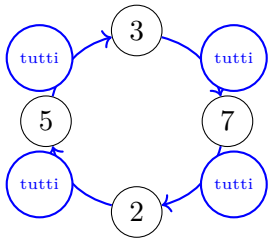
Requisiti: Il protocollo All-the-Way richiede che ogni nodo conosca la dimensione n dell'anello a priori, oppure che i link siano FIFO (in tal caso, quando un nodo riceve il proprio ID, ha visto tutti).

3.2.3 Visual Walkthrough: All-the-Way in Azione

Come si comportano gli ID nell'anello? Vediamolo.



(a) Round 1: Tutti gli ID vengono inoltrati (nessun filtraggio in All-the-Way).



(b) Ogni ID fa il giro completo. Dopo n round, tutti hanno visto tutti gli ID.

Figura 2: All-the-Way: Ogni ID attraversa l'intero anello. Messaggi totali: n^2.

Problema: Come so quando ho visto tutti gli ID?
Due soluzioni:

- **Conoscenza di** n: Conto i messaggi distinti
- **FIFO**: Quando rivedo il mio ID, ho visto tutti

Complessità:

- Messaggi: n^2 (ogni ID attraversa n nodi)
- Tempo: $O(n)$

Nota

All-the-way è semplice ma inefficiente. AFAIC migliora su questo, e Controlled Distance va ancora oltre.

3.2.4 Protocollo "As Far As It Can" (AFAIC/LCR)

Intuizione: Ogni nodo manda il suo ID "in giro" per l'anello. L'ID più piccolo "sopravvive" più a lungo.

Regole:

1. Ogni nodo manda il proprio ID al vicino (in senso orario)
2. Quando ricevi un ID:
   - Se è **più piccolo** del tuo → inoltralo (potrebbe vincere)
   - Se è **più grande** del tuo → scartalo (non può vincere)
   - Se è **uguale** al tuo → SEI IL LEADER! (ha fatto il giro completo)

Pseudocodice Formale (LeLann-Chang-Roberts):

```
STATI: {ASLEEP, CANDIDATE, FOLLOWER, LEADER}
VARIABILI: send: booleano (se devo inviare il mio ID)

--- INITIATOR ---
IF stato = ASLEEP (spontaneous awakening)
    action:
        send("Election", id(x)) al vicino destro
        become(CANDIDATE)

--- CANDIDATE ---
IF stato = CANDIDATE
    event: ricevi("Election", id_received)
      action:
        if id_received < id(x):
            // ID piu' piccolo del mio: lo inoltro
            send("Election", id_received) al vicino destro
            become(FOLLOWER)
        else if id_received > id(x):
            // ID piu' grande del mio: lo scarto
            nil    // do nothing
        else:  // id_received == id(x)
            // Il mio ID ha fatto il giro completo!
            send("Notify") al vicino destro
            become(LEADER)

--- FOLLOWER ---
IF stato = FOLLOWER
    event: ricevi("Election", id_received)
      action:
```

```
        if id_received < id(x):
            send("Election", id_received) al vicino destro
        // se >= id(x), scarta

    event: ricevi("Notify")
      action:
        send("Notify") al vicino destro
        become(DONE)
```

Importante

Differenza chiave con All-the-Way: In AFAIC, gli ID vengono **scartati** non appena incontrano un ID più piccolo, riducendo il numero di messaggi nel caso medio.

Esempio: Anello con ID [3, 7, 2, 5]

- ID 2 attraversa tutti e 4 i nodi → 4 messaggi
- ID 3 attraversa 2 nodi (si ferma quando incontra 2 o 7) → ??? messaggi
- ID 5 attraversa 1 nodo → 1 messaggio
- ID 7 non va da nessuna parte → 0 messaggi (scartato subito)

3.2.5 Trace Table: AFAIC Esecuzione

Configurazione anello (senso orario): **3** → **7** → **2** → **5** → (torna a 3).

Nota

Regola AFAIC: Un nodo con ID X che riceve un messaggio con ID Y:

- Se $Y > X$ → **Inoltra** Y (potrebbe vincere, è maggiore del mio)
- Se $Y < X$ → **Scarta** Y (non può vincere, il mio ID è maggiore)
- Se $Y = X$ → **LEADER!** (il mio ID ha fatto il giro)

Step 1 — Tutti inviano il proprio ID:

Da → A	Messaggio	Confronto	Risultato
3 → 7	ID=3	$3 < 7$	7 scarta 3 (il suo ID è maggiore)
7 → 2	ID=7	$7 > 2$	2 inoltra 7 (7 è maggiore)
2 → 5	ID=2	$2 < 5$	5 scarta 2 (il suo ID è maggiore)
5 → 3	ID=5	$5 > 3$	3 inoltra 5 (5 è maggiore)

Step 2 — Gli ID sopravvissuti (5 e 7) continuano:

Da → A	Messaggio	Confronto	Risultato
2 → 5	ID=7 (inoltrato)	$7 > 5$	5 inoltra 7
3 → 7	ID=5 (inoltrato)	$5 < 7$	7 scarta 5

Step 3 — Solo l'ID 7 sopravvive:

Da → A	Messaggio	Confronto	Risultato
5 → 3	ID=7	$7 > 3$	3 inoltra 7
3 → 7	ID=7	$7 = 7$	**7 è LEADER!**

Importante

Osservazione: L'ID 7 (il massimo) ha fatto il giro completo dell'anello. Gli altri ID sono stati "uccisi" appena hanno incontrato un ID più grande. Totale: 8 messaggi (non $n^2 = 16$).

3.2.6 Analisi di AFAIC

Caso peggiore: Gli ID sono ordinati in senso **antiorario crescente**

In questo caso:

- L'ID minimo attraversa n nodi
- Il secondo minimo attraversa $n - 1$ nodi
- ...
- L'ID massimo attraversa 1 nodo

$$Message_{worst}(AFAIC) = n + (n-1) + \ldots + 1 = \frac{n(n+1)}{2} = O(n^2)$$

Caso migliore: $O(n)$ (se l'ID minimo è subito "visibile" a tutti)
Caso medio: $O(n \log n)$ (analisi probabilistica)

3.3 Protocollo Controlled Distance (Hirschberg-Sinclair)

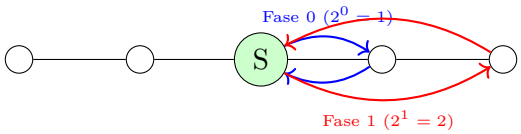
3.3.1 Il Problema di AFAIC

AFAIC può essere $O(n^2)$ nel caso peggiore. Possiamo fare meglio?

3.3.2 L'Idea: Fasi di Distanza Crescente

Invece di mandare l'ID "per sempre", procediamo in **fasi**:

1. **Fase 0**: Ogni candidato manda il suo ID a distanza 1 (ai vicini immediati)
2. **Fase 1**: I sopravvissuti mandano a distanza 2
3. **Fase k**: I sopravvissuti mandano a distanza 2^k



(Sonda avanti e torna indietro)

Regola di sopravvivenza: Un candidato sopravvive alla fase k se il suo ID è il minimo tra tutti quelli che raggiunge.

Pseudocodice Formale (Hirschberg-Sinclair, 1980):

```
STATI: {ASLEEP, CANDIDATE, FOLLOWER, LEADER}
VARIABILI LOCALI:
    phase: fase corrente (inizia da 0)
    limit: distanza di esplorazione = 2^phase
    replies_left: numero di risposte attese
    replies_right: numero di risposte attese

--- INIZIALIZZAZIONE (tutti candidati) ---
```

```
action:
    phase = 0
    limit = 1
    SEND_PROBES()

PROCEDURA SEND_PROBES():
    // Invia sonde in entrambe le direzioni
    send("Forth", id(x), limit, 1) a sinistra
    send("Forth", id(x), limit, 1) a destra
    replies_left = 1
    replies_right = 1

--- RICEZIONE SONDA "FORTH" ---
event: ricevi("Forth", id_sender, ttl, hopcount) da direzione D
  action:
    if id_sender < id(x):
        // Ho visto un ID piu' piccolo: NON sono candidato
        become(FOLLOWER)
        // Ma continuo a propagare la sonda
        if hopcount < ttl:
            send("Forth", id_sender, ttl, hopcount+1) nella stessa dir D
        else:
            send("Back", id_sender) nella direzione opposta
    else if id_sender > id(x):
        // Il mio ID e' piu' piccolo: la sonda muore
        nil
    else:  // id_sender == id(x)
        // La mia sonda e' tornata a me (giro completo)!
        become(LEADER)

--- RICEZIONE "BACK" ---
event: ricevi("Back", id_sender) da direzione D
  action:
    if id_sender == id(x):
        // E' una mia risposta
        if D == LEFT:
            replies_left = replies_left - 1
        else:
            replies_right = replies_right - 1
        CHECK_PHASE()
    else:
        // Inoltra la risposta
        send("Back", id_sender) nella direzione opposta

PROCEDURA CHECK_PHASE():
    if replies_left == 0 AND replies_right == 0:
        // Fase completata con successo!
        phase = phase + 1
        limit = 2^phase
        SEND_PROBES()  // Inizia la fase successiva
```

Nota

Messaggi: Ogni candidato alla fase k invia sonde fino a distanza 2^k in entrambe le direzioni. Poiché ci sono al massimo $n/2^k$ candidati, e ogni candidato genera $4 \cdot 2^k$ messaggi, il totale per fase è $O(n)$. Con $O(\log n)$ fasi, otteniamo $O(n \log n)$ totali.

3.3.3 Perché Funziona?

A ogni fase, almeno metà dei candidati viene eliminata!

Dimostrazione. Se due candidati sono "troppo vicini" (distanza $\leq 2^k$), quello con ID maggiore viene eliminato. Quindi tra ogni coppia di candidati sopravvissuti c'è almeno un candidato eliminato. □

3.3.4 Analisi Approfondita: Il "Budget Costante" (Perché $O(n \log n)$?)

La matematica formale ci dice che la somma è $O(n \log n)$, ma l'intuizione fisica è molto più elegante. Immaginiamo di calcolare il "costo energetico" globale della rete per ogni fase.

In ogni fase avviene uno scambio perfetto tra **quantità di candidati** e **lunghezza del viaggio**.

- **Fase 0 (Tanti e Corti)**:
  - Abbiamo tutti gli n nodi candidati.
  - Ognuno manda sonde cortissime (distanza 1).
  - Costo totale della rete: $n \times 1 = n$ messaggi.
- **Fase 1 (Metà e Doppi)**:
  - Sopravvivono al massimo $n/2$ candidati.
  - Ma devono mandare sonde a distanza doppia (distanza 2).
  - Costo totale: $\frac{n}{2} \times 2 = n$ messaggi.
- **Fase 2 (Pochi e Lunghi)**:
  - Sopravvivono $n/4$ candidati.
  - Sonde a distanza 4.
  - Costo totale: $\frac{n}{4} \times 4 = n$ messaggi.

Importante

Il Principio del Budget Costante: In ogni fase, la rete "spende" complessivamente circa $4n$ messaggi (considerando andata e ritorno e due direzioni). Poco importa se sono "tanti messaggi brevi" o "pochi messaggi lunghi": il prodotto rimane costante ($O(n)$).

Poiché raddoppiamo la distanza a ogni passo, ci servono solo $\log n$ fasi per coprire l'intero anello.

$$\text{Costo Totale} = (\text{Costo per Fase}) \times (\text{Numero Fasi}) = O(n) \times O(\log n) = O(n \log n)$$

Importante

Miglioramento

Da $O(n^2)$ (AFAIC worst case) a $O(n \log n)$ (Controlled Distance sempre)!

3.3.5 Lower Bound $\Omega(n \log n)$ per Ring

Un risultato fondamentale è che Controlled Distance è **asintoticamente ottimale**:

Teorema: Lower Bound per Leader Election (Fredrickson & Lynch, 1987)

Qualsiasi algoritmo di leader election su anello con n nodi richiede almeno $\Omega(n \log n)$ messaggi nel caso peggiore.

Dimostrazione. La dimostrazione usa una tecnica di **costruzione avversariale** e **counting argument**.

Passo 1: Definizione di "arco attivo"

Un arco (u, v) è **attivo** in un'esecuzione se almeno un messaggio lo attraversa (in almeno una direzione). Sia M il numero totale di messaggi: $M \geq$ numero di archi attivi.

Passo 2: Costruzione ricorsiva dell'anello "cattivo"

Costruiamo una famiglia di anelli R_k con $n = 2^k$ nodi, per $k \geq 1$:

- R_1: Anello con 2 nodi, ID $\{1, 2\}$.
- R_k: Costruito da due copie di R_{k-1}, collegate in modo che:
  - Le due copie sono "simmetriche" dal punto di vista locale.
  - Gli ID sono assegnati in modo che nessun nodo può determinare se è nella metà "vincente" o "perdente" senza comunicare con l'altra metà.

Passo 3: Argomento di indistinguibilità

Consideriamo due esecuzioni E_1 e E_2 su anelli che differiscono solo per la posizione del nodo con ID minimo.

Claim: Se un arco (u, v) non è attivo, allora u e v non possono distinguere tra E_1 e E_2.

Conseguenza: Se troppi pochi archi sono attivi, alcuni nodi non possono determinare correttamente se sono leader o follower.

Passo 4: Counting argument

Sia $A(R_k)$ il numero minimo di archi attivi per qualsiasi algoritmo corretto su R_k.

Per la costruzione simmetrica:

$$A(R_k) \geq 2 \cdot A(R_{k-1}) + \frac{n}{2}$$

(Le due metà devono comunicare, forzando almeno $n/2$ archi attivi tra le metà.)

Risolvendo la ricorrenza con $A(R_1) \geq 1$:

$$A(R_k) \geq \frac{n \cdot k}{2} = \frac{n \log_2 n}{2}$$

Conclusione: $M \geq A(R_k) = \Omega(n \log n)$. □

Nota

Intuizione information-theoretic:

- Ci sono $n!$ possibili assegnamenti di ID ai nodi.
- Per determinare il leader, il sistema deve distinguere tra questi $n!$ casi.
- L'informazione necessaria è $\log(n!) = \Theta(n \log n)$ bit.
- Ogni messaggio porta $O(\log n)$ bit di informazione.
- Servono almeno $\Omega(n)$ messaggi... ma l'argomento più raffinato mostra che servono $\Omega(n \log n)$ perché l'informazione deve propagarsi in modo "distribuito".

Domanda Killer: Ottimalità di HS

Il protocollo Hirschberg-Sinclair (Controlled Distance) è **ottimale a meno di fattori costanti**: usa $O(n \log n)$ messaggi e il lower bound dice che non si può fare meglio di $\Omega(n \log n)$.

Algoritmo	**Messaggi**	**Ottimale?**
All-the-Way	$O(n^2)$	No
AFAIC/LCR	$O(n^2)$ worst, $O(n \log n)$ avg	No (worst case)
Controlled Distance (HS)	$O(n \log n)$ sempre	**Sì!**
Lower Bound	$\Omega(n \log n)$	—

3.3.6 Visual Walkthrough: Controlled Distance in Azione

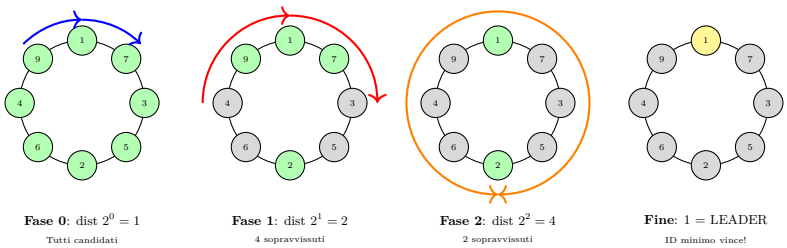


Figura 3: Controlled Distance: a ogni fase i candidati si dimezzano. Dopo $O(\log n)$ fasi resta solo l'ID minimo.

3.4 Protocollo FloodMax (Sincrono)

Per **grafi generici** (non solo anelli o alberi), esiste un protocollo semplice ed efficace se il sistema è **sincrono** e si conosce il **diametro** D del grafo (o un upper bound).

Idea: Flooding del massimo ID. In ogni round, ogni nodo propaga ai vicini il massimo ID che ha visto finora.

```
Inizializzazione:
    max_seen = id(x)
    round = 0

Ad ogni round (fino a D):
    1. Invia max_seen a tutti i vicini
    2. Ricevi messaggi dai vicini
    3. max_seen = max(max_seen, ID ricevuti)
    4. round++

Dopo D round:
    Se max_seen == id(x) -> LEADER
    Altrimenti -> FOLLOWER
```

Perché funziona?

- L'ID massimo nel sistema si propaga di almeno 1 hop per round (come un'onda).
- Dopo D round, l'ID massimo avrà raggiunto tutti i nodi del grafo, anche quello più lontano.
- Tutti avranno `max_seen` uguale all'ID massimo globale.

Complessità:

- **Messaggi**: $O(m \times D)$ (in ogni round si attraversano tutti gli archi, $2m$ messaggi per round)
- **Tempo**: $O(D)$ round

Nota

Vantaggio: Funziona su qualsiasi topologia. **Svantaggio**: Richiede sincronia e conoscenza di D.

3.5 Leader Election su Grafi Generici

3.5.1 L'Approccio: Spanning Tree + Saturation

Per grafi che non sono anelli, combiniamo tecniche già viste:

1. **Costruisci uno spanning tree** (SHOUT+ $\rightarrow 2m$ messaggi)
2. **Usa saturation** per trovare il minimo ID ($\leq 4n$ messaggi)
3. Il nodo con ID minimo è il leader

Complessità totale: $O(m + n)$

3.5.2 Altri Protocolli: Mega-Merger

Un altro protocollo per leader election su grafi generici è **Mega-Merger**, che costruisce uno spanning tree in modo efficiente e restituisce la radice come leader.

Protocollo	Message Complexity
ST + Saturation	$O(m+n)$
YO-YO	$O(m \log n)$
Mega-Merger	$O(m + n \log n)$

Nota

YO-YO è un algoritmo elegante per grafi generici che trasforma il grafo in un DAG orientando gli archi dal nodo con ID minore verso quello con ID maggiore. Opera in iterazioni: nella fase **YO-** (discendente) le sorgenti propagano l'ID minimo verso i sink; nella fase **-YO** (ascendente) i sink votano YES/NO e gli archi vengono "flippati" in base ai voti. Ad ogni iterazione almeno una sorgente viene eliminata. Complessità: $O(m \log n)$ messaggi.

Mega-Merger è un protocollo ancora più complesso con complessità $O(m + n \log n)$, competitivo per grafi sparsi.

3.6 Sistemi Sincroni: Il Potere del Silenzio

3.6.1 Il Silenzio è Informazione

Nei sistemi **sincroni**, il tempo è diviso in round precisi. Questo ci dà un superpotere che non abbiamo nei sistemi asincroni: possiamo comunicare **non facendo nulla**.

Analogia del Mondo Reale: L'Asta Silenziosa

Immagina un'asta dove vince chi ha il numero più basso.

- **Approccio Asincrono (Gridare)**: Tutti devono urlare il proprio numero. Il banditore deve ascoltare tutti e 100 i partecipanti per trovare il minimo. (Tanti messaggi).
- **Approccio Sincrono (Aspettare)**: Il banditore dice: "Contiamo i secondi. Se il tuo numero corrisponde ai secondi passati, alza la mano".
  - 0... nessuno.
  - 1... nessuno.
  - 2... nessuno.
  - 3... Il tizio col 3 alza la mano!

  Risultato: Abbiamo trovato il minimo (3) con **un solo gesto** (messaggio). I secondi di silenzio (0, 1, 2) hanno trasmesso l'informazione "nessuno ha 0, 1 o 2".

3.6.2 Protocollo "Universal Waiting" (Time-Slice)

Questo protocollo porta l'analogia all'estremo. Vogliamo eleggere il nodo con ID minimo usando il minor numero possibile di messaggi.

Algoritmo:

1. Ogni nodo x calcola un tempo di attesa proporzionale al proprio ID: $Wait = ID(x) \times$ costante.

2. Il nodo aspetta in silenzio.
3. Se il timer scade e non ha ricevuto nulla: **Si dichiara LEADER** e invia "Elected" agli altri.
4. Se riceve "Elected" mentre aspetta: Diventa FOLLOWER e si ferma.

Il Trade-off Estremo:

- **Messaggi**: Ottimo! $n - 1$ (o anche meno, basta che il leader raggiunga tutti).
- **Tempo**: Pessimo! Dipende dal valore dell'ID minimo. Se il minimo è 1.000.000, aspettiamo un'eternità.

Questo dimostra che nei sistemi sincroni possiamo scambiare **Tempo** per **Messaggi**.

3.6.3 Protocollo Speeding

Una variante più pratica: invece di aspettare prima di partire, i messaggi partono subito ma viaggiano a velocità diverse.

- ID piccoli → Ferrari (viaggiano veloci, processati subito).
- ID grandi → Trattori (vengono trattenuti dai nodi per un po' prima di essere inoltrati).

L'effetto è lo stesso: l'ID piccolo fa il giro e "uccide" gli ID grandi che sono ancora bloccati nel traffico (trattenuti dai nodi). Risparmiamo i messaggi degli ID grandi che non faranno mai molta strada.

3.7 Algoritmi Randomizzati: Rompere la Simmetria senza ID

3.7.1 L'Idea

Se non abbiamo ID univoci, possiamo usare la **casualità** per rompere la simmetria!

3.7.2 Due Tipi di Algoritmi Randomizzati

Tipo	Correttezza	Terminazione
Monte Carlo	Probabilistica (potrebbe sbagliare)	Garantita
Las Vegas	**Garantita** (corretto sempre)	Probabilistica (tempo variabile)

Attenzione

Per la leader election, preferiamo **Las Vegas**: un leader "sbagliato" è inaccettabile!
Un algoritmo che a volte elegge 0 o 2 leader non è utile.

3.7.3 Un Protocollo Las Vegas

Setup: Anello sincrono, nodi senza ID, tutti iniziatori
Ogni round:

1. Ogni candidato sceglie un bit casuale $b \in \{0, 1\}$
2. Tutti annunciano il proprio bit

3. Se esiste un **unico** nodo con bit "diverso" (es. unico 1 tra tutti 0), sopravvive
4. Altrimenti, tutti sopravvivono e si ripete
5. Se resta un solo candidato → è il LEADER

3.7.4 Ottimizzazione: Probabilità $p = 1/n$

Invece di $p = 1/2$, scegliamo $p = 1/n$ (probabilità di scegliere 1).

Perché?

Vogliamo massimizzare la probabilità che *esattamente uno* scelga 1:

$$P(\text{esattamente uno sceglie 1}) = n \cdot p \cdot (1-p)^{n-1}$$

Derivando e ponendo $= 0$, troviamo $p^* = 1/n$.

Con $p = 1/n$ e n grande:

$$P(\text{successo per round}) \approx \frac{1}{e} \approx 0.368$$

Numero atteso di round: $e \approx 2.7$ (circa 3 round!)

Complessità:

- Messaggi attesi: $O(n)$
- Tempo atteso: $O(n)$ o $O(D)$

3.8 Riepilogo: Quale Algoritmo Usare?

Condizioni	Algoritmo	Messaggi
Anello, ID univoci	All-the-way	$O(n^2)$
Anello, ID univoci	AFAIC	$O(n^2)$ worst, $O(n)$ best
Anello, ID univoci	Controlled Distance	$O(n \log n)$
Grafo generico, Sincrono	FloodMax	$O(m \cdot D)$
Grafo generico	ST + Saturation	$O(m + n)$
Grafo generico	Mega-Merger	$O(m + n \log n)$
Senza ID	Las Vegas	$O(n)$ expected
Anonimo, deterministico	**IMPOSSIBILE**	-

Key Takeaways

1. **Angluin (1980)**: Impossibile eleggere leader in anello anonimo deterministico
2. **Anello**: HS ($O(n \log n)$) è ottimale. AFAIC è $O(n^2)$ worst case.
3. **Grafi Generici**:
   - **FloodMax** (Sincrono): $O(m \cdot D)$
   - **Mega-Merger**: $O(m + n \log n)$
4. **Randomizzato**: Senza ID, probabilità $p = 1/n$ ottimale, ~ 2.7 round attesi

Errori Comuni da Evitare

- **Angluin**: Vale solo per anelli anonimi con start simultaneo, non per tutti i grafi!
- **AFAIC worst case**: Gli ID ordinati in senso antiorario crescente
- **Sincrono $\neq$ Asincrono**: Nel sincrono il silenzio è informazione, nell'asincrono no
- **Las Vegas vs Monte Carlo**: Per leader election, preferire sempre Las Vegas (corretto!)

3.9 Esercizi e Domande

3.9.1 Domande Concettuali

1. Spiega con parole tue perché Angluin dimostra l'impossibilità.
2. Cosa succederebbe se in AFAIC ti arrivasse il tuo stesso ID?
3. Perché Controlled Distance garantisce che almeno metà dei candidati viene eliminata ogni fase?
4. In un sistema sincrono, perché "non ricevere nulla" è informazione utile?
5. Perché $p = 1/n$ è la probabilità ottimale nell'algoritmo randomizzato?
6. Differenza tra Monte Carlo e Las Vegas - quale preferire per leader election?

3.9.2 Esercizi di Dimostrazione

1. **Enuncia e dimostra il teorema di Angluin**: perché è impossibile eleggere un leader in un anello anonimo?
2. **Descrivi il protocollo AFAIC** e analizza la complessità nel caso peggiore e medio.
3. **Come funziona Controlled Distance**? Dimostra perché è $O(n \log n)$ usando il principio del "budget costante".
4. **Descrivi il protocollo Universal Waiting** e spiega il trade-off tempo/messaggi.
5. **Dimostra il lower bound** $\Omega(n \log n)$ per leader election su anello.

Tracce di Soluzione

Soluzioni Domande Concettuali:

1. **Angluin**: In anello anonimo con start simultaneo, per simmetria tutti i nodi sono indistinguibili. Stesso stato iniziale, stessi messaggi ricevuti $\Rightarrow$ stesso comportamento. Se uno dice "Leader", tutti lo dicono.
2. **AFAIC - proprio ID**: Se ricevo il mio stesso ID, ha fatto il giro completo dell'anello senza essere bloccato da ID più piccoli $\Rightarrow$ io ho l'ID minimo $\Rightarrow$ LEADER!
3. **HS metà eliminati**: Due candidati a distanza $\leq 2^k$ si "vedono" nella fase k. Quello con ID maggiore viene eliminato. Tra ogni coppia di sopravvissuti c'è almeno un eliminato $\Rightarrow$ sopravvive $\leq$ metà.
4. **Sincrono silenzio**: In sistemi sincroni, se non ricevo entro timeout Δ, so per certo che nessun messaggio è stato inviato. Posso "contare" i tick di silenzio.
5. **p=1/n**: Massimizza $P(\text{esattamente 1 sceglie 1}) = n \cdot p \cdot (1-p)^{n-1}$. Derivando: $p^* = 1/n$.
6. **Las Vegas vs MC**: Las Vegas corretto sempre, termina prob. 1. Monte Carlo termina sempre, corretto con probabilità. Per LE serve correttezza assoluta $\Rightarrow$ Las Vegas.

Soluzioni Esercizi di Dimostrazione:

1. **Angluin dimostrazione**: Per simmetria, tutti i nodi fanno le stesse transizioni. Se uno diventa leader, tutti lo diventano. Contraddizione.
2. **AFAIC complessità**: Worst case $O(n^2)$ (ID ordinati antiorario crescenti). Medio $O(n \log n)$.
3. **Budget costante**: Fase k: $n/2^k$ candidati $\times$ distanza $2^k = O(n)$ msg/fase. Fasi: $\log n$. Totale: $O(n \log n)$.
4. **Universal Waiting**: Aspetto $f(ID)$ tick prima di dichiararmi. Trade-off: pochi msg ($O(n)$), tanto tempo (dipende da ID minimo).
5. **Lower bound**: Argomento information-theoretic. Costruzione avversariale di anelli dove ogni nodo deve ricevere $\Theta(\log n)$ messaggi per distinguersi.

4 Routing Distribuito

Prerequisiti

- Capitolo 2: Modello distribuito, comunicazione via messaggi
- Algoritmo di Dijkstra centralizzato (per confronto)
- Concetto di cammino minimo in un grafo pesato

Problemi
- **SPST**: Single-Source
- **APSP**: All-Pairs

Link State
- Map Gossip $O(mn)$
- Dijkstra Distr. $O(n^3)$

Scambia: topologia

Distance Vector
- Bellman-Ford
- Count-to-∞

Scambia: distanze

In questo capitolo affrontiamo il problema di come i messaggi trovano la strada in una rete. Vedremo due approcci fondamentali e i loro trade-off.

4.1 Il Problema: Come Arriviamo a Destinazione?

4.1.1 Cosa Significa "Routing"?

Quando invii un'email, il messaggio deve attraversare molti computer per arrivare a destinazione. Ma come fa ogni computer a sapere "dove mandarlo dopo"?

Definizione: Routing

Il **routing** è il problema di far arrivare un messaggio da una sorgente s a una destinazione d seguendo il **cammino più breve**.

4.1.2 Due Approcci al Routing

Approccio	Come Funziona
Source Routing	La sorgente decide l'intero percorso e lo inserisce nel messaggio. Ogni nodo legge "vai al prossimo nella lista".
Hop-by-Hop	Ogni nodo decide solo il **prossimo passo**. Il messaggio dice solo "voglio arrivare a d", e ogni nodo consulta la sua tabella.

I sistemi distribuiti usano Hop-by-Hop perché:

- La sorgente non conosce l'intera rete
- La rete può cambiare durante il viaggio del messaggio

Analogia del Mondo Reale: Il Viaggio in Auto

Source Routing è come stampare le indicazioni da Google Maps prima di partire: "Gira a destra, poi a sinistra, poi dritto". Se c'è un incidente a metà strada, le istruzioni sono inutili. **Hop-by-Hop** è come seguire i cartelli stradali: a ogni incrocio decidi dove andare in base alla destinazione. Se c'è un blocco, segui una deviazione locale.

4.1.3 La Routing Table

Ogni nodo mantiene una **tabella di routing**:

Destinazione	Next Hop	Distanza
Nodo A	Vicino 1	3
Nodo B	Vicino 2	5
Nodo C	Vicino 1	2

Lettura: "Per raggiungere il Nodo A, manda al Vicino 1, che è a distanza 3 da A".

Nota

Costi della Routing Table:

- **Routing Memory**: $O(n \log w)$ bit, dove w = ID massimo (ogni entry ha destinazione + next hop + costo)
- **Search Time**: $O(\log n)$ con ricerca binaria sulla tabella ordinata

4.2 Il Principio di Ottimalità: Perché Basta il Next Hop

4.2.1 Il Teorema di Bellman

Teorema: Principio di Ottimalità

Se il cammino minimo da A a C passa per B, allora il sottocammino da B a C è anch'esso il cammino minimo da B a C.

Perché è vero? Per assurdo: se esistesse un cammino più corto da B a C, potremmo sostituirlo nel cammino $A \to C$ e ottenere un cammino ancora più corto, contraddicendo l'ottimalità. □

4.2.2 L'Implicazione Pratica

Importante

Non serve memorizzare l'intero cammino!

Se conosco il **primo passo** del cammino ottimo verso d, posso mandare il messaggio a quel vicino, che a sua volta conosce il *suo* primo passo, e così via.

Il messaggio "rimbalza" di nodo in nodo, ma ogni rimbalzo è sulla strada ottima!

4.3 Map Gossip vs Bellman-Ford: Due Filosofie a Confronto

Prima di scendere nei dettagli, capiamo la differenza profonda tra i due approcci con un'analogia quotidiana.

Map Gossip (Link State)	Bellman-Ford (Distance Vector)
Come usare Google Maps: Hai scaricato l'intera mappa della città sul tuo telefono. Vedi ogni incrocio e ogni via. **Calcolo**: Se una strada chiude, ricalcoli il percorso da solo (Dijkstra) perché conosci l'intera topologia. **Costo**: Tanta memoria (devi tenere tutta la mappa) ma poche sorprese.	**Come chiedere indicazioni**: Non hai una mappa. Ti fermi a ogni incrocio e chiedi ai passanti: "Quanto manca per il centro?". **Calcolo**: Se il passante ti dice "10 minuti", tu aggiungi il tempo per arrivare da lui e aggiorni la tua stima. **Costo**: Pochissima memoria (solo una lista di distanze) ma rischi di seguire informazioni obsolete.

4.4 Approccio 1: Map Gossip

4.4.1 L'Idea: Condividiamo Tutto

Filosofia: Ogni nodo condivide la sua conoscenza dei vicini con tutti. Alla fine, tutti conoscono l'intera mappa della rete!

Procedura:

1. Costruisci uno **spanning tree**
2. Ogni nodo manda le info sui propri vicini a tutti (via spanning tree)
3. Ogni nodo ora ha la **mappa completa**
4. Ogni nodo calcola localmente i cammini minimi (es. algoritmo di Dijkstra)

4.4.2 Complessità di Map Gossip

Messaggi: Ogni nodo manda info sui suoi $deg(x)$ vicini a tutti gli n nodi. **Messaggi**: Ogni nodo manda info sui suoi $deg(x)$ vicini a tutti gli n nodi.

$$Message(MapGossip) = \sum_x deg(x) \times O(n) = 2m \times n = O(mn)$$

Memoria per nodo: Deve memorizzare l'intera mappa.

$$Space = O(n + m) \text{ per nodo}$$

4.4.3 Pro e Contro

Pro	Contro
Vista globale della rete	Memoria alta
Routing sempre corretto	Molti messaggi da scambiare
Funziona con metriche complesse	Poco scalabile

4.5 Approccio 2: Bellman-Ford Distribuito

4.5.1 L'Idea: Condividiamo Solo le Distanze

Filosofia: Non serve conoscere l'intera mappa! Basta conoscere le **distanze** verso ogni destinazione.

Ogni nodo mantiene un **Distance Vector**:

$$DV_x = [d(x,1), d(x,2), \ldots, d(x,n)]$$

dove $d(x,i)$ è la distanza stimata da x a i.

4.5.2 L'Equazione di Bellman-Ford

Importante

L'Equazione Chiave

Per ogni destinazione y:

$$d(x,y) = \min_{v \in N(x)} \{w(x,v) + d(v,y)\}$$

Lettura: La distanza da x a y è il minimo tra:

- Costo per andare al vicino v (che è $w(x,v)$)
- PIÙ la distanza da v a y (che v ci comunica)

4.5.3 Il Protocollo

```
Inizializzazione:
    d(x,x) = 0
    d(x,y) = infinito per ogni y != x
    next_hop(x,y) = null

Ripeti fino a convergenza:
    Invia DV_x a tutti i vicini
    Per ogni DV_v ricevuto dal vicino v:
        Per ogni destinazione y:
            nuova_distanza = w(x,v) + d(v,y)
            se nuova_distanza < d(x,y):
                d(x,y) = nuova_distanza
                next_hop(x,y) = v
                d(x,y) = nuova_distanza
                next_hop(x,y) = v
```

4.5.4 Esempio Passo-Passo

Consideriamo una rete:

A —1— B —2— C

Inizialmente:

	d(· ,A)	d(· ,B)	d(· ,C)
A	0	∞	∞
B	∞	0	∞
C	∞	∞	0

Dopo Round 1:

	d(· ,A)	d(· ,B)	d(· ,C)
A	0	1	∞
B	1	0	2
C	∞	2	0

Dopo Round 2:

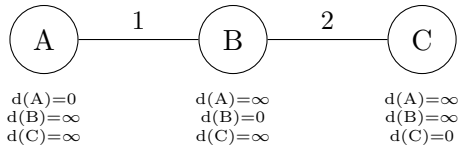
	d(· ,A)	d(· ,B)	d(· ,C)
A	0	1	**3**
B	1	0	2
C	**3**	2	0

A scopre che può raggiungere C via B con costo $1 + 2 = 3$!

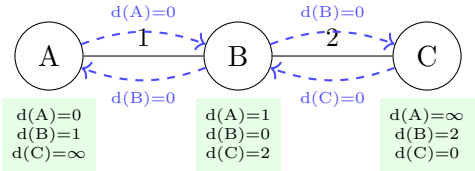
4.5.5 Visual Walkthrough: Convergenza Bellman-Ford

Vediamo visivamente come si propagano le informazioni.

Round 0



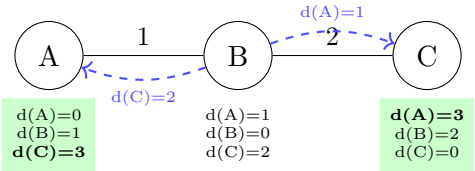
Round 1



Spiegazione:

- **R1**: I vicini diretti si scoprono. A vede B (costo 1). B vede C (costo 2).
- **R2**: B annuncia a A che sa raggiungere C in 2 hop. A calcola: costa 1 per andare a B + 2 per C = 3.
- **Convergenza**: A ha scoperto C (indiretto).

Round 2



4.5.6 Complessità di Bellman-Ford

Iterazioni: Al più $n - 1$ (il diametro della rete)

Messaggi per iterazione: Ogni nodo manda il suo DV a ogni vicino $= 2m$

$$Message(BF) = O(n) \times O(m) = O(nm)$$

Ma ogni DV ha n entry, quindi:

$$Message(BF) = O(n^2 m) \text{ (contando la dimensione dei messaggi)}$$

Memoria per nodo: Solo il Distance Vector

$$Space = O(n) \text{ per nodo}$$

4.6 Min-Hop Routing (BFS Distribuita)

Se i link non hanno costi (o costi tutti uguali), il problema del routing diventa trovare il cammino con il **minor numero di hop**. Questo equivale a costruire un **Breadth-First Spanning Tree** (BFST).

4.6.1 L'Algoritmo Distribuito

L'algoritmo procede per **livelli** (iterazioni). All'iterazione i, scopriamo tutti i nodi a distanza esatta i dalla sorgente.

Procedura:

1. **Start**: La radice (sorgente) avvia l'iterazione i.
2. **Broadcast**: Il messaggio "Start i" scende lungo l'albero parziale già costruito (nodi a distanza $< i$).
3. **Esplorazione**: I nodi al livello $i - 1$ (foglie correnti dell'albero) esplorano i loro vicini inviando "Explore i".
4. **Risposta**:
   - Se un vicino non è ancora nell'albero, risponde **ACK-OK** (diventa figlio).
   - Se è già nell'albero, risponde **ACK-KO**.
5. **Convergecast**: Una volta ricevute tutte le risposte, i risultati risalgono alla radice. Se non ci sono nuovi nodi, l'algoritmo termina.

4.6.2 Complessità Min-Hop

Messaggi per iterazione i:

1. Broadcast nell'albero parziale: $O(N_i)$ messaggi (dove N_i è il numero di nodi scoperti finora).
2. Explore sui nuovi link: Ogni arco viene esplorato esattamente una volta in tutto l'algoritmo.
3. Convergecast: $O(N_i)$.

Totale: Sommando su tutte le iterazioni (pari al diametro D):

$$\text{Messaggi} = O(n \cdot D + m)$$

Se $D \approx n$, allora $O(n^2)$. Se D è piccolo, è molto efficiente. In dettaglio: $2(n-1)D + 2m \in O(n^2)$.

Tempo: Anche il tempo è dominato dal diametro: $O(D^2)$ (poiché ogni iterazione impiega tempo proporzionale all'altezza dell'albero corrente).

4.7 Distributed Dijkstra (Shortest Path Spanning Tree)

Per grafi con **pesi positivi**, possiamo adattare l'algoritmo di Dijkstra in modo distribuito.

4.7.1 L'Idea

Come nell'algoritmo sequenziale, costruiamo l'SPST aggiungendo **un nodo alla volta**: quello più vicino alla sorgente tra quelli non ancora esplorati.

Iterazione: 1. La radice chiede a tutti i nodi nell'albero corrente di trovare il loro "miglior arco uscente" (quello che minimizza $dist(root, u) + w(u, v)$). 2. Ogni nodo nell'albero calcola il minimo locale e lo invia su (Convergecast). 3. La radice sceglie il minimo globale. 4. Il nuovo nodo viene aggiunto all'albero (Notifica).

4.7.2 Complessità

- **Messaggi**: $O(n^2)$ (per costruire un singolo albero).
- **Nota**: Se volessimo costruire gli alberi per *tutti* i nodi (All-Pairs Shortest Path), servirebbero $O(n^3)$ messaggi.

Confronto:

- **Map Gossip**: $O(mn)$ per tutti i nodi.
- **Distributed Dijkstra**: $O(n^3)$ per tutti i nodi.

Per grafi sparsi ($m \approx n$), Gossip vince (n^2 vs n^3). Per grafi densi ($m \approx n^2$), sono equivalenti (n^3).

4.8 Il Problema del Count-to-Infinity (Approfondimento)

4.8.1 Cosa Succede Quando le Cose Vanno Male?

Map Gossip e Bellman-Ford funzionano benissimo... finché la rete non **cambia**.

Attenzione

Nota dalle Slide Le slide correnti accennano al problema della manutenzione delle tabelle in caso di cambiamenti (costoso!), ma non entrano nei dettagli del Count-to-Infinity. Questa sezione è un approfondimento standard.

Il Count-to-Infinity: Quando un link **fallisce** (il costo diventa ∞), Bellman-Ford può entrare in un loop infinito!

4.8.2 Il Count-to-Infinity: Il Problema dell'Eco

Il "Count-to-Infinity" è uno dei fenomeni più famosi (e fastidiosi) delle reti distribuite. Per capirlo senza impazzire tra i numeri, dobbiamo capire la sua natura logica.

Importante

L'essenza del problema: Il loop nasce quando un nodo **crede a una voce che parla di lui stesso**.

Nell'esempio precedente:

1. B sa arrivare ad A (distanza 1).
2. C sa arrivare ad A **tramite B** (distanza 2).
3. Quando A scompare, B cerca disperatamente un'altra strada.
4. C dice: "Ehi, io so arrivare ad A in 2 passi!".
5. B si dimentica che C intendeva "2 passi passando da te" e pensa: "Ottimo! Allora io ci arrivo in $2 + 1 = 3$ passi passando da C".

B sta seguendo un'informazione che lui stesso ha generato in passato. È come un gatto che si rincorre la coda.

4.8.3 Esempio del Problema

Rete originale: A —1— B —1— C
Distanze iniziali corrette:

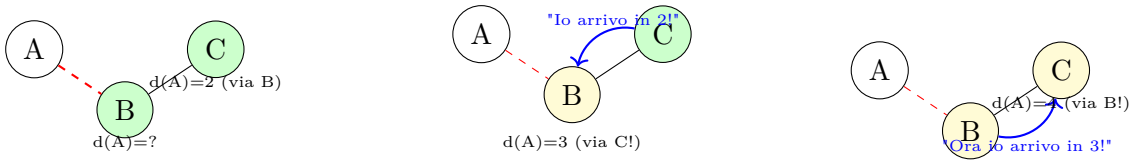
- B conosce: d(B,A) = 1, next_hop = A
- C conosce: d(C,A) = 2, next_hop = B

Ora il link A-B si rompe!

1. B non può più raggiungere A direttamente
2. B chiede a C: "Quanto disti da A?" C risponde: 2
3. B pensa: "Posso raggiungere A via C con costo 1+2=3"
4. B dice a C: "La mia distanza da A è 3"
5. C pensa: "Posso raggiungere A via B con costo 1+3=4"
6. C dice a B: "La mia distanza da A è 4"
7. B pensa: "Via C costa 1+4=5"
8. ...

Le distanze crescono all'infinito perché B e C si "rimbalzano" informazioni obsolete!

4.8.4 Visual Walkthrough: Il Bounce Disastroso



(a) Step 1: Link rotto. B perde A. C sa ancora "2".

(b) Step 2: C "aiuta" B. B setta d=3.

(c) Step 3: B aggiorna C. C setta d=4.

Figura 4: Il "Rimbalzo": B crede a C, poi C crede a B, e il costo sale all'infinito.

4.8.5 Trace Table: Count-to-Infinity

Scenario: Rete lineare **A** –1– **B** –1– **C**. Prima del guasto, B raggiunge A con costo 1, C raggiunge A con costo 2 (via B).

Nota

Poi il link A-B si rompe! B non può più raggiungere A direttamente... ma C non lo sa ancora e crede ancora di poter raggiungere A via B.

Step 0 — Stato Iniziale (prima del guasto):

Nodo	dist(A)	via
B	1	diretto ad A
C	2	via B

Step 1 — Il link A-B si rompe:

- B rileva il guasto: `dist(A) = ∞` (non ho più via diretta)
- C non sa nulla del guasto, crede ancora: `dist(A) = 2 via B`

Step 2 — C annuncia a B (C non sa del guasto!):

- C dice: "Io raggiungo A con costo 2"
- B pensa: "Fantastico! Passo da C: costo = 2 + 1 = 3"
- B aggiorna: `dist(A) = 3 via C` ← **ERRORE!**

Step 3 — B annuncia a C:

- B dice: "Io raggiungo A con costo 3"
- C pensa: "Strano, prima era 2... aggiorno: costo = 3 + 1 = 4"
- C aggiorna: `dist(A) = 4 via B`

Step 4+ — Il loop continua all'infinito:

Step	B: dist(A)	C: dist(A)
2	3	2
3	3	4
4	5	4
5	5	6
...	$\to \infty$	$\to \infty$

Attenzione

Il Problema: B e C si "rimbalzano" informazioni obsolete. Ognuno crede che l'altro abbia una via per A, ma nessuno dei due ce l'ha! I costi crescono verso l'infinito.

4.8.6 Soluzioni al Count-to-Infinity

Esistono diverse tecniche per mitigare questo problema:

1. Split Horizon

Definizione: Split Horizon

Non annunciare a un vicino una rotta che **passa per quel vicino**.

Analogia del Mondo Reale: Non vendermi i miei stessi segreti

Se io ho insegnato a te la strada per andare al mare, e tu vieni da me e mi dici: "Ehi, so una strada per il mare che ci mette 10 minuti", io dovrei chiederti: "Passa da me?". Se la risposta è sì, non dirmelo nemmeno! Io quella strada la conosco già, anzi, sono io che l'ho spiegata a te.

Esempio: Se C raggiunge A passando per B, C **non** dice a B "d(C,A) = 2 via B".

Perché funziona? Se C non annuncia la rotta a B, B non può "rimbalzare" l'informazione obsoleta.

2. Poison Reverse (Split Horizon with Poisoned Reverse)

Definizione: Poison Reverse

Annuncia esplicitamente distanza ∞ per le rotte che passano per il destinatario.

Esempio: C dice a B: "d(C,A) = ∞" (anche se C raggiunge A via B con costo 2).

Vantaggio: Più aggressivo di Split Horizon. B riceve esplicitamente l'informazione che C non ha rotte alternative.

Tecnica	Comportamento
Split Horizon	Ometti le rotte che passano per il destinatario
Poison Reverse	Annuncia ∞ per le rotte che passano per il destinatario

3. Triggered Updates

Definizione: Triggered Updates

Invece di aspettare il prossimo ciclo di aggiornamento, invia **immediatamente** un update quando una rotta cambia significativamente.

Vantaggio: Le "cattive notizie" si propagano più velocemente.

4. Hold-Down Timer

Quando una rotta diventa irraggiungibile, il nodo la "congela" per un certo tempo, ignorando aggiornamenti che potrebbero provenire da informazioni obsolete.

Attenzione

Limite delle soluzioni

Split Horizon e Poison Reverse risolvono solo i loop tra **2 nodi**!

Per loop più lunghi (A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ A), servono meccanismi più sofisticati come TTL (Time To Live) o protocolli Link-State.

Nota

Buone notizie vs Cattive notizie

- **Buone notizie** (nuova rotta più corta): Si propagano velocemente! Ogni nodo adotta subito il miglioramento.
- **Cattive notizie** (rotta rotta): Si propagano lentamente. Count-to-infinity prima di convergere.

4.9 Gestione dei Cambiamenti (Dinamicità)

Cosa succede quando la rete cambia (es. costi dei link variano o link falliscono)?

4.9.1 Map Update (Link State)

Poiché ogni nodo possiede la mappa completa (come in Map Gossip), la gestione è "semplice" ma costosa in termini di comunicazione.

- Quando un nodo rileva un cambiamento locale, aggiorna la sua mappa.

- Invia un messaggio "Update" in broadcast a tutti (Flooding).
- Tutti i nodi aggiornano la mappa e ricalcolano le rotte (Dijkstra).

Pro: Adattivo e robusto. **Contro**: Alto volume di traffico.

4.9.2 Distance Vector Update

Qui sorgono i problemi veri.

- Se i costi cambiano, i nodi devono ricalcolare i vettori.
- Questo può richiedere molte iterazioni (vedi problema Count-to-Infinity nell'approfondimento).
- È computazionalmente costoso e lento a convergere.

Nota

Conclusione delle Slide: Mantenere routing tables ottime in reti dinamiche è molto costoso. Un'alternativa (non trattata in dettaglio) è rilassare il requisito di "cammino ottimo" e accontentarsi di un cammino "buono" (Min-Hop o altro).

4.10 Riepilogo

Metodo	Messaggi	Memoria	Note
Map Gossip	$O(mn)$	$O(n+m)$	Mappa completa
Bellman-Ford	$O(n^2m)$	$O(n)$	Solo distanze
Min-Hop (BFS)	$O(n \cdot D + m)$	$O(n)$	Solo hop count
Dijkstra Distr. (1 SPST)	$O(n^2)$	$O(n)$	Pesi positivi
Dijkstra Distr. (tutti)	$O(n^3)$	$O(n)$	Costoso

Nota

Trade-off Gossip vs SPST: Map Gossip costruisce **tutte** le routing table in $O(mn)$ messaggi. Dijkstra Distribuito costruisce **un** SPST in $O(n^2)$, ma per tutti servono $O(n^3)$. Per grafi sparsi ($m \approx n$), Gossip è migliore!

Key Takeaways

1. **Principio di ottimalità**: Basta memorizzare il next hop.
2. **Map Gossip**: $O(mn)$ msg — Mappa completa, Dijkstra locale.
3. **Bellman-Ford**: $O(n^2m)$ bit — Solo vettori distanze, iterativo.
4. **Min-Hop (BFS)**: $O(n^2)$ msg — Per reti non pesate, usa iterazioni a livelli.
5. **Distributed Dijkstra**: $O(n^2)$ msg per 1 albero — Costruisce l'SPST nodo per nodo.

Errori Comuni da Evitare

- **Dijkstra Distribuito**: Efficiente per un solo albero, ma costoso ($O(n^3)$) se tutti devono farlo.
- **Count-to-infinity**: Problema specifico di Bellman-Ford (Distance Vector).
- **Dinamicità**: Map Update è robusto ma floodoso; DV Update è lento a convergere.

4.11 Esercizi e Domande

4.11.1 Domande Concettuali

1. Perché il principio di ottimalità ci permette di memorizzare solo il next hop?
2. Qual è la differenza fondamentale tra Map Gossip e Bellman-Ford?
3. Come funziona l'algoritmo Min-Hop (BFS Distribuita)?
4. Perché Distributed Dijkstra aggiunge un solo nodo alla volta?
5. In che situazione Map Gossip è preferibile a Distributed Dijkstra?
6. Cos'è il Count-to-Infinity? Come si risolve?

4.11.2 Esercizi di Dimostrazione

1. **Enuncia il Principio di Ottimalità** e dimostra perché basta memorizzare il next hop.
2. **Descrivi Map Gossip**: fasi, complessità, pro/contro.
3. **Scrivi l'equazione di Bellman-Ford** distribuita e mostra un esempio di convergenza.
4. **Analizza il Count-to-Infinity**: fai un esempio e spiega Split Horizon e Poisoned Reverse.
5. **Confronta Distance Vector e Link State**: vantaggi, svantaggi, protocolli reali.

Tracce di Soluzione

Soluzioni Domande Concettuali:

1. **Principio Ottimalità**: Sottocammini di cammini ottimi sono ottimi. Dimostrazione per assurdo: se esistesse sottocammino migliore, potrei sostituirlo e ottenere cammino totale migliore.
2. **Map Gossip vs BF**: Gossip scambia topologia completa, ogni nodo fa Dijkstra locale. BF scambia solo vettori distanze, iterativo.
3. **Min-Hop**: Iterazioni a livelli. Radice avvia, foglie esplorano, risultati risalgono. $O(n \cdot D + m)$ messaggi.
4. **Dijkstra un nodo**: Simula versione sequenziale. Convergecast per trovare arco minimo uscente, broadcast per notificare.
5. **Gossip vs Dijkstra**: Per tutti i nodi: Gossip $O(mn)$, Dijkstra $O(n^3)$. Per grafi sparsi ($m \approx n$), Gossip vince.
6. **Count-to-Infinity**: Loop routing per info obsolete. Split Horizon: non annuncio rotta a chi la usa per me. Poisoned Reverse: annuncio ∞.

Soluzioni Esercizi di Dimostrazione:

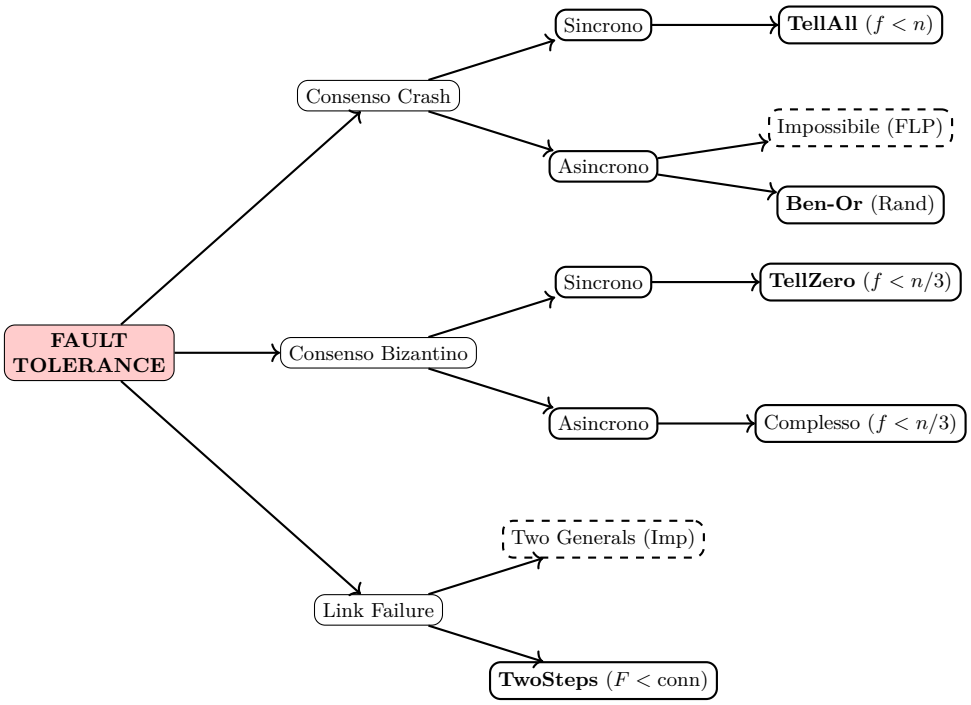
1. **Principio Ottimalità dimostrazione**: Per assurdo. Se esistesse sottocammino P2 migliore di P1 (parte di cammino ottimo), sostituendo P1 con P2 otterrei cammino totale più corto. Contraddizione.
2. **Map Gossip**: Flood topology info via spanning tree. $O(mn)$ msg. Pro: mappa completa. Contro: memoria $O(m + n)$.
3. **Bellman-Ford**: $d_x(y) = \min_v\{w_{xv} + d_v(y)\}$. Converge in $n - 1$ iterazioni senza cicli negativi.
4. **Count-to-Infinity esempio**: A-B-C, link A-B rotto. B chiede a C distanza per A. C dice 2 (via B!). B aggiorna a 3. C aggiorna a 4. Loop infinito.
5. **DV vs LS**: DV (RIP): poca RAM, lento a convergere. LS (OSPF): tanta RAM, converge rapido. BGP usa Path Vector.

5 Fault Tolerance e Consenso Distribuito

Prerequisiti

- Capitolo 2: Modello distribuito, differenza sincrono/asincrono
- Concetto di connettività di un grafo
- (Opzionale) Probabilità di base per gli algoritmi randomizzati

In questo capitolo affrontiamo la dura realtà: le cose si rompono!



5.1 Il Modello dei Guasti: Definizioni e Classificazioni

In un sistema reale, l'affidabilità totale è praticamente inesistente. L'obiettivo è progettare protocolli capaci di resistere a un certo numero di guasti.

5.1.1 Classificazione dei Guasti

I guasti (*faults*) possono essere classificati secondo tre dimensioni principali (Slide 06):

- **Per Causa**:
  - **Execution faults**: Avvengono durante l'esecuzione di un'entità.
  - **Transmission faults**: Dovuti a malfunzionamenti nella trasmissione dei messaggi.
  - **Component failure**: Disattivazione o arresto di un'entità o di un link.
- **Per Durata**:
  - **Transient faults**: Appaiono e scompaiono da soli (es. interferenze temporanee).
  - **Permanent faults**: Persistono finché il componente non viene riparato. *Assumeremo d'ora in poi guasti permanenti.*
- **Per Estensione**:

- **Localized faults**: Solo un insieme fisso (ma ignoto a priori) di componenti è guasto.
- **Ubiquitous faults**: Tutti i componenti mostreranno prima o poi comportamenti anomali.

5.1.2 Tipi di Fallimenti delle Entità

I fallimenti delle entità si classificano per **pericolosità** (gravità):

1. **Crash (fail-stop)**: L'entità lavora correttamente e poi improvvisamente si ferma. È il caso più semplice da gestire.
2. **Send/Receive Omission**: L'entità occasionalmente perde messaggi in ricezione o non invia messaggi pronti.
3. **Byzantine**: L'entità non segue il protocollo e può compiere **qualsiasi** azione (comportamento arbitrario o malevolo).

5.1.3 Tipi di Fallimenti dei Link

- **Omission**: Il messaggio inviato non viene mai consegnato.
- **Addition**: Viene consegnato un messaggio mai spedito (messaggi fantasma).
- **Corruption**: Il messaggio consegnato è diverso da quello inviato.

Nota

Nota sulla rilevabilità: I guasti rilevabili possono essere superati più facilmente; quelli non rilevabili (come i bizantini) hanno conseguenze peggiori.

5.2 Fattori Topologici: Connettività e Robustezza

5.2.1 Gerarchia Visuale dei Fallimenti

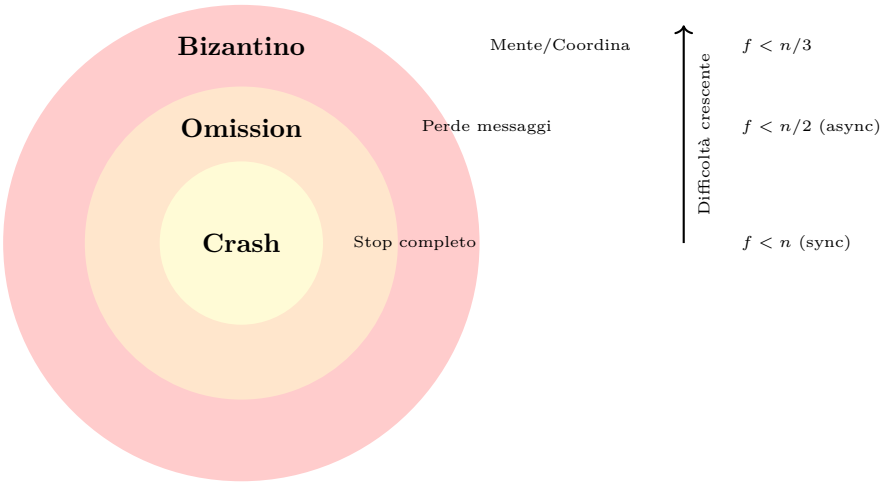


Figura 5: Gerarchia dei fallimenti: Crash $\subset$ Omission $\subset$ Byzantine. Ogni classe include la precedente.

5.2.2 Tipi di Fallimenti dei Link

Anche i link possono fallire in vari modi:

Tipo	Descrizione	Difficoltà
Omission	Il messaggio viene perso (mai consegnato)	$\star$
Addition	Viene consegnato un messaggio fantasma (non spedito)	$\star\star$
Corruption	Il messaggio arriva modificato	$\star\star$

Nota

Perché "Bizantino"?

Il nome viene dal "Problema dei Generali Bizantini": generali che devono coordinarsi per un attacco, ma alcuni potrebbero essere traditori che inviano informazioni false.

Analogia del Mondo Reale: Il Gruppo WhatsApp

Immagina di dover organizzare una cena con gli amici su WhatsApp.

- **Crash**: Mario non risponde più perché gli è caduto il telefono nel water.
- **Omission**: I messaggi di Luigi non arrivano perché il server è sovraccarico.
- **Bizantino**: Wario dice a Mario "Pizzata alle 8" e a Luigi "Sushi alle 9" solo per creare caos.

5.3 Fattori Topologici: Quanto Robusta Deve Essere la Rete?

La tolleranza ai guasti dipende criticamente dalla **connettività** del grafo.

Definizione: Connettività

- **Edge Connectivity (k-edge-connected)**: Numero minimo di archi da rimuovere per disconnettere il grafo.
- **Node Connectivity (k-node-connected)**: Numero minimo di nodi da rimuovere per disconnettere il grafo.

Teorema: Impatto della Connettività sul Broadcast

- Se possono fallire k link, è impossibile garantire il broadcast a meno che la rete sia $(k+1)$-edge-connected.
- Se possono fallire k nodi, è impossibile garantire il broadcast a meno che la rete sia $(k+1)$-node-connected.

5.3.1 Connettività delle Topologie Comuni

Topologia	Node Conn.	Edge Conn.	Max Degree
Albero	1	1	$\leq n-1$
Anello (bidirez.)	2	2	2
Grafo Completo	$n-1$	$n-1$	$n-1$
Ipercubo	$\log n$	$\log n$	$\log n$

Importante

Regola generale: Node Connectivity $\leq$ Edge Connectivity $\leq$ Min Degree.

Teorema:

Per tollerare F fallimenti di link/nodi, la rete deve avere connectivity $\geq F+1$.

5.4 Il Problema dell'Accordo (Agreement)

Il problema del consenso è in realtà un caso specifico di un problema più generale chiamato **p-Agreement**.

Definizione: p-Agreement Problem

Input: Ogni entità x ha un valore iniziale $v(x)$ preso da un insieme noto.

Output: Almeno p entità devono decidere (concordare) sullo stesso valore $d(x)$ entro un tempo finito.

Vincoli:

1. **Agreement**: Almeno p entità decidono lo stesso valore.
2. **Non-triviality**: Se tutti i valori iniziali sono uguali a V, la decisione deve essere V.
3. **Termination**: Tutti decidono in tempo finito.

Nota

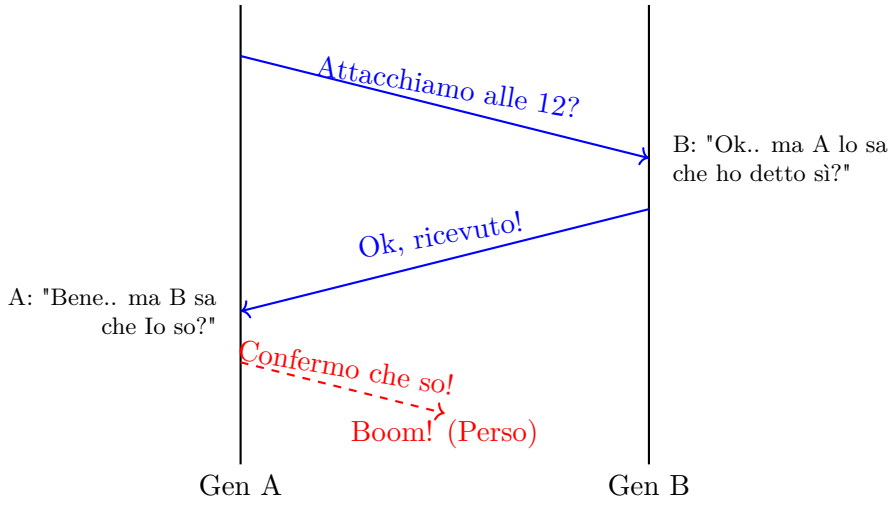
Quando $p = n$, il problema viene chiamato **Consenso**.

5.5 Il Problema dei Due Generali: Quando è Impossibile

5.5.1 Lo Scenario

Due generali devono coordinare un attacco **simultaneo**. L'unico modo per comunicare è tramite messaggeri che attraversano una valle nemica (potrebbero essere intercettati).

$$\text{Generale A} \longleftrightarrow \text{messaggeri} \longleftrightarrow \text{Generale B}$$



Problema: Come possono essere *sicuri* di attaccare insieme?

5.5.2 Perché è Impossibile

1. A manda: "Attacchiamo domani alle 8!"
2. B risponde: "OK, ricevuto"
3. Ma A non sa se la risposta arriverà...
   quindi manda: "OK, ho ricevuto il tuo OK"
4. Ma B non sa se il messaggio di A arriverà...
   quindi manda: "OK, ho ricevuto il tuo OK del mio OK"
5. ...e così all'infinito!

Teorema: Impossibilità dei Due Generali

È **impossibile** raggiungere il consenso con link inaffidabili, anche in un sistema completamente sincrono.

Domanda Killer: Two Generals

Attenzione: questa dimostrazione si basa sulla perdita dell'**ultimo messaggio**. Chiedono spesso di spiegarla!

Dimostrazione Formale. **Lemma**: In ogni esecuzione in cui i due generali decidono di attaccare, almeno un messaggio deve essere consegnato.

Prova per contraddizione: Supponiamo esista un protocollo P che risolve il problema con $k \geq 1$ messaggi.

Sia E un'esecuzione di P in cui:

- Entrambi i generali decidono di attaccare
- Vengono consegnati esattamente k messaggi (il minimo possibile)

Consideriamo l'esecuzione E' identica a E, eccetto che l'**ultimo messaggio viene perso**.

Osservazione chiave: Per il mittente dell'ultimo messaggio, le esecuzioni E ed E' sono **indistinguibili** (non può sapere se il messaggio è arrivato o meno).

Quindi:

- Il mittente prende la stessa decisione in E e in E': **attacca**
- Poiché il protocollo è corretto, anche il destinatario deve attaccare in E'
- Ma in E' sono stati consegnati solo $k-1$ messaggi!

Contraddizione: E era l'esecuzione con il minimo numero di messaggi per attaccare.
Ripetendo l'argomento: $k-1$, $k-2$, ..., fino a $k=0$.
Ma con $k=0$ messaggi, B non può distinguere tra "A decide di non attaccare" e "A attacca ma nessun messaggio arriva". **Impossibile**. □

Nota

Analogia moderna: Le "spunte blu" di WhatsApp!

- 1 spunta: inviato (ma non sai se arrivato)
- 2 spunte: consegnato
- 2 spunte blu: letto

WhatsApp "risolve" il problema usando un server centrale come "terza parte fidata".

5.6 Il Problema del Consenso

5.6.1 Definizione Formale

Definizione: Consenso

Input: Ogni entità x ha un valore iniziale $v(x) \in \{0, 1\}$
Output: Ogni entità deve decidere un valore $d(x)$
Proprietà richieste:

1. **Agreement**: Tutti decidono lo stesso valore
2. **Validity**: Se tutti partono con lo stesso valore V, tutti decidono V
3. **Termination**: Tutti decidono in tempo finito

5.6.2 Perché è Fondamentale?

Il consenso è alla base di moltissimi problemi reali:

- **Distributed Database**: Accordarsi se eseguire il *commit* o l'*abort* di una transazione.
- **Diagnostic**: Accordarsi se classificare un componente del sistema come guasto o meno.
- **Voting**: Accordarsi sulla stima di una misura letta da più sensori (es. altimetri in un aereo).
- **Blockchain**: I nodi devono accordarsi su quali blocchi accettare.
- **Leader Election**: Tutti devono concordare su chi è il leader.

5.7 Consenso con Fallimenti di Link

Se possono fallire fino a F link:

Teorema:

Il consenso è raggiungibile solo se la rete è $(F+1)$-connessa.

5.7.1 Protocollo TwoSteps (per grafi completi)

Se abbiamo un grafo completo e $F < n - 1$:

```
1. La sorgente invia il valore a F+1 vicini
2. Chi riceve, inoltra a tutti gli altri
```

Perché funziona? Sappiamo che $F + 1$ vicini ricevono il messaggio. Anche se F link falliscono, almeno uno dei vicini riceverà il valore e lo inoltrerà a tutti gli altri. Poiché ogni nodo ha al massimo F link guasti e la rete è un grafo completo con $n - 1 > F$ link per nodo, ogni nodo ha almeno un link funzionante verso i nodi che hanno ricevuto il valore al passo 1.

Complessità: $O(Fn)$ messaggi.

5.7.2 Leader Election: Protocollo FT-BroadElection

In un grafo completo con al più F fallimenti di link, è possibile eleggere un leader combinando il broadcast tollerante ai guasti:

1. Ogni entità esegue un broadcast del proprio ID usando **TwoSteps**.
2. Ogni entità, dopo aver ricevuto tutti gli ID (o aver atteso il tempo necessario in un sistema sincrono), calcola il minimo.
3. L'entità con l'ID minimo diventa il LEADER.

Complessità: $O(Fn^2)$ messaggi.

5.7.3 Leader Election: Protocollo FT-BroadElection

In un grafo completo con al più F fallimenti di link, è possibile eleggere un leader combinando il broadcast tollerante ai guasti:

1. Ogni entità esegue un broadcast del proprio ID usando **TwoSteps**.
2. Ogni entità, dopo aver ricevuto tutti gli ID (o aver atteso il tempo necessario in un sistema sincrono), calcola il minimo.
3. L'entità con l'ID minimo diventa il LEADER.

Complessità: $O(Fn^2)$ messaggi.

5.8 Consenso con Crash: Il Teorema FLP

5.8.1 L'Impossibilità nel Caso Asincrono (FLP)

Teorema: Fischer-Lynch-Paterson, 1985

È **impossibile** raggiungere il consenso con un protocollo **deterministico** in un sistema **asincrono**, anche con **un solo** crash.

Analogia del Mondo Reale: Il Dilemma delle Spunte Grigie

Immagina di mandare un messaggio WhatsApp urgente al tuo partner: "Pizza o Sushi? Devo ordinare ora!". Vedi una sola spunta grigia (inviato, ma non consegnato/letto). E aspetti. Passa 1 minuto. Passano 10 minuti.

Hai due teorie:

1. **Teoria Crash**: Al partner è caduto il telefono nel water. È morto. Non risponderà mai. Dovresti decidere tu (es. Pizza) per non morire di fame.
2. **Teoria Lentezza**: Il partner è in ascensore o in galleria. Il telefono funziona, ma il messaggio è lento. Tra 1 minuto risponderà "Sushi!".

Il problema: Dal tuo punto di vista (spunta grigia), **questi due scenari sono identici**. Non c'è modo fisico di distinguerli.

- Se decidi "Pizza" (assumendo Crash), potresti sbagliare se era solo Lentezza (e lui voleva Sushi).
- Se aspetti (assumendo Lentezza), potresti aspettare in eterno se era Crash.

L'algoritmo si blocca in questo limbo di indecisione. Non può garantire contemporaneamente di *terminare* (non aspettare per sempre) e di essere *corretto* (mettersi d'accordo).

5.8.2 Dimostrazione Formale di FLP

La dimostrazione originale di Fischer, Lynch e Paterson (1985) è tecnica, ma l'idea centrale è elegante.

Definizione: Configurazione e Valenza

- Una **configurazione** C descrive lo stato di tutti i processi e tutti i messaggi in transito.
- Una configurazione è **0-valente** se da C tutte le esecuzioni possibili portano alla decisione 0.
- Una configurazione è **1-valente** se da C tutte le esecuzioni possibili portano alla decisione 1.
- Una configurazione è **bivalente** se da C esistono esecuzioni che portano a 0 e altre che portano a 1.

La dimostrazione procede in due passi:
Passo 1: Esiste una configurazione iniziale bivalente.

Dimostrazione. Per assurdo. Supponiamo che tutte le configurazioni iniziali siano univalenti (0-valenti o 1-valenti).

Consideriamo due configurazioni iniziali C_0 e C_1 che differiscono solo per il valore iniziale di un processo p:

- In C_0, tutti i processi hanno valore iniziale 0 $\Rightarrow$ per **validity**, la decisione deve essere 0, quindi C_0 è 0-valente.
- In C_1, tutti i processi hanno valore iniziale 1 $\Rightarrow$ per **validity**, la decisione deve essere 1, quindi C_1 è 1-valente.

Esiste una catena di configurazioni $C_0 = D_0, D_1, \ldots, D_n = C_1$ dove configurazioni adiacenti differiscono solo per il valore di un processo.

Poiché D_0 è 0-valente e D_n è 1-valente, esiste un i tale che D_i è 0-valente e D_{i+1} è 1-valente.

Ma D_i e D_{i+1} differiscono solo per il valore del processo p. Se p crasha immediatamente (prima di fare qualsiasi cosa), le esecuzioni da D_i e D_{i+1} sono **indistinguibili** per tutti gli altri processi.

Quindi devono decidere lo stesso valore — **contraddizione**! □

Passo 2: Da una configurazione bivalente, l'avversario può sempre mantenere la bivalenza.

Idea. L'avversario controlla lo **scheduling** dei messaggi (chi riceve cosa e quando).

Sia C una configurazione bivalente e sia e un evento (ricezione di un messaggio m da parte del processo p).

L'avversario mostra che:

- Se e è applicabile a C, esiste sempre un modo per "ritardare" e o altri eventi in modo che la configurazione risultante sia ancora bivalente.
- In particolare, se applicare e porta a una configurazione 0-valente, l'avversario può prima applicare altri eventi che portano a una configurazione 1-valente, e viceversa.

L'argomento chiave usa il fatto che un processo può essere "sospettato" di essere crashato quando è solo lento. L'avversario sfrutta questa ambiguità per mantenere la bivalenza indefinitamente. □

Importante

Significato di FLP:

- FLP **non** dice che il consenso è impossibile in pratica.
- Dice che non esiste un algoritmo **deterministico** che **garantisce** terminazione.
- L'avversario deve essere molto potente (controllo totale dello scheduling) per bloccare l'algoritmo.
- In pratica, algoritmi come Paxos/Raft funzionano benissimo perché l'avversario reale non è così malevolo.

5.8.3 Come Aggirare FLP: Sfruttare il Tempo

Nei sistemi **sincroni**, il silenzio diventa un'informazione preziosa (*Silence is Expressive*). Se B non risponde entro il tempo massimo Δ, so per certo che è morto.

Esempio: Il silenzio è espressivo

In un sistema sincrono, è possibile trasmettere un valore X usando solo 2 bit:

1. Il mittente invia un bit al tempo t.
2. Il mittente attende X tick di clock.
3. Il mittente invia un secondo bit al tempo $t + X$.
4. Il destinatario riceve il primo bit a $t' = t + 1$, il secondo a $t'' = t + X + 1$, e calcola $X = t'' - t'$.

5.9 TellAll_Crash: Consenso Sincrono con Crash

In un sistema **sincrono** con al più $f < n$ crash, possiamo raggiungere il consenso in $f+1$ round. L'idea è che ogni entità propaghi "tutto ciò che sa".

Definizione: Ogni entità x mantiene un report $r(x,t)$ al tempo t:

- $r(x,0) = v(x)$ (valore iniziale $0,1$).
- $r(x,t) = r(x,t-1)$ AND $m(y_1,t)$ AND $\dots$ AND $m(y_k,t)$.
- Dove $m(y_i,t)$ è il messaggio del vicino y_i, oppure 1 se y_i è crashato (il silenzio indica crash).

Algoritmo:

```
for t = 1 to f + 1:
    send r(x, t-1) to all neighbors
    wait for messages
    update r(x, t) using AND of all received values
decide r(x, f+1)
```

5.9.1 Perché servono $f+1$ round? (Spiegazione Narrativa)

La dimostrazione formale per assurdo è corretta, ma ecco l'intuizione ("La Staffetta della Menzogna").

Immagina che ci siano $f = 2$ traditori (nodi che crasheranno) in una rete. Vogliamo che tutti sappiano se esiste uno "0" nella rete.

1. **Round 1**: Il nodo A (che ha lo "0") lo dice a B, ma crasha subito dopo. Solo B sa dello "0". Gli altri pensano "1".
2. **Round 2**: Ora B è l'unico custode della verità. B dice lo "0" a C, ma crasha subito dopo. Ora solo C sa dello "0".
3. **Round 3**: Se ci fermassimo qui (f round), C saprebbe "0" e gli altri "1". Disastro (no agreement)! Ma al round 3 ($f+1$), C (che è onesto, perché i 2 crash sono finiti) dirà lo "0" a tutti.

Importante

Il Principio: Un avversario può "nascondere" un'informazione facendola saltare di nodo in nodo tramite crash a catena. Ogni crash "compra" un round di invisibilità. Se ci sono f crash disponibili, l'avversario può tenere l'informazione nascosta alla maggioranza per f round. Al round $f+1$, ha finito i crash: l'ultimo custode rimarrà vivo e diffonderà la verità.

5.9.2 Ottimizzazione dei Messaggi

Poiché il valore 1 è il default, un'entità può inviare lo 0 **solo la prima volta** che lo vede. Questo riduce la complessità a $O(n^2)$ messaggi totali invece di $O(n^2 f)$.

Proprietà finale:

- Se almeno un'entità corretta parte con 0, tutti decidono 0.
- Se tutte le entità corrette partono con 1, e anche quelle guaste partono con 1, tutti decidono 1.
- Se tutte le corrette hanno 1 ma una guasta ha 0, tutti decidono 0 (se la guasta riesce a mandare lo 0 prima di morire) oppure tutti decidono 1. L'importante è che **tutti decidano lo stesso**.

5.10 Ben-Or: Consenso Randomizzato Asincrono

5.10.1 L'Idea Chiave

Il teorema FLP dice che non possiamo avere consenso *deterministico*. Ben-Or (1983) aggira il limite usando la casualità (**Las Vegas**): il protocollo è sempre corretto e termina con probabilità 1.

5.10.2 Gestione dei Messaggi e Round

Poiché il sistema è asincrono, i nodi possono trovarsi in round diversi. Ogni messaggio è etichettato con il numero di round r:

- **USE**: Se il messaggio è del round corrente r, processalo.
- **IGNORE**: Se il messaggio è di un round passato $r' < r$, scartalo.
- **SAVE**: Se il messaggio è di un round futuro $r' > r$, salvalo in coda e processalo quando raggiungerai quel round.

5.10.3 Il Protocollo

Ogni round r si divide in due fasi (**Propose** e **Adapt**):

```
while non_deciso:
    # FASE 1: PROPOSE
    broadcast MyValue(r, mio_valore)
    wait for n-f MyValue messages of round r
    if almeno n/2+1 messaggi hanno lo stesso valore v:
        broadcast Propose(r, v)
    else:
        broadcast Propose(r, "?")

    # FASE 2: ADAPT
    wait for n-f Propose messages of round r
    if almeno un Propose(r, v) con v != "?":
        mio_valore = v
        if almeno f+1 messaggi sono Propose(r, v):
            DECIDI v (e continua per aiutare gli altri)
    else:
        mio_valore = random(0, 1) # Caso sfortunato: scelta casuale
    r = r + 1
```

5.10.4 Perché Funziona?

- **Tolleranza**: $f < n/2$ crash (garantisce che $n - f > n/2$, quindi le maggioranze si intersecano).
- **Accordo**: Se un nodo decide v, tutti gli altri riceveranno almeno un *Propose(v)* e adotteranno v per il round successivo, decidendo a loro volta.
- **Terminazione**: Anche se i nodi continuano a scegliere a caso, prima o poi (per pura probabilità) sceglieranno tutti lo stesso valore nello stesso round, raggiungendo il consenso. Il numero atteso di round è $O(2^n)$.

5.11 Fallimenti Bizantini: Il Caso Peggiore

5.11.1 Cosa Può Fare un Nodo Bizantino

Un nodo bizantino è il "cattivo" perfetto. Non solo si rompe, ma cerca attivamente di sabotare il sistema.

- Può seguire il protocollo quando vuole o deviare arbitrariamente.
- Può **mentire** sui propri dati iniziali o sullo stato interno.
- Può mandare **messaggi diversi** a nodi diversi (attacco di sdoppiamento).

5.11.2 L'Intuizione: Perché 3 non bastano per 1 traditore? (Il Triangolo del Sospetto)

La condizione $n > 3f$ è la colonna portante della tolleranza ai guasti bizantini. Vediamo perché con $n = 3$ e $f = 1$ il sistema crolla inevitabilmente.

Immaginiamo tre generali: **A** (il Comandante), **B** e **C**. Uno di loro è un traditore (bizantino).

- **Scenario 1: Il Comandante è Onesto**.
  - A ordina "ATTACCO" a tutti.
  - B (onesto) riceve "ATTACCO" da A.
  - C (traditore) mente e dice a B: "Ehi, A mi ha ordinato RITIRATA!".
  - **Cosa vede B?**: Vede un ordine di Attacco e un rapporto di Ritirata. È un pareggio 1 contro 1. Chi mente? A o C?
- **Scenario 2: Il Comandante è il Traditore**.
  - A manda "ATTACCO" a B e "RITIRATA" a C.
  - B riceve "ATTACCO".
  - C (onesto) riceve "RITIRATA" e lo riferisce a B: "A mi ha detto RITIRATA".
  - **Cosa vede B?**: Vede esattamente la stessa cosa dello Scenario 1! Un ordine di Attacco e un rapporto di Ritirata.

Attenzione

Il vicolo cieco logico: Poiché B non può distinguere se il bugiardo è il Comandante (A) o il suo collega (C), non può prendere nessuna decisione sicura. In un sistema a 3, un bugiardo può sempre creare un "pareggio" informativo.

Per rompere questo pareggio, serve un quarto nodo. Con $n = 4$ e $f = 1$, i 3 nodi onesti formano una maggioranza che il bugiardo non può manipolare.

5.11.3 Limite Teorico

Teorema: Limite Byzantine (Lamport, Shostak, Pease, 1982)

In un sistema (anche sincrono) con n entità, il consenso è **impossibile** se il numero di bizantini $f \geq n/3$.

Dimostrazione Formale per Riduzione. Dimostriamo che se esistesse un protocollo per $n = 3f$, potremmo risolvere il caso $n = 3, f = 1$ (che sappiamo essere impossibile).

Costruzione: Supponiamo di avere un protocollo P che funziona con $n = 3f$ nodi e f bizantini.

Dividiamo i $3f$ nodi in tre gruppi di f nodi ciascuno: A, B, C.

Simuliamo P con soli 3 "super-nodi" A', B', C', dove ogni super-nodo simula f nodi del gruppo corrispondente.

Scenario 1: A' è onesto, B' è onesto, C' è bizantino.

- A' simula f nodi con valore 0, B' simula f nodi con valore 1.
- C' può comportarsi arbitrariamente (simula f bizantini).
- Per correttezza di P, A' e B' devono decidere lo stesso valore.

Scenario 2: Ora consideriamo un sistema con solo 3 nodi A', B', C' (problema originale dei 3 generali).

- A' ha valore 0, B' ha valore 1.
- Il bizantino C' simula esattamente il comportamento di f bizantini verso A' e f bizantini verso B'.

Indistinguibilità: Dal punto di vista di A':

- Nello Scenario 1, riceve messaggi da B' (che simula f nodi con valore 1) e da C' (bizantino).
- Nello Scenario 2, riceve gli stessi messaggi (indistinguibili).

Quindi A' deve decidere lo stesso valore in entrambi gli scenari.

Analogamente per B'.

Ma nello Scenario 2, se C' è il bizantino, A' e B' devono accordarsi nonostante partano con valori diversi — **impossibile** per il caso $n = 3, f = 1$.

Conclusione: Non può esistere un protocollo per $n = 3f$. Serve $n > 3f$, cioè $f < n/3$. □

Nota

Conseguenze pratiche:

- Per tollerare 1 bizantino: servono almeno 4 nodi.
- Per tollerare 2 bizantini: servono almeno 7 nodi.
- Per tollerare f bizantini: servono almeno $3f + 1$ nodi.

Questo spiega perché i sistemi Byzantine Fault Tolerant (BFT) come PBFT hanno overhead significativo.

5.11.4 Meccanismo RegisteredMail: La "Raccomandata con Ricevuta di Ritorno"

Per contrastare le bugie bizantine (come inviare 0 ad alcuni e 1 ad altri), si usa un meccanismo chiamato *RegisteredMail*. L'idea è che un messaggio viene accettato solo se si ha la "prova" che il mittente non sta sdoppiando l'informazione.

Analogia del Mondo Reale: L'Invito al Matrimonio con RSVP Pubblico

Immagina che Wario (bizantino) voglia invitare gli amici a cena, ma ad alcuni voglia dire "Pizza" e ad altri "Sushi" per farli litigare.

- Con la posta normale, può farlo facilmente.
- Con **RegisteredMail**, ogni volta che qualcuno riceve l'invito, deve urlare a tutti gli altri: "Ehi, ho ricevuto un invito per PIZZA da Wario!".
- Se Mario riceve l'invito "Pizza" e sente che altri $f+1$ amici hanno ricevuto "Pizza", allora può fidarsi.
- Se sente che qualcuno ha ricevuto "Sushi", smaschera Wario immediatamente.

Il sistema "echeggia" (ECHO) i messaggi finché non diventano una verità pubblica inattaccabile.

Regole per un'entità corretta x:

1. **INIT**: Per iniziare un *wake-up* al tempo t, x invia broadcast $(init, 0, id(x), t)$.
2. **ECHO (diretto)**: Se riceve $(init, 0, id(y), t)$ da y al tempo $t+1$, invia broadcast $(echo, 0, id(y), t)$.
3. **ECHO (indiretto)**: Se riceve $(echo, 0, id(y), t)$ da almeno $f+1$ entità diverse al tempo $t' \geq t+2$, invia broadcast $(echo, 0, id(y), t)$ (se non l'aveva già fatto). *Questo garantisce che almeno una entità corretta ha fatto echo.*
4. **ACCEPT**: x accetta il *wake-up* da y solo se riceve $(echo, 0, id(y), t)$ da almeno $n-f$ entità.

5.11.5 Protocollo TellZero_Byz (Sistema Sincrono)

Attenzione

Nota Esame: Secondo gli appunti del corso, questo specifico protocollo (Byzantino Sincrono) è spesso considerato argomento avanzato o facoltativo. Concentrarsi maggiormente sul caso Asincrono (impossibilità) e sulle definizioni.

Questo protocollo usa *RegisteredMail* per forzare il consenso sul valore 0. Funziona in $f+2$ stadi, ognuno di 2 unità di tempo.

1. **Tempo 0**: Ogni entità corretta con valore iniziale 0 avvia *RegisteredMail* per il proprio 0.
2. **Tempo** $2i$ **(**$1 \leq i \leq f+1$**)**: Un'entità corretta x avvia *RegisteredMail* per il proprio 0 se e solo se:
   - Ha accettato *wake-up* da **più di** $f+i-1$ entità distinte.
   - Non ha già avviato un *wake-up* in precedenza.
3. **Tempo** $2(f+2)$: Un'entità corretta decide 0 se ha accettato **almeno** $2f+1$ *wake-up*. Altrimenti decide 1.

Complessità: $O(n^3)$ messaggi. **Tolleranza**: $f < n/3$.

5.12 Riepilogo: Limiti di Tolleranza

Tipo di Fault	Condizione	Tolleranza Max
Crash (sincrono)	Grafo completo	$f < n$
Crash (asincrono, randomizzato)	Grafo completo	$f < n/2$
Bizantino (sincrono)	Grafo completo	$f < n/3$
Bizantino (randomizzato)	Grafo completo	$f < n/3$
Link failures	Qualsiasi	$F <$ connectivity

Key Takeaways

1. **Gerarchia guasti**: Crash $\subset$ Omission $\subset$ Bizantino (crescente difficoltà)
2. **Two Generals**: Consenso impossibile con link inaffidabili
3. **FLP (1985)**: Consenso deterministico impossibile in sistema asincrono con 1 crash
4. **Limiti tolleranza**: Crash sync $f < n$, Crash async $f < n/2$, Byzantine $f < n/3$
5. **Ben-Or**: Algoritmo randomizzato Las Vegas che aggira FLP

Errori Comuni da Evitare

- **FLP non dice "impossibile"**: Dice impossibile *deterministico* — randomizzazione funziona!
- **Byzantine $\neq$ Crash**: Con $f \geq n/3$ bizantini, impossibile anche con sincronismo
- **Two Generals**: Il problema è l'ultimo messaggio — quanti ACK servono?
- **Connectivity**: Per tollerare k guasti serve $(k+1)$-connectivity

5.13 Esercizi e Domande

5.13.1 Domande Concettuali

1. Spiega la differenza tra crash, omission e bizantino.
2. Perché il problema dei due generali è impossibile?
3. Cosa dice il teorema FLP? Come si aggira?
4. Perché Ben-Or può funzionare mentre algoritmi deterministici no?
5. Perché i bizantini sono più difficili da gestire dei crash?
6. Cos'è l'edge/node connectivity e perché è importante per la fault tolerance?

5.13.2 Esercizi di Dimostrazione

1. **Enuncia e dimostra** il teorema dei Two Generals (usa l'argomento dell'ultimo messaggio).
2. **Perché il consenso è impossibile** in sistemi asincroni con crash? (FLP)
3. **Descrivi TellAll_Crash**: idea, perché servono $f+1$ round, complessità.
4. **Come funziona Ben-Or**? Perché tollera $f < n/2$?
5. **Cos'è il Registered Mail Protocol**? A cosa serve contro i bizantini?
6. **Dimostra che** per tollerare f bizantini servono $n > 3f$ nodi.

Tracce di Soluzione

Soluzioni Domande Concettuali:

1. **Gerarchia**: Crash (stop). Omission (perde msg). Bizantino (arbitrario/malevolo). Crash $\subset$ Omission $\subset$ Byzantine.
2. **Two Generals**: Ultimo messaggio può perdersi. Mittente non sa se arrivato $\Rightarrow$ non può decidere con certezza.
3. **FLP**: In asincrono non distinguo crash da lentezza. Se aspetto, potrei aspettare per sempre. Se decido, potrei sbagliare.
4. **Ben-Or**: Randomizzato Las Vegas. Rompe simmetria FLP con casualità. Corretto sempre, termina con prob. 1.
5. **Bizantino difficile**: Non solo si ferma, ma mente e coordina attacchi. Può mandare msg diversi a nodi diversi.
6. **Connectivity**: Min nodi/archi da rimuovere per disconnettere. Per tollerare k guasti serve connectivity $k + 1$.

Soluzioni Esercizi di Dimostrazione:

1. **Two Generals dimostrazione**: Argomento dell'ultimo messaggio. Se esiste protocollo con k msg, considera esecuzione dove ultimo viene perso. Mittente indistinguibile $\Rightarrow$ stessa decisione. Ma solo $k - 1$ msg consegnati $\Rightarrow$ contraddizione per induzione fino a $k = 0$.
2. **FLP dimostrazione**: Esiste configurazione bivalente iniziale. Da questa, l'avversario può sempre scegliere scheduling che mantiene bivalenza, impedendo decisione.
3. **TellAll_Crash**: $f + 1$ round. Avversario può "nascondere" info con crash a catena. A round $f + 1$, finiti i crash, verità emerge. $O(n^2)$ msg.
4. **Ben-Or tolleranza**: $f < n/2$ perché serve maggioranza che si interseca. Due fasi: Propose e Adapt.
5. **Registered Mail**: Contro sdoppiamento bizantino. Messaggio accettato solo se $\geq f + 1$ echo confermano. Almeno 1 onesto tra $f + 1$.
6. **n > 3f dimostrazione**: Con $n = 3, f = 1$, il traditore crea pareggio 1 vs 1. Nessuna maggioranza distingue verità da menzogna. Serve quarto nodo per maggioranza onesta.

6 Distributed Hash Tables e Chord

Prerequisiti

- Capitolo 6: Concetto di sistema P2P e tolleranza ai guasti
- Concetto di hash table (chiave → valore)
- Aritmetica modulare (per lo spazio circolare)

In questo capitolo esploriamo come organizzare dati su milioni di computer in modo che qualsiasi dato sia trovabile in pochi passi. È la magia dietro BitTorrent, Ethereum e molti altri sistemi P2P.

6.1 Contesto: Le Reti Peer-to-Peer (P2P)

6.1.1 Definizione e Filosofia

Definizione: Peer-to-Peer

Un sistema in cui tutte le entità partecipanti (**peer**) hanno lo stesso ruolo e responsabilità ("equal standing"). È un principio organizzativo dove ognuno agisce sia come client che come server.

Proprietà tipiche dei sistemi P2P:

- **Self-organizing**: Nessuna gestione centrale.
- **Collaborazione volontaria**: Basata sulle risorse messe a disposizione dai peer.
- **Unreliable/Unmanaged/Uncoordinated**: I nodi vanno e vengono senza preavviso (*churn*).
- **Resiliente agli attacchi**: Data la natura decentralizzata.
- **Grande collezione di risorse**: Sfrutta storage e banda di milioni di utenti.

6.1.2 Unstructured vs Structured Networks

Caratteristica	Unstructured (A-strutturata)	Structured (Strutturata)
Topologia	Grafo generico (casuale)	Struttura rigida (es. anello, ipercubo)
Posizionamento	Nodi e oggetti liberi di stare ovunque	La struttura determina dove stanno i dati
Ricerca	Flooding / Random Walk	Routing deterministico (es. DHT)
Efficienza	Bassa per file rari	Alta ($O(\log N)$)

6.1.3 Evoluzione delle Architetture P2P

Importante

Cronologia e Lezioni Apprese

- **Napster (1999)**: Indice centralizzato. Peak 13.6M utenti (Feb 2001). Chiuso per ordine giudiziario a Luglio 2001.
- **Gnutella (2000)**: Flooding puro. Decentralizzato. Nel 2010 il client LimeWire è stato chiuso legalmente.
- **KaZaA (2001)**: Super-nodi ibridi. Nel 2006 ha pagato 100M$ di danni e si è convertito a servizio legale.
- **DHT (2001+)**: Chord, CAN, Kademlia. Soluzione strutturata.

Il Concetto di Overlay Network: Una rete virtuale costruita sopra la rete IP fisica. I vicini nell'overlay hanno un cammino IP che li connette, ma non sono necessariamente vicini fisici.

Attenzione

Attenzione alla Distanza: In Chord, il nodo 10 e il nodo 11 sono "vicini di casa" (successore e predecessore). Tuttavia:

- Il nodo 10 potrebbe essere un computer a Roma.
- Il nodo 11 potrebbe essere un server a Tokyo.

Il "passo" logico tra 10 e 11 sulla DHT richiede in realtà che un pacchetto attraversi decine di router fisici in giro per il mondo. Chord ottimizza il numero di **salti logici**, non la latenza fisica dei cavi.

6.2 Il Problema: Trovare un Ago in un Pagliaio

6.2.1 Lo Scenario

Immagina milioni di computer collegati in rete. Ogni computer possiede alcuni file. Tu vuoi un file specifico.

Domanda: Come lo trovi?

6.2.2 Approcci e Problemi

Approccio	Come Funziona	Problema
Server centrale	Un server conosce tutto	Single Point of Failure
Flooding	Chiedi a tutti	$O(N)$ messaggi, non scala
DHT	Organizzazione intelligente	$O(\log N)$ **messaggi!**

Analogia del Mondo Reale: La Rubrica Telefonica

Immagina di dover trovare il numero di "Mario Rossi".

- **Server (Napster)**: C'è un centralinista. Chiedi a lui, ti risponde subito. Se si ammala, nessuno trova nulla.
- **Flooding (Gnutella)**: Urli in piazza "Chi conosce Mario?". 100 persone urlano ad altre 100. Caos.
- **DHT (Chord)**: Ti metti d'accordo: A tiene i numeri che iniziano per 'A', B per 'B', ecc. Se cerchi Mario, sai che devi chiedere a M (o a qualcuno che sa dov'è M). Ordine e velocità.

6.3 L'Idea Geniale: Distributed Hash Table

6.3.1 Scalable Distributed Data Structures

Una struttura dati distribuita è gestita dalle entità di un sistema. È **scalabile** se il numero di entità può aumentare per adattarsi alla crescita della dimensione della struttura.

Le sfide principali (**Challenges**) sono:

- Mantenere le primitive della struttura dati (`put`, `get`).
- Implementare le primitive in modo efficiente (basso numero di messaggi).
- Bilanciare il carico di memorizzazione (*load balancing*).
- Gestire nodi che entrano ed escono (*join/leave*).

6.3.2 Cos'è una DHT?

Definizione: DHT

Una **DHT** implementa le funzionalità di una hash table in un sistema distribuito dove:

- I dati sono **distribuiti** su molti nodi
- Nessun nodo conosce tutto
- Possiamo trovare qualsiasi dato in $O(\log N)$ passi

6.4 Consistent Hashing: La Base di Tutto

Per capire il Consistent Hashing, dobbiamo prima capire perché l'approccio classico fallisce disastrosamente.

6.4.1 L'Approccio Ingenuo: Hashing Modulare

Immagina di avere $N = 3$ server (Server 0, 1, 2). La regola classica per assegnare una chiave K a un server è:

$$\text{Server} = hash(K) \pmod N$$

Esempio di Disastro: Hai 100 chiavi.

- Chiave K_{10} ha hash 10. $10 \pmod 3 = 1$. Va al Server 1.
- Chiave K_{11} ha hash 11. $11 \pmod 3 = 2$. Va al Server 2.

Ora aggiungi un server (N diventa 4). La formula cambia per **tutte** le chiavi!

- Chiave K_{10}: $10 \pmod 4 = 2$. Deve spostarsi dal Server 1 al Server 2.
- Chiave K_{11}: $11 \pmod 4 = 3$. Deve spostarsi dal Server 2 al Server 3.

Il problema logico: Aggiungere un nodo cambia il **denominatore** della formula per tutti. Risultato: ogni volta che un nodo entra o esce, quasi il 100% dei dati deve essere spostato. In un sistema P2P dove i nodi vanno e vengono, la rete sarebbe paralizzata dal continuo trasferimento di file.

6.4.2 La Soluzione: Consistent Hashing (L'Anello)

Il Consistent Hashing cambia la logica: **disaccoppia** le chiavi dal numero di server.

Concetto Passo-Passo:

1. **Lo Spazio Fisso**: Immagina un cerchio (un orologio) con 2^{160} tacche. Questo spazio non cambia mai, indipendentemente da quanti server hai.
2. **Posizionare i Server**: I server si posizionano sull'anello in punti casuali (usando l'hash del loro IP).
   - Es. Server A a ore 3, Server B a ore 6, Server C a ore 9.
3. **Posizionare i Dati**: Anche le chiavi (file) vengono mappate sull'anello.
   - Es. File X finisce a ore 4.
4. **La Regola del Successore**: Chi possiede il File X? *"Il primo server che incontri muovendoti in senso orario dalla posizione del file."*
   - Da ore 4 (File X) cammino in senso orario $\rightarrow$ incontro Server B a ore 6.
   - B possiede X.

6.4.3 Perché è Geniale (La Magia del Join)

Cosa succede se aggiungiamo un nuovo Server D a ore 5?

- **Hashing Modulare**: Avremmo ricalcolato tutto.
- **Consistent Hashing**:
  1. Il File X è a ore 4.
  2. Camminiamo in senso orario. Prima incontravamo B (ore 6).
  3. Ora incontriamo D (ore 5)!
  4. Quindi X passa da B a D.

Chi altro viene influenzato? Un file a ore 8 (gestito da C a ore 9) non cambia proprietario. Un file a ore 2 (gestito da A a ore 3) non cambia.

Nota

Risultato Logico: Quando un nodo entra, "ruba" le chiavi solo al suo successore (e solo quelle che si trovano tra lui e il predecessore). Il resto dell'anello non viene toccato. Lo spostamento dati è **minimale** (K/N in media).

6.5 Chord: L'Implementazione

Chord (2001) è un protocollo specifico che implementa una DHT usando il consistent hashing.

6.5.1 Identificatori e Spazio delle Chiavi

- **Funzione di Hash**: Usa **SHA-1** per mappare sia i nomi dei nodi (IP:porta) che i nomi degli oggetti in uno spazio a **160 bit** ($m = 160$).
- **Assegnamento**: Una chiave k è memorizzata sul suo **successore**: il primo nodo n tale che $ID(n) \geq ID(k)$.

6.5.2 Cosa Mantiene Ogni Nodo

Informazione	Descrizione
ID	$hash(IP : porta)$
Successor	Il primo nodo con ID maggiore (in senso orario)
Predecessor	Necessario per semplificare Join e Stabilization
Finger Table	Tabella di routing per salti esponenziali

6.5.3 Il Lookup Semplice (e Lento)

Idea base: Segui i successori finché non trovi il responsabile. Questo richiede $O(N)$ messaggi nel caso peggiore.

6.6 La Finger Table: Il Trucco per $O(\log N)$

6.6.1 L'Idea: Salti e Balzi verso l'Obiettivo

Immagina di vivere su un anello di 2^{160} posizioni. Senza scorciatoie, per trovare un file dovresti chiedere a ogni singolo vicino, facendo il giro dell'anello un passo alla volta. Sarebbe lentissimo.

La Finger Table risolve questo problema dando a ogni nodo dei "salti" a distanze raddoppiate.

Analogia del Mondo Reale: La Rubrica delle Distanze

Immagina di essere il nodo 0 in un anello di 1000 nodi. La tua Finger Table contiene i contatti di chi sta a:

- +1 casa di distanza
- +2 case
- +4 case
- +8, +16, +32, +64, +128, +256, +512...

Se devi cercare un file alla casa **600**:

1. Non puoi saltare a 1024 (troppo lontano).
2. Il salto più lungo che non supera 600 è quello a **512**.
3. Ti ritrovi alla casa 512. Ora l'obiettivo (600) dista solo **88** passi da te.
4. Da 512, cerchi nella sua tabella: il salto più lungo verso 600 è **64**.
5. Ti ritrovi a $512 + 64 = 576$. Ora mancano solo **24** passi.

A ogni salto, **dimezzi la distanza rimanente**. Come piegare un foglio di carta a metà ripetutamente: bastano pochissimi pieghi (logaritmici) per arrivare a una dimensione infinitesima (il nodo cercato).

6.6.2 Analisi Approfondita: Perché il Lookup è Veloce?

Perché la complessità è $O(\log N)$? Non è magia, è matematica.

Immagina di essere al nodo 0 e di cercare la chiave K che si trova esattamente a metà dell'anello (distanza D).

1. La mia Finger Table ha entrate a distanze $1, 2, 4, 8, \ldots$.
2. L'entrata più lontana che **non supera** D è quella a distanza $\approx D/2$ (la potenza di 2 più vicina).
3. Quindi il mio primo salto mi porta a metà strada verso l'obiettivo.
4. Dal nuovo nodo, la distanza rimanente è $D - D/2 = D/2$.
5. Ripeto il ragionamento: il prossimo salto coprirà metà della distanza rimanente ($D/4$).

Nota

Analogia: È esattamente come la **Ricerca Binaria** in un array ordinato. Invece di saltare all'indice medio dell'array, qui "saltiamo" fisicamente verso un nodo che è a metà strada. Poiché dimezziamo la distanza a ogni passo, in $\log_2(\text{Distanza})$ passi arriviamo a destinazione. Poiché la distanza massima è 2^m, servono m passi ($O(\log N)$).

6.7 L'Algoritmo di Lookup

```
n.find_successor(id):
    if id in (n, successor]:
        return successor
    else:
        n_prime = closest_preceding_node(id)
        return n_prime.find_successor(id)

n.closest_preceding_node(id):
    for j = m down to 1:
        if finger[j] in (n, id):
            return finger[j]
    return n
```

6.8 Join e Leave: Dinamicità

6.8.1 Quando un Nodo Entra (Join)

Un nuovo nodo n può unirsi conoscendo un nodo n' già nel sistema.

1. **Initialize**: Chiede a n' di trovare il suo successore. Inizializza i propri finger chiedendo al successore.
2. **Update**: I nodi esistenti devono aggiornare i propri puntatori (fatto tramite stabilization).
3. **Transfer**: Le chiavi nel range $(pred(n), n]$ passano dal vecchio successore a n.

Complessità Join: $O(\log^2 N)$ messaggi in media.

6.8.2 Stabilization: Il Cerchio che si Auto-Guarisce

Chord usa un protocollo periodico per gestire join concorrenti e fallimenti. È il "cuore pulsante" che mantiene l'anello unito.

Analogia del Mondo Reale: Chi ti stringe la mano?

Immagina di essere in un cerchio di persone che si tengono per mano. Tu stringi la mano destra a **B** (il tuo successore).

1. Ogni tanto chiedi a B: "Chi ti sta stringendo la mano sinistra? (ovvero, chi è il tuo predecessore?)".
2. Se B risponde: "È **X**!", e tu vedi che X è in piedi tra te e B, allora capisci che X si è appena unito al cerchio.
3. A quel punto lasci la mano di B e afferri quella di X. Ora X è il tuo nuovo successore.
4. Poi dici a X: "Ehi, guarda che ora sono io il tuo predecessore!".

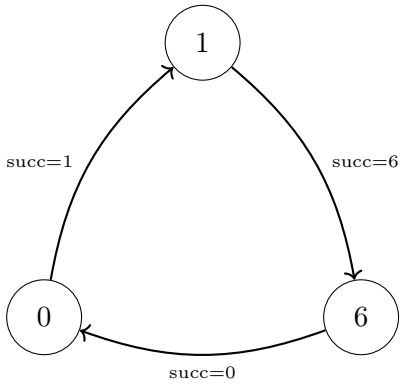
Così, un nodo alla volta, tutti si sistemano nella posizione corretta.

```
n.stabilize():
    x = successor.predecessor
    if x in (n, successor):
        successor = x
    successor.notify(n)

n.notify(n_prime):
    if (predecessor is nil) or (n_prime in (predecessor, n)):
        predecessor = n_prime
```

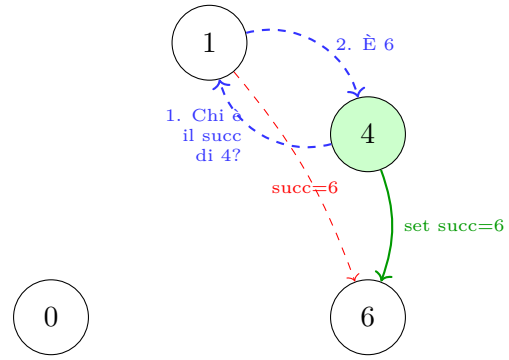
Logica delle slide: Se A fa stabilize verso B (*succ*), B risponde col suo *pred*. Se questo *pred* è tra A e B, A aggiorna il proprio *succ*. Infine A notifica B che potrebbe essere il suo nuovo *pred*.

6.8.3 Visual Walkthrough: Join Simplificato

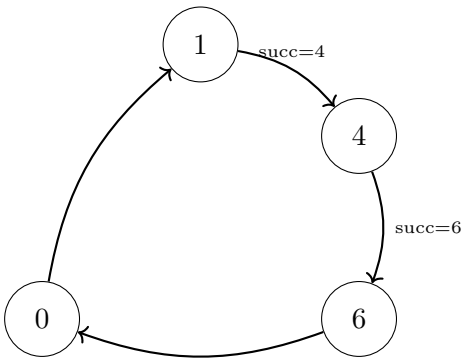
Nuovo nodo 4 entra tra 1 e 6.



1. Prima



2. Join



3. Stabilizzato

Dopo stabilize(), 1 scopre 4
e aggiorna il suo puntatore.

6.8.4 Stabilization: Mantenere la Coerenza

Per gestire join concorrenti e fallimenti, Chord usa un protocollo di **stabilizzazione** periodica.

Obiettivo: Assicurarsi che *successor* e *predecessor* siano corretti.

```
n.stabilize():
    x = successor.predecessor
    if (x esiste e x è tra n e successor):
        successor = x  // Ho trovato un nodo più vicino!

    send notify(n) to successor
```

```
n.notify(n'):
    if (predecessor is nil OR n' è tra predecessor e n):
        predecessor = n' // n' è il mio nuovo predecessore legittimo
```

Scenario Tipico:

1. A e B sono adiacenti.
2. X entra tra A e B. X imposta $succ(X) = B$.
3. X esegue `stabilize`: non fa nulla (B non ha pred o è A). X notifica B.
4. B riceve `notify(X)`: vede che X è tra A e B. Imposta $pred(B) = X$.
5. A esegue `stabilize`: chiede a B il pred. B risponde "X".
6. A vede che X è tra A e B. Imposta $succ(A) = X$.
7. **Risultato**: Catena A → X → B corretta!

6.8.5 Trace Table: Stabilization in Azione

Analizziamo la sequenza temporale di un nodo X che entra tra A e B:

Step	Chi	Azione	Successor	Predecessor
0	Sistem	Stato Iniziale	A→B, B→C	B.pred=A
1	X	Join(A)	A (entry point)	nil
2	X	FindSuccessor(X)	X→B	nil
3	X	Stabilize()	Chiede pred a B	nil
4	B	Risp: pred=A	-	-
5	X	Update	-	X vede A (lontano)
6	X	Notify(X) → B	-	-
7	B	Receive Notify	-	**B.pred = X**
8	A	Stabilize()	Chiede pred a B	-
9	B	Risp: pred=X	-	-
10	A	Update	**A.succ = X**	-

Nota

Notate come la correzione si "propaga". X si aggancia a B. Poi avvisa B. Poi A si accorge di X interrogando B.

Perché serve? I nodi possono entrare "nel mezzo" dell'anello. Stabilization aggiusta i puntatori.

6.8.6 Fix Fingers

Anche i finger devono essere aggiornati:

```
n.fix_fingers():
    scegli un finger casuale i
    finger[i] = find_successor(n + 2^(i-1))
```

6.9 Tolleranza ai Guasti in Chord

6.9.1 Lista di Successori

Per tollerare crash, invece di un solo successore, mantieni una **lista di r successori**.

Se il primo non risponde, prova il secondo, poi il terzo...

6.9.2 Replicazione dei Dati

Le chiavi possono essere replicate sui k successori:

- Se un nodo crasha, la copia è sul successore
- Più copie = più robustezza, ma più overhead

6.9.3 Node Leave: Uscita Volontaria vs Crash

Uscita volontaria (graceful leave):

1. Notifica il successore e il predecessore
2. Migra le chiavi al successore
3. I vicini aggiornano i loro puntatori

Crash (failure):

- I nodi scoprono il crash quando non rispondono
- Usano la lista di r successori come backup
- Se un finger non risponde, usano il finger precedente
- Le chiavi replicate sui successori sopravvivono

6.9.4 Il Problema del Churn

Il **Churn** indica la frequenza con cui i nodi entrano ed escono dal sistema (turnover).

- Se il churn è alto, la finger table diventa presto obsoleta.
- È necessario aumentare la frequenza di `stabilize()` e `fix_fingers()`.
- **Trade-off**: Più messaggi di manutenzione vs Rischio di lookup falliti.

6.10 Complessità di Chord

Operazione	**Complessità**
Lookup	$O(\log N)$ messaggi
Join	$O(\log^2 N)$ messaggi
Spazio per nodo	$O(\log N)$ (la finger table)
Chiavi per nodo	$\sim K/N$ (bilanciato)

6.11 Confronto con Altri Sistemi

Sistema	Lookup	Tipo	Uso
Napster	$O(1)$	Centralizzato	Chiuso nel 2001
Gnutella	$O(N)$	Flooding	LimeWire
Chord	$O(\log N)$	DHT	Accademico
Kademlia	$O(\log N)$	DHT	BitTorrent, IPFS

6.11.1 Applicazioni P2P Attuali (Legali)

Importante

P2P non è solo file sharing illegale!

- **Skype** (fino al 2017): usava P2P per le chiamate
- **Spotify** (fino al 2014): distribuzione musica P2P
- **Bitcoin/Blockchain**: rete P2P decentralizzata (come applicazione del P2P)
- **BitTorrent**: distribuzione legale software (es. Linux)
- **IPFS**: filesystem distribuito

Key Takeaways

1. **DHT**: Hash table distribuita su N nodi, lookup in $O(\log N)$
2. **Consistent Hashing**: Spazio circolare, chiave $\rightarrow$ primo nodo in senso orario
3. **Finger Table**: $finger[i] = succ(n + 2^{i-1})$ — salti esponenziali
4. **Lookup**: A ogni passo la distanza si dimezza $\rightarrow O(\log N)$
5. **Stabilization**: Mantiene corretti successor/predecessor durante join/leave

Errori Comuni da Evitare

- **Hashing normale vs Consistent**: Con hash normale, se N cambia quasi tutto si sposta!
- **Finger table size**: Solo $O(\log N)$ entry, non $O(N)$
- **Join senza stabilization**: I puntatori diventano inconsistenti, lookup fallisce
- **Chord vs Kademlia**: Kademlia (usato in BitTorrent) usa metrica XOR, simile complessità

6.12 Esercizi e Domande

6.12.1 Domande Concettuali

1. Perché il consistent hashing minimizza il movimento di chiavi?
2. Come è costruita la finger table? Quali distanze copre?
3. Perché la ricerca è $O(\log N)$ e non $O(N)$?
4. Come gestisce Chord il fallimento di un nodo?
5. Cos'è il churn e perché è un problema per le DHT?

6.12.2 Esercizi di Dimostrazione

1. **Cos'è una DHT**? Quali vantaggi offre rispetto a flooding e server centrale?
2. **Descrivi il Consistent Hashing**: perché aggiungere un nodo sposta solo K/N chiavi?
3. **Cos'è la Finger Table**? Dimostra perché il lookup è $O(\log N)$.
4. **Scrivi l'algoritmo di lookup** in Chord e fai un esempio passo-passo.
5. **Come funziona la stabilization**? Fai un esempio con un nodo che entra.
6. **Confronta Chord con Kademlia**: somiglianze e differenze.

Tracce di Soluzione

Soluzioni Domande Concettuali:

1. **Consistent Hashing**: Chiavi mappate su anello fisso. Nodo nuovo "ruba" solo chiavi tra lui e predecessore. Non cambia denominatore come hash modulare.
2. **Finger Table**: $finger[i] = succ(n + 2^{i-1})$. Copre distanze $1, 2, 4, 8, \ldots, 2^{m-1}$.
3. $O(\log N)$: A ogni salto la distanza si dimezza (come ricerca binaria). Da distanza D, salto a $D/2$, poi $D/4$, ... Log passi.
4. **Fallimento**: Lista di r successori. Se primo non risponde, uso secondo. Dati replicati su k successori.
5. **Churn**: Frequenza join/leave. Alto churn = finger obsolete = lookup falliti. Serve stabilization frequente.

Soluzioni Esercizi di Dimostrazione:

1. **DHT vantaggi**: Hash table distribuita. Flooding $O(N)$, server centrale SPF, DHT $O(\log N)$ scalabile.
2. K/N **chiavi**: Nodo nuovo a posizione p. Solo chiavi in $(pred(p), p]$ si spostano. In media K/N su K chiavi totali.
3. **Lookup dimostrazione**: A ogni hop, finger più lontano che non supera target dimezza distanza. Log passi per coprire 2^m.
4. **Lookup algoritmo**: Se id in $(n, succ]$, return succ. Else, inoltra a finger più lontano in (n, id). Distanza dimezza $\Rightarrow \log N$ hop.
5. **Stabilization esempio**: X entra tra A e B. X.succ = B. X notifica B. B.pred = X. A stabilize, chiede a B.pred, trova X. A.succ = X.
6. **Chord vs Kademlia**: Entrambi DHT $O(\log N)$. Chord usa anello, Kademlia usa metrica XOR (più simmetrica, lookup paralleli).

A Cheatsheet - Formule e Teoremi

Prerequisiti (Triennale)

Grafo	$G = (V, E)$ con $n = \|V\|$ nodi e $m = \|E\|$ archi. Diametro D = max distanza.
Albero	Grafo connesso senza cicli. Sempre $m = n - 1$. Unico cammino tra ogni coppia.
O, Ω, Θ	Upper bound, lower bound, tight bound. Ordine: $\log n < n < n \log n < n^2$.
BFS / DFS	Visita livello per livello (coda) / profondità prima (stack).
Dijkstra	Shortest path da sorgente singola: $O(n^2)$ o $O(m \log n)$ con heap.

Cap. 1-2: Broadcast e Spanning Tree

Algoritmo	Complessità	Come Funziona
Flooding	$2m - (n - 1)$ msg	Ogni nodo inoltra a tutti i vicini tranne il mittente. Semplice ma tanti messaggi.
Wake-up	$2m - (n - k)$ msg	Come flooding ma con k iniziatori invece di 1.
Tree Broadcast	$n - 1$ msg	Se hai già lo spanning tree, mandi solo sugli archi dell'albero. Ottimo!
SHOUT	$4m - 2n + 2$ msg	Flooding che costruisce ST. Ogni arco usato 2 volte (Q + risposta).
SHOUT+	$2m$ msg	SHOUT senza messaggi "NO" inutili.
DFS	$2m$ msg, $2m$ t	Visita sequenziale in profondità. Lento ma costruisce ST.
Saturation	$\leq 4n$ msg	Dalle foglie verso il centro. Veloce: tempo $O(D)$.

Cap. 3: Leader Election

Teorema di Angluin: In un anello anonimo (nodi senza ID unici) è **impossibile** eleggere un leader in modo deterministico. Serve almeno: ID unici O randomizzazione.

Algoritmo	Complessità	Come Funziona
AFAIC	$O(n^2)$ worst	"As Far As It Can": inoltra ID solo se è il max visto. Caso peggiore se IDs ordinati.
Controlled Distance	$O(n \log n)$	Fasi con distanze 2^k. Sopravvive solo chi è max locale. Ottimo per anelli!
ST + Saturation	$O(m + n)$	Prima costruisci ST, poi eleggi la radice con saturation. Per grafi generici.
Random (Las Vegas)	$O(n)$ atteso	Genera ID random, poi AFAIC. Funziona anche su anonimi.

Cap. 4: Routing

Problema: Costruire tabelle di routing distribuite senza conoscenza globale.

Equazione di Bellman-Ford: $d(x, y) = \min_v \{w(x, v) + d(v, y)\}$ (la distanza minima passa per un vicino)

Algoritmo	Complessità	Come Funziona / Problema
Map Gossip	$O(mn)$ msg	Ogni nodo diffonde la mappa completa. Semplice ma tanto spazio.
Distance Vector	$O(n^2m)$ msg	Scambio distanze coi vicini. **Problema**: count-to-infinity!
Link State	$O(mn)$ msg	Ogni nodo inonda info sui propri link. Nessun loop.

Count-to-Infinity: Quando un link si rompe, B e C si "rimbalzano" info obsolete, i costi crescono all'infinito. Soluzioni: Split Horizon, Poison Reverse, o usare Link State.

Cap. 5: Fault Tolerance

Due Teoremi di Impossibilità:

- **Two Generals**: Impossibile raggiungere consenso se i messaggi possono perdersi.
- **FLP**: Impossibile consenso deterministico in sistema asincrono con anche solo 1 crash.

Tipo Guasto	Limite	Cosa Significa
Crash Sincrono	$f < n$	Con timeout funziona sempre. Basta che resti almeno 1 nodo.
Crash Asincrono	$f < n/2$	Serve maggioranza onesta. Usa Ben-Or (randomizzato).
Byzantine	$f < n/3$	Nodi malevoli. Serve $n \geq 3f + 1$ per mascherare i bugiardi.

Cap. 6: DHT / Chord

Idea: Anello di 2^m posizioni. Ogni nodo/chiave ha un ID hash. $succ(k)$ = primo nodo con ID $\geq k$.

Finger Table: $finger[i] = succ(n + 2^{i-1})$. Salti esponenziali per lookup in $O(\log N)$.

Operazione	Complessità	Spiegazione
Lookup	$O(\log N)$	Segui i finger, dimezzi la distanza a ogni hop.
Join	$O(\log^2 N)$	Trova il tuo posto + aggiorna finger table.
Spazio/nodo	$O(\log N)$	Solo m entries nella finger table.

Quick Decision Guide

Devo eleggere un leader su un anello?	→ Controlled Distance
Devo eleggere un leader su grafo generico?	→ ST + Saturation
Devo fare routing senza loop?	→ Link State
Consenso in sistema asincrono con crash?	→ Ben-Or (probabilistico)
Lookup efficiente in rete P2P?	→ Chord

Notazione: n nodi, m archi, D diametro, f guasti, N dimensione DHT

Buona fortuna per l'esame!

B Fondamenti e Teoria della Complessità

Prerequisiti

- Conoscenza base di algoritmi (ordinamento, ricerca)
- Nozioni di base sui grafi (nodi, archi, cammini)
- Familiarità con la notazione matematica di base

In questo capitolo costruiremo le basi teoriche per capire quando un problema è "facile" o "difficile" da risolvere, e cosa fare quando ci troviamo di fronte a problemi intrattabili.

B.1 Problemi Computazionali

B.1.1 Cos'è un problema computazionale?

Prima di parlare di algoritmi, dobbiamo capire cosa stiamo cercando di risolvere.

Definizione: Problema Computazionale

Un **problema computazionale** è una domanda generale che dipende da alcuni parametri. Formalmente, è una funzione:

$$\Pi : I \to S$$

dove:

- I = insieme di tutti i possibili **input** (le "domande" che possiamo fare)
- S = insieme di tutte le possibili **soluzioni** (le "risposte" che cerchiamo)

Esempio: Ordinamento

Consideriamo il problema dell'ordinamento:

- **Domanda generale**: "Dato un insieme di numeri, restituirli in ordine crescente"
- **Input** I: tutti i possibili insiemi di numeri (es. 3, 1, 4, 7, 2, 5, ...)
- **Output** S: le corrispondenti sequenze ordinate (es. [1, 3, 4], [2, 7], [5], ...)

Quando fissiamo l'input a 3, 1, 4, 1, 5, otteniamo un'**istanza** del problema.

Nota

La distinzione tra **problema** e **istanza** è fondamentale: un algoritmo deve funzionare per *tutte* le istanze di un problema, non solo per alcune!

B.1.2 Classificazione dei Problemi

I problemi si dividono in tre categorie a seconda di cosa ci viene chiesto:

Tipo	Cosa chiede	Esempio
Decisionale	Una risposta SI/NO	"Il numero n è primo?"
Ricerca	Trovare *una* soluzione che soddisfa certi criteri	"Trova un ordinamento dei numeri"
Ottimizzazione	Trovare la soluzione *migliore* secondo un criterio	"Trova il cammino più breve tra A e B"

Analogia del Mondo Reale: Il Cuoco e la Ricetta

Immagina un **Problema** come il compito di "fare una torta".

- L'**Input** sono gli ingredienti disponibili.
- L'**Algoritmo** è la ricetta: una serie finita di passi precisi.
- Un problema **Decisionale** è: "Ho abbastanza farina?" (Sì/No).
- Un problema di **Ottimizzazione** è: "Qual è la torta più buona che posso fare con questi ingredienti?"

Perché i problemi decisionali sono importanti?

Potresti pensare che i problemi decisionali siano "più facili" perché richiedono solo SI/NO. In realtà:

Attenzione

Se un problema decisionale è difficile, il corrispondente problema di ottimizzazione è *almeno* altrettanto difficile!

Esempio: Se è difficile rispondere "Esiste un cammino da A a B di lunghezza $\leq k$?", sarà ancora più difficile trovare il cammino più corto in assoluto.

B.2 Algoritmi: Cosa Sono e Cosa Vogliamo da Loro

B.2.1 Definizione Precisa

Definizione: Algoritmo

Un **algoritmo** è una procedura che soddisfa quattro proprietà:

1. **Finitezza**: È composto da un numero finito di istruzioni
2. **Non ambiguità**: Ogni passo è precisamente definito (nessun "fai qualcosa di sensato")
3. **Eseguibilità**: Ogni passo può essere effettivamente realizzato
4. **Terminazione**: L'esecuzione termina sempre in tempo finito

B.2.2 Correttezza

Un algoritmo è **corretto** se, per *ogni* possibile input, produce l'output corretto.

Attenzione

Attenzione: un algoritmo che funziona nel 99% dei casi *non è corretto*! In informatica, "quasi sempre" non basta.

Se un algoritmo non è corretto, può:

- Non terminare mai (loop infinito)
- Terminare con una risposta sbagliata

B.2.3 Efficienza: Perché la Velocità Conta

Avere un algoritmo corretto non basta: deve anche essere **veloce**. Ma cosa significa "veloce"?

Il problema con i secondi: Misurare il tempo in secondi dipende dal computer, dal linguaggio, da chi scrive il codice... vogliamo qualcosa di più universale.

Soluzione: Contiamo il numero di **operazioni elementari** (confronti, somme, accessi in memoria) in funzione della dimensione dell'input n.

Esempio:

Un algoritmo che fa $3n^2+5n+7$ operazioni su un input di dimensione n è "più lento" di uno che ne fa $10n$ operazioni, perché al crescere di n, il primo cresce come n^2 mentre il secondo cresce solo come n.

B.3 Notazione Asintotica: Il Linguaggio dell'Efficienza

Per parlare di efficienza in modo preciso, usiamo la **notazione asintotica**.

B.3.1 L'idea intuitiva

Quando n è molto grande, i termini di ordine inferiore e le costanti moltiplicative diventano irrilevanti:

$$f(n) = 3n^2 + 100n + 1000000 \approx n^2 \text{ per } n \text{ grande}$$

B.3.2 Le tre notazioni

Notazione		Intuizione	Definizione Formale
$f(n) = O(g(n))$	$\Rightarrow$	"f cresce *al massimo* come g"	$\exists c > 0, n_0$ t.c. $\forall n \geq n_0$: $f(n) \leq c \cdot g(n)$
$f(n) = \Omega(g(n))$	$\Rightarrow$	"f cresce *almeno* come g"	$\exists c > 0, n_0$ t.c. $\forall n \geq n_0$: $f(n) \geq c \cdot g(n)$
$f(n) = \Theta(g(n))$	$\Rightarrow$	"f cresce *esattamente* come g"	$f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$

Esempio: Capire la notazione O

$f(n) = 3n^2 + 5n = O(n^2)$ perché:

- Scegliamo $c = 4$ e $n_0 = 5$
- Per ogni $n \geq 5$: $3n^2 + 5n \leq 3n^2 + n \cdot n = 4n^2$
- Quindi $f(n) \leq 4 \cdot n^2$ per n abbastanza grande ✓

B.3.3 Gerarchia delle complessità

Dal più veloce al più lento:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Importante

La differenza tra $O(n^2)$ e $O(2^n)$ è **enorme**!
Con $n = 100$:

- $n^2 = 10.000$ operazioni $\rightarrow$ microsecondi
- $2^n = 10^{30}$ operazioni $\rightarrow$ miliardi di anni!

B.4 Classe P: I Problemi "Facili" (Risolvibili)

Questi sono i problemi che i computer riescono a risolvere velocemente.

Definizione: Classe P (Tempo Polinomiale)

Un problema è in **P** se esiste un algoritmo che lo risolve in un tempo ragionevole (polinomiale, come $O(n^k)$).

Cosa significa in pratica: All'aumentare dei dati in ingresso, il tempo di calcolo cresce in modo gestibile. extbfEsempi: Ordinare un elenco telefonico, trovare il percorso più breve su Google Maps (Dijkstra).

B.5 Classe NP: I Problemi "Verificabili"

Qui c'è il primo grande equivoco: **NP non significa "Non Polinomiale"**! Significa *Nondeterministic Polynomial time*, ma la definizione pratica è molto più semplice.

Definizione: Classe NP

Un problema è in **NP** se, data una soluzione proposta, possiamo **verificare velocemente** se è corretta.

La differenza con P: In P sai *trovare* la soluzione velocemente. In NP sai *controllare* la soluzione velocemente se qualcun altro te la fornisce.

Analogia del Mondo Reale: Il Sudoku

- **Trovare la soluzione** (Risolvere): È difficile. Se ti do una griglia vuota, potresti metterci ore a riempirla.
- **Verificare la soluzione** (NP): Se ti do una griglia già compilata e ti chiedo "è giusta?", ci metti un attimo. Controlli solo che non ci siano numeri ripetuti.

Nota: Tutti i problemi P sono anche in NP (se sai risolverlo, sai anche verificarlo). La grande domanda è se vale il contrario ($P = NP$?).

B.6 La Chiave di Volta: La Riduzione ($A \leq_p B$)

Per capire i problemi più difficili, dobbiamo capire la **Riduzione**.

Idea: "Se avessi una macchina magica che sa risolvere B, potrei usarla per risolvere anche A?"

Definizione: Riduzione

Dire che A si riduce a B ($A \leq_p B$) significa che posso trasformare ogni istanza del problema A in un'istanza del problema B in tempo polinomiale.

Attenzione

COME NON FARSI FREGARE DALLA RIDUZIONE ($A \leq_p B$)

La confusione nasce sempre dalla direzione. Memorizza questa analogia:

L'Analogia del Risolutore Universale: Immagina che il problema B sia "Disinnescare una bomba nucleare". Immagina che il problema A sia "Accendere un fiammifero".

Se io ti dico: "Guarda, se sai disinnescare una bomba (B), allora sicuramente sai anche accendere un fiammifero (A)", ha senso. Sto dicendo che B è **almeno tanto difficile quanto** A.

$$\text{Fiammifero} \leq_p \text{Bomba}$$

L'Errore Classico: Dire "Riduco il mio problema MY a SAT" ($MY \leq_p SAT$). Questo significa: "Se so risolvere SAT (Bomba), so risolvere MY". Questo ci dice solo che MY non è *più difficile* di SAT. Ma potrebbe essere facilissimo (un fiammifero)! Per dimostrare che MY è difficile, devi fare il contrario: $SAT \leq_p MY$ ("Se so risolvere MY, allora so risolvere persino SAT!").

Conseguenza Logica: Se $A \leq_p B$, allora B è **almeno tanto difficile quanto** A. È come dire: "Se sai guidare un tir (B), sai sicuramente guidare anche una panda (A)". Il problema B include la difficoltà di A.

B.7 NP-Hard vs NP-Completo: Qual è la differenza?

Questa è la domanda da esame per eccellenza. Spesso si usano come sinonimi, ma **non lo sono**.

B.7.1 NP-Hard: I "Cattivi" (Hardest)

Un problema H è **NP-Hard** se è *almeno* difficile quanto il problema più difficile di NP.

- **Definizione**: Ogni problema $L \in NP$ si riduce a H ($L \leq_p H$).
- **Caratteristica Chiave**: Non è richiesto che H sia in NP. Potrebbe essere molto peggio!
- **Esempio**: Il problema dell'arresto (Halting Problem) è NP-Hard (posso ridurre SAT a esso), ma non è in NP (non è nemmeno decidibile!).
- **Esempio TSP**: "Trova il tour di costo minimo assoluto". È NP-Hard. Se ti do una soluzione, non puoi verificare in tempo polinomiale se è davvero la *minima* (dovresti confrontarla con tutte le altre). Quindi non è in NP.

B.7.2 NP-Completo: L'Elite (Hardest ∩ NP)

Un problema C è **NP-Completo** se soddisfa due requisiti:

1. È **NP-Hard** (è difficile quanto tutti gli altri).
2. È in **NP** (se ti do la soluzione, la puoi verificare).

Nota

L'Analogia del Club:

- **NP**: È il club delle persone che sanno *leggere* velocemente una mappa.
- **NP-Hard**: Sono le persone capaci di *disegnare* la mappa più difficile del mondo.
- **NP-Completo**: Sono quei geni che sanno SIA disegnare la mappa difficile (Hard) SIA leggerla velocemente (NP). Sono l'intersezione perfetta.

Riassunto Visivo:

NP-Hard = "Almeno difficile quanto NP" (Limite inferiore)
NP-Completo = "Almeno difficile quanto NP" AND "Dentro NP" (Esattamente difficile quanto i peggiori di NP)

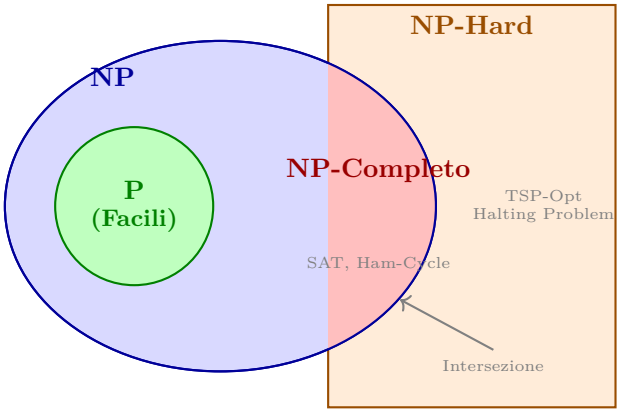


Figura 6: Mappa delle classi di complessità. **P** $\subset$ **NP**. **NP-Completo** = NP $\cap$ NP-Hard. Problemi come TSP-Ottimizzazione sono NP-Hard ma *non* in NP.

B.8 Algoritmi di Approssimazione: "Amazon non può aspettare"

Quando un problema è NP-Hard (come trovare il percorso perfetto per 100 corrieri), trovare la soluzione ottima richiederebbe secoli. Ma nel mondo reale, i camion devono partire tra 10 minuti.

Soluzione Pragmatica: Non cerchiamo la perfezione ($\alpha = 1$), ma un compromesso garantito.

B.8.1 L'idea (α-Approssimazione)

Un algoritmo si dice α-approssimante se:

- È veloce (Polinomiale).
- Garantisce che l'errore non supera mai un fattore α.

$$Costo(Mio) \leq \alpha \cdot Costo(Ottimo)$$

Esempio: $\alpha = 2$ significa "Al massimo paghi il doppio, mai di più". $\alpha = 1.5$ significa "Al massimo paghi il 50% in più".

B.8.2 L'Algoritmo di Christofides: Guida Visiva Definitiva

L'algoritmo di Christofides per il TSP Metrico è famoso per il suo rapporto di approssimazione $\alpha = 3/2$. Questo significa: **Non sbaglieremo mai per più del 50% rispetto alla soluzione perfetta.**

Ma perché 1.5? Da dove esce questo numero? Il costo totale è la somma di due componenti:

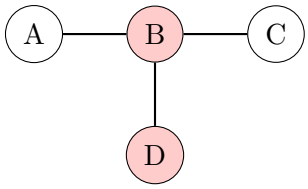
$$\text{Costo Totale} = \underbrace{\text{Costo(MST)}}_{\leq 1 \cdot OPT} + \underbrace{\text{Costo(Matching)}}_{\leq 0.5 \cdot OPT}$$

Analizziamo e visualizziamo ogni componente.

Parte 1: La Base Solida (MST)

Partiamo dal Minimum Spanning Tree (T^*).

- **Costo**: Il costo dell'MST è sicuramente inferiore a quello del Tour Ottimo (OPT). Se prendi il tour ottimo (un anello) e rimuovi un arco, ottieni un cammino (che è un albero). L'MST è l'albero più leggero possibile, quindi pesa meno di quel cammino, e a fortiori meno del tour.
- **Problema**: L'MST ha dei "vicoli ciechi". I nodi con grado dispari (foglie o biforcazioni complesse) ci impediscono di fare un giro continuo senza ripassare sui nostri passi.



Analisi Gradi MST:
A: 1 (Dispari) → Vicolo cieco
B: 3 (Dispari) → Incrocio dispari
C: 1 (Dispari) → Vicolo cieco
D: 1 (Dispari) → Vicolo cieco
Impossibile fare un ciclo unico!

Figura 7: L'MST iniziale. I nodi evidenziati hanno grado dispari e rompono il ciclo.

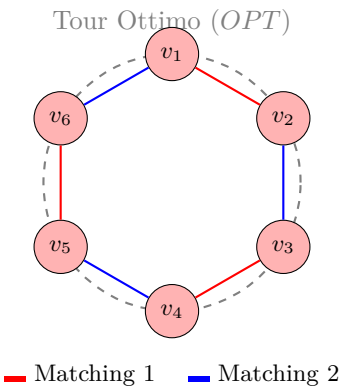
Parte 2: La Correzione (Minimum Weight Perfect Matching)

Questa è la parte magica. Dobbiamo "riparare" i nodi dispari aggiungendo archi finché non diventano pari. Per spendere poco, usiamo un **Minimum Weight Perfect Matching** solo sui nodi dispari.

Importante

Dimostrazione Visiva: Perché il Matching costa metà dell'Ottimo? Molti studenti imparano a memoria "$0.5 \cdot OPT$" senza capirlo. Vediamolo con i nostri occhi.

Immaginiamo il Tour Ottimo completo (il cerchio grigio tratteggiato sotto). Evidenziamo sul tour i nodi che nel nostro MST avevano grado dispari (nodi rossi).



Logica della Dimostrazione:

1. Prendi i nodi dispari lungo OPT e collegali in ordine (scorciatoie).
2. Ottieni un ciclo "misto" con archi alternati (Rosso + Blu).
3. Per disuguaglianza triangolare: Rosso + Blu $\leq$ Costo(OPT).
4. Quindi, il più leggero dei due (es. Blu) deve pesare $\leq 0.5 \cdot OPT$.
5. Il nostro Matching M^* sarà ancora più leggero (o uguale) del Blu!

Figura 8: Dimostrazione visiva del costo del Matching. Scomponendo il ciclo dei nodi dispari in due matching alternati (rosso e blu), almeno uno dei due pesa metà o meno del totale.
Il matching che calcoliamo noi (M^*) è il *migliore possibile*, quindi costerà sicuramente meno (o uguale) di quel matching Blu ipotetico ricavato dal tour ottimo.

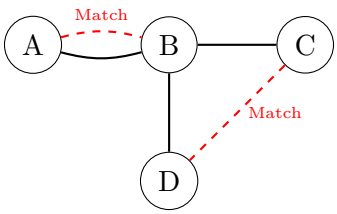
$$Cost(M^*) \leq Cost(\text{Matching Blu}) \leq \frac{1}{2} Cost(OPT)$$

Parte 3: Unione e Ciclo Euleriano

Ora sommiamo i grafi: $G' = MST + M^*$.

- I nodi che erano pari nell'MST restano pari (non toccati dal matching).
- I nodi che erano dispari nell'MST hanno ricevuto esattamente 1 arco dal matching $\rightarrow$ Ora sono pari!

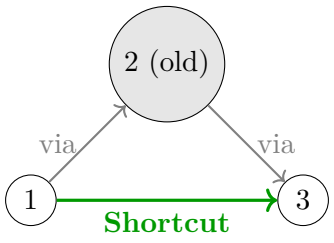
Tutti i gradi sono pari $\rightarrow$ Esiste un Ciclo Euleriano.



Unione: MST (Nero) + Matching (Rosso). Controlliamo B: Aveva 3 archi, ora ne ha 4. Pari! Il grafo è ora percorribile interamente.

Parte 4: Shortcutting (Il tocco finale)

Seguiamo il ciclo euleriano. Se ci porta in un nodo già visitato, prendiamo una scorciatoia. Grazie alla disuguaglianza triangolare, la scorciatoia (ipotenusa) non è mai più lunga della strada originale (cateti).



Saltare il nodo 2 riduce la distanza (o la lascia uguale). **Il costo non aumenta mai**.

B.8.3 Riassunto Finale del Costo

Abbiamo costruito la soluzione sommando due oggetti:

1. **MST**: Costa $\leq 1 \cdot OPT$.
2. **Matching**: Costa $\leq 0.5 \cdot OPT$ (come provato dalla figura del cerchio rosso/blu).

$$\text{Costo Christofides} \leq 1 \cdot OPT + 0.5 \cdot OPT = \mathbf{1.5 \cdot OPT}$$

Abbiamo dimostrato visivamente e logicamente che l'algoritmo non supera mai questa soglia.

B.8.4 Teoria Formale per l'Esame

Teorema: Lemma sul Costo del Matching Minimo

Sia V' l'insieme dei nodi di grado dispari nell'MST e sia Γ il ciclo che visita i nodi di V' nell'ordine in cui appaiono nel tour ottimo T_{OPT}. Allora:

$$cost(M^*) \leq \frac{cost(\Gamma)}{2} \leq \frac{cost(T_{OPT})}{2}$$

Dimostrazione. Consideriamo i nodi $V' = \{v_1, v_2, \ldots, v_{2k}\}$ nell'ordine in cui appaiono lungo T_{OPT} (il numero è sempre pari per il lemma delle strette di mano).
Il ciclo Γ che li connette in quest'ordine ha costo $\leq cost(T_{OPT})$ per la disuguaglianza triangolare (ogni arco (v_i, v_{i+1}) in Γ è un "shortcut" del cammino tra v_i e v_{i+1} in T_{OPT}).
Ora, Γ può essere partizionato in due matching perfetti:

- $M_1 = \{(v_1, v_2), (v_3, v_4), \ldots, (v_{2k-1}, v_{2k})\}$ (archi dispari)
- $M_2 = \{(v_2, v_3), (v_4, v_5), \ldots, (v_{2k}, v_1)\}$ (archi pari)

Poiché $cost(M_1) + cost(M_2) = cost(\Gamma)$, almeno uno tra M_1 e M_2 ha costo $\leq cost(\Gamma)/2$.
Quindi il matching minimo $M^* \leq \min(M_1, M_2) \leq cost(\Gamma)/2 \leq cost(T_{OPT})/2$. □

Analisi Formale del Fattore di Approssimazione:

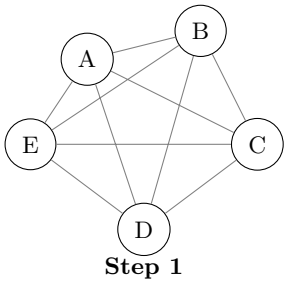
$$\begin{aligned} cost(Tour_{ALG}) &\leq cost(T^*) + cost(M^*) \\ &\leq cost(Tour_{OPT}) + \frac{cost(Tour_{OPT})}{2} \\ &= \frac{3}{2} \cdot cost(Tour_{OPT}) \end{aligned}$$

Quindi $\alpha = 3/2$ (garantiamo una soluzione che è al massimo il 50% peggiore dell'ottimo).

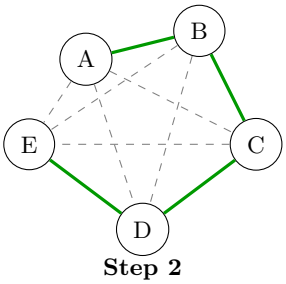
B.8.5 Visual Walkthrough: Christofides Step-by-Step

(Vedere Figura seguente per la sequenza grafica completa).

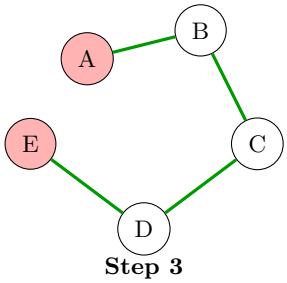
Esempio con 5 città (A, B, C, D, E)



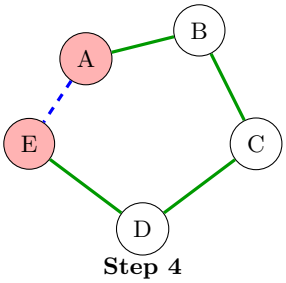
Step 1
Grafo completo
(tutte le distanze)



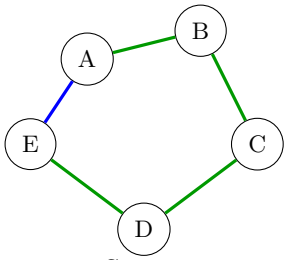
Step 2
MST (costo minimo)
Costo $\leq$ OPT



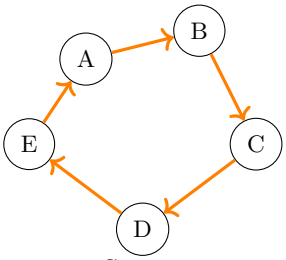
Step 3
Nodi dispari
A: grado 1, E: grado 1



Step 4
Matching A-E
Costo $\leq 0.5\cdot$ OPT



Step 5
Tutti gradi pari
Ciclo: A-B-C-D-E-A



Step 6
Tour Hamiltoniano
A→B→C→D→E→A

Analisi del costo: Tour $\leq MST + Matching \leq OPT + 0.5 \cdot OPT = \mathbf{1.5 \cdot OPT}$

Nota

Nota: In questo esempio il ciclo euleriano coincide con il tour hamiltoniano (nessun nodo ripetuto). In casi più complessi con più nodi dispari, il ciclo euleriano potrebbe passare per alcuni nodi più volte, e lo shortcutting salterebbe queste ripetizioni.

B.8.6 Tightness del Bound 3/2 di Christofides

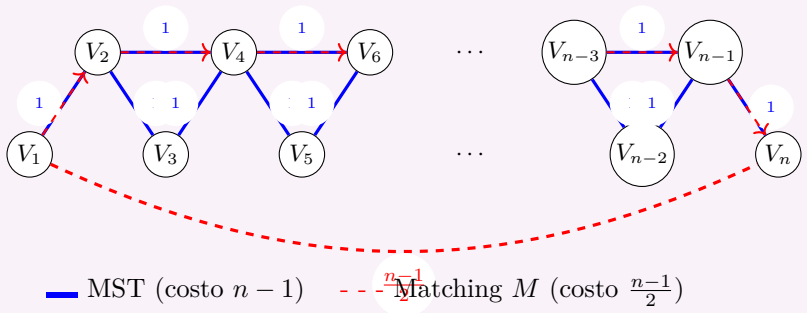
Il fattore di approssimazione $\alpha = 3/2$ è **tight**: esistono famiglie di istanze dove il rapporto $\frac{\text{Christofides}}{\text{OPT}}$ converge esattamente a $\frac{3}{2}$.
Definizione di tightness: Un bound α per un algoritmo di approssimazione è *tight* se:

$$\forall \varepsilon > 0, \exists \text{ istanza } I : \frac{\text{ALG}(I)}{\text{OPT}(I)} \geq \alpha - \varepsilon$$

Esempio: Costruzione: Catena di Triangoli (Bad Case)

Sia n dispari. Costruiamo un grafo $G = (V, E)$ con n nodi disposti su due livelli formando una catena di triangoli.
Struttura dei nodi:

- **Livello inferiore (dispari)**: $V_1, V_3, V_5, \ldots, V_n$
- **Livello superiore (pari)**: $V_2, V_4, V_6, \ldots, V_{n-1}$



Definizione delle distanze:

- Ogni arco diagonale e orizzontale nel disegno ha peso **1**
- L'arco diretto $V_1 \to V_n$ (linea inferiore lunga) ha peso $\frac{n-1}{2}$
- Tutte le altre distanze sono definite come cammini minimi (metrica del grafo)

Analisi Rigorosa:

1. **Minimum Spanning Tree (MST)**:

   L'MST connette tutti i nodi seguendo la struttura a zigzag dei triangoli. Ogni arco ha peso 1, e servono $n-1$ archi per connettere n nodi:

$$\boxed{w(MST) = n - 1}$$

2. **Nodi di grado dispari nell'MST**:

   Nell'MST a catena di triangoli:

   - V_1 (estremo sinistro): grado 1 — **dispari**
   - V_n (estremo destro): grado 1 — **dispari**
   - Tutti gli altri nodi: grado 2 — **pari**

   Insieme dei nodi dispari: $O = \{V_1, V_n\}$ (solo 2 nodi!)

3. **Matching Perfetto Minimo su O**:

   Con solo due nodi dispari, il matching è unico: l'arco (V_1, V_n).

$$\boxed{w(M) = \frac{n-1}{2}}$$

4. **Costo Tour Christofides**:

   Il multigrafo $MST \cup M$ ha tutti i nodi di grado pari. Il ciclo Euleriano attraversa tutti gli archi dell'MST (costo $n-1$) più l'arco del matching (costo $\frac{n-1}{2}$):

$$\boxed{w(\text{Christofides}) = w(MST) + w(M) = (n-1) + \frac{n-1}{2} = \frac{3(n-1)}{2} = \frac{3n}{2} - \frac{3}{2}}$$

5. **Tour Ottimo**:

   Il tour ottimo segue il "bordo esterno" della catena di triangoli:

$$V_1 \to V_2 \to V_4 \to V_6 \to \cdots \to V_{n-1} \to V_n \to V_{n-2} \to \cdots \to V_3 \to V_1$$

   Questo tour usa solo archi di peso 1, e ne servono esattamente n:

$$\boxed{w(OPT) = n}$$

Calcolo del Rapporto di Approssimazione:

$$\frac{w(\text{Christofides})}{w(OPT)} = \frac{\frac{3n}{2} + O(1)}{n} = \frac{3}{2} + O\left(\frac{1}{n}\right) \xrightarrow{n\to\infty} \boxed{\frac{3}{2}}$$

Importante

Perché questo esempio è "cattivo" per Christofides?
L'algoritmo è "ingannato" dalla struttura:

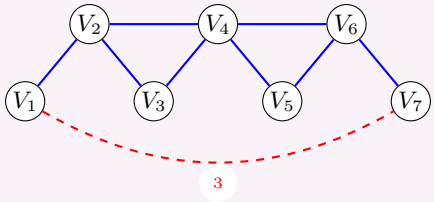
1. L'MST è quasi un cammino, con solo i due estremi di grado dispari
2. Il matching deve collegare V_1 a V_n, ma la distanza diretta è $\frac{n-1}{2}$ (non 1!)
3. Il tour ottimo invece "gira intorno" usando solo archi corti

Nota

Significato della Tightness:

1. L'analisi dell'algoritmo di Christofides è **ottimale**: non si può dimostrare un fattore migliore di 3/2 con la stessa tecnica di analisi.
2. Per ottenere un fattore $< 3/2$ servono **algoritmi strutturalmente diversi** (e infatti per decenni 3/2 è rimasto il miglior bound noto per TSP metrico).
3. Nel 2020, Karlin, Klein e Oveis Gharan hanno dimostrato un algoritmo con fattore $\frac{3}{2} - \varepsilon$ per un piccolo $\varepsilon > 0$, rompendo la barriera dopo 44 anni.

Esempio: Esempio Numerico: $n = 7$ (7 nodi)



- $w(MST) = 6$ (tutti archi di peso 1)
- Nodi dispari: $O = \{V_1, V_7\}$
- $w(M) = \frac{7-1}{2} = 3$ (arco V_1–V_7)
- $w(\text{Christofides}) = 6 + 3 = 9$
- $w(OPT) = 7$ (tour: $V_1 \to V_2 \to V_4 \to V_6 \to V_7 \to V_5 \to V_3 \to V_1$)
- Rapporto: $\frac{9}{7} \approx 1.286$ (converge a 1.5 per $n \to \infty$)

B.8.7 Vertex Cover con Approssimazione 2

Problema: Dato un grafo, trovare il minimo insieme di nodi che "copre" tutti gli archi (almeno un estremo di ogni arco è nell'insieme).
Algoritmo Greedy:

```
C = insieme vuoto
while esistono archi non coperti:
    scegli un arco (u, v) qualsiasi
    aggiungi sia u che v a C
    rimuovi tutti gli archi incidenti a u o v
return C
```

Perché funziona con $\alpha = 2$?
Gli archi scelti formano un **matching** (non condividono nodi). Se abbiamo scelto k archi:

- $|C| = 2k$ (abbiamo aggiunto 2 nodi per arco)
- L'ottimo deve coprire tutti questi k archi, quindi $|C_{OPT}| \geq k$
- Quindi $|C| = 2k \leq 2 \cdot |C_{OPT}|$

B.8.8 Tecnica Relax-and-Round ($\alpha = 2$)

Un approccio alternativo più generale basato sulla programmazione lineare.
Fase 1: Formulazione ILP

Variabili:	$X_v \in \{0, 1\}$ per ogni $v \in V$ $X_v = 1$ se $v \in$ cover, 0 altrimenti
Minimizza:	$\sum_v X_v$
Vincoli:	$X_u + X_v \geq 1$ per ogni arco $(u, v) \in E$

Fase 2: Rilassamento LP
Sostituisci $X_v \in \{0, 1\}$ con $X_v \in [0, 1]$. Ora è un LP, risolvibile in tempo polinomiale!
Fase 3: Arrotondamento
Data la soluzione ottima LP con valori X_v^*:

$$\hat{X}_v = \begin{cases} 1 & \text{se } X_v^* \geq 1/2 \\ 0 & \text{se } X_v^* < 1/2 \end{cases}$$

Nota

Perché il cover è valido?
Per ogni arco (u, v): $X_u^* + X_v^* \geq 1$ (vincolo LP).
Se un valore è $< 1/2$, l'altro deve essere $> 1/2$. Dopo l'arrotondamento, almeno uno è $1 \rightarrow$ arco coperto!

Analisi del fattore di approssimazione:
Per ogni v: $\hat{X}_v \leq 2 \cdot X_v^*$

- Se $\hat{X}_v = 0$: $0 \leq 2 \cdot X_v^*$ (vero sempre)
- Se $\hat{X}_v = 1$: $X_v^* \geq 1/2$, quindi $1 \leq 2 \cdot X_v^*$ ✓

Sommando: $\sum \hat{X}_v \leq 2 \cdot \sum X_v^*$

$$cost(approx) \leq 2 \cdot cost(opt_{LP}) \leq 2 \cdot cost(opt_{ILP}) \implies \alpha = 2$$

B.8.9 Tightness dell'Analisi

Il bound $\alpha = 2$ è **tight** per entrambi gli algoritmi.

Esempio: Caso peggiore: Grafo Bipartito Completo

Per $K_{n/2,n/2}$:

- **Soluzione ottima**: $n/2$ nodi (tutti quelli di una parte)
- **Greedy può selezionare**: n nodi (tutti!)
- **Rapporto**: $n/(n/2) = 2$

B.9 Riepilogo del Capitolo

Concetto	Intuizione
Classe P	Problemi risolvibili "velocemente" (tempo polinomiale)
Classe NP	Problemi verificabili "velocemente"
NP-Completo	I problemi più difficili di NP
NP-Hard	Almeno difficili quanto NP-completo
Riduzione $A \leq_p B$	"Se so risolvere B, so risolvere anche A"
Approssimazione	Soluzioni "abbastanza buone" in tempo ragionevole

Key Takeaways

1. **P**: problemi risolvibili in tempo polinomiale ("facili")
2. **NP**: problemi verificabili in tempo polinomiale (certificato + verificatore)
3. **NP-Completo**: i "più difficili" di NP — se uno è in P, tutti sono in P
4. **Approssimazione**: quando l'ottimo è NP-hard, accontentati di α-approssimazione
5. **Christofides**: TSP con $\alpha = 3/2$ (il migliore conosciuto per TSP metrico)

Errori Comuni da Evitare

- **NP $\neq$ "Non Polinomiale"**: NP sta per *Nondeterministic Polynomial*
- **P $\subseteq$ NP**: Ogni problema in P è anche in NP (non sono disgiunti!)
- **NP-hard $\neq$ NP-completo**: NP-hard può non essere in NP (es. TSP ottimizzazione)
- **α-approssimazione**: Non significa "errore del α%", ma rapporto $\leq \alpha$

B.10 Esercizi e Domande

B.10.1 Domande Concettuali

1. Spiega con parole tue la differenza tra P e NP.
2. Perché la classe NP non significa "Non Polinomiale"?
3. Cosa succederebbe alla crittografia se si dimostrasse P = NP?
4. L'algoritmo di Christofides garantisce il tour ottimo? Perché sì/no?
5. Se un problema è NP-hard ma non NP-completo, cosa significa?
6. Cos'è un certificato? Fai un esempio con Ciclo Hamiltoniano.
7. Cosa significa che un'analisi è "tight"?

B.10.2 Esercizi di Dimostrazione

1. **Definisci formalmente la classe NP** e dimostra che $P \subseteq NP$.
2. **Dimostra che TSP è NP-hard** usando riduzione da Ciclo Hamiltoniano.
3. **Descrivi l'algoritmo di Christofides** e dimostra rigorosamente il fattore di approssimazione 3/2.

4. **Spiega la tecnica Relax-and-Round** applicata a Vertex Cover e dimostra che $\alpha = 2$.

Tracce di Soluzione

Soluzioni Domande Concettuali:

1. **P vs NP**: P = risolvibile in tempo polinomiale. NP = verificabile in tempo polinomiale.
2. **NP non è "Non Polinomiale"**: NP sta per Nondeterministic Polynomial. P $\subseteq$ NP!
3. **P = NP e crittografia**: Molti sistemi crittografici (RSA) si basano sull'assunzione P $\neq$ NP. Se P = NP, sarebbero vulnerabili.
4. **Christofides ottimo?**: No! Garantisce solo fattore 1.5. Il tour potrebbe essere subottimo.
5. **NP-hard non NP-completo**: Il problema potrebbe non essere in NP (es. TSP ottimizzazione non è un problema di decisione).
6. **Certificato**: Una "prova" verificabile in tempo polinomiale che la risposta è SI. Per Ham. Cycle: la sequenza di nodi del ciclo.
7. **Tight**: Esiste un'istanza dove l'algoritmo raggiunge esattamente il bound peggiore.

Soluzioni Esercizi di Dimostrazione:

1. **Definizione NP**: Classe dei problemi verificabili in tempo polinomiale. **P $\subseteq$ NP**: Se risolvibile in tempo polinomiale, possiamo verificare risolvendo.
2. **TSP NP-hard**: Si riduce HAM-CYCLE a TSP assegnando peso 1 agli archi esistenti e peso 2 agli altri. Tour di costo $\leq n$ esiste $\Leftrightarrow$ esiste ciclo hamiltoniano.
3. **Christofides**: 1. MST ($\leq$ OPT), 2. Matching minimo su nodi dispari ($\leq 0.5\cdot$ OPT), 3. Ciclo Euleriano, 4. Shortcut. Totale: $\leq 1.5\cdot$ OPT.
4. **Relax-and-Round**: Rilassare $X_v \in \{0,1\}$ a $X_v \in [0,1]$, risolvere LP, arrotondare (1 se ≥ 0.5, 0 altrimenti). $\hat{X}_v \leq 2X_v^* \Rightarrow \alpha = 2$.